



HyperPanel OS: Time-line Oriented and Hardware Driven Operating System Architecture

J. Y. Astier

jean-yves.astier@hyperpanel.fr

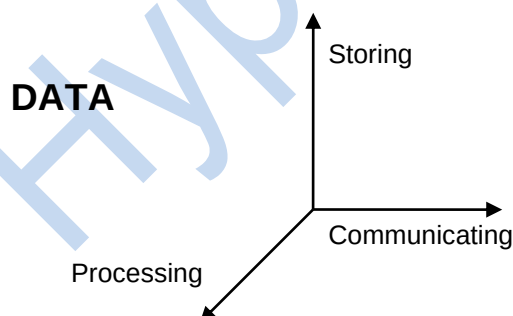
Summary

Current computer hardware architectures are optimized in order to achieve maximum data throughputs and drastically reduce power consumption. However, many Operating systems like Windows or Linux are far away to provide the performances and the energy savings the underlying hardware is able of. This is a consequence of the very incorrect assumption that modern CPU's power is such that a full support of other available hardware would be useless. We introduce the concept of "Time-line Oriented Design Principle". We detail what are the three fundamentals hardware blocks to be used for IO's. We explain how software activity switches may also rely on this same "hardware trinity" and the huge benefits of doing so. We detail what are Finite State Machines. Finally, we will develop the concept of a two container's time-line oriented and hardware driven Operating System Architecture using Finite State Machines that allows for spectacular IO performances increase, huge energy savings and LAN spread OS.

Introduction – Computer and Operating System

Computer

At a first glance, it may seem obvious to state that a "computer" is a machine intended to execute "computations". But for the sake of clarity of the subject we intend to cover here, we have to be more accurate. A "computer" is a machine intended to automatically handle data. A "computer" is able to store data, to communicate data and to process data. Any particular "computer" may be characterized by evaluating its capacities and performances against those three dimensions:



A computer handles **DATA**.

It **stores** data, **communicates** data and **processes** data.

We remark that the English word "computer", as a derivative of the "compute" verb, is very limitative. "Computing" is only a subset of "Processing" and also it does not take in account two dimensions out of the three we introduced here. The French language is using the word "ordinateur": in 1955, the French subsidiary of IBM directors felt that simply translating "computer" to the French "calculateur" was too much restrictive, considering the possibilities of those machines. A philology teacher, Jacques Perret, proposed to derive "from the "ordinator" latin word (the one who organizes) the "ordinateur" neologism. One may feel that such a consideration really does not matters and that the writer is only digressing. But for sure, many modern Operating Systems, if not all, are amazingly weak and slow regarding the "storing data" and "communicating data" dimensions, compared to what bare hardware is able of.

Operating System and Tasks

We will not dare to give a formal definition of the “Operating System” term, being aware that disputes may arise about that subject. We will rather recall what an Operating System does, using the three hardware dimensions previously identified:



This view of an Operating System is often considered to be universal. However, we insist on the fact this is an external point of view of provided services. We will see later on that a fundamental issue is deriving from “task” and “scheduling tasks” concepts. A key point is to allow for a more general concept than the “task scheduling” concept, even though we may hide it from the user sights. We will call it “activity switching”. We humbly beg our reader to give us the time to explain what is the fundamental issue, how to solve it and detail the profits of this solution.

Two unsolved challenges

The Challenge of I/O Science

Historians generally consider that the first machine deserving to be called a computer is the ENIAC (Electronic Numerical Integrator and Computer). In 1948, in a famous article called *The Eniac*, two engineers who took part in the project, J.G. Brainerd and T.K. Sharpless, wrote the following: “Current developments in large-scale general-purpose digital computing devices are devoted to a considerable extent to obtaining speedier input and output mechanisms.”

In 1999, in an article entitled *Introduction to “The Eniac”* (referencing the 1948 article *The Eniac*), W. Burks (one of the main designers of the electronics of the ENIAC) and E. Davidson wrote:

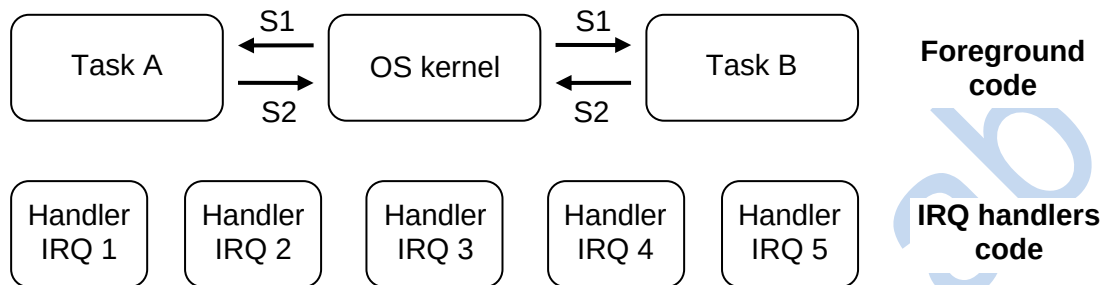
“... and once again, as with the ENIAC, computation rate is not the performance-limiting factor, rather it is still the communication, the I/O, the setup for the computation. It seems that communication science may be at the heart of the problem after all.”

However, on the hardware point of view, all that is needed to obtain the maximum possible I/O throughputs was already available with all the computers of the late 70's. This hardware is usually named “The Interrupt Request sub-system” and “DMA channels”. The first micro-computers IBM PC 5150 of 1981 only had a single Intel 8088 processor running at 4.77 MHz, but however a full “Interrupt Request sub-system” hardware was available. Unfortunately, the operating systems like MS/DOS did not use it at all. Only with OS/2 (1987) and Windows NT (1993) decent use of the hardware “Interrupt Request Systems” was implemented but they were still far from the true capabilities of the hardware devices, because they still partly used all the available interrupt sub-system capabilities. Today, Linux use of “Interrupt Request Systems” is significantly inferior to OS/2, and so I/O performances are “light-years” away from what the hardware (disks, telecommunication lines) could do.

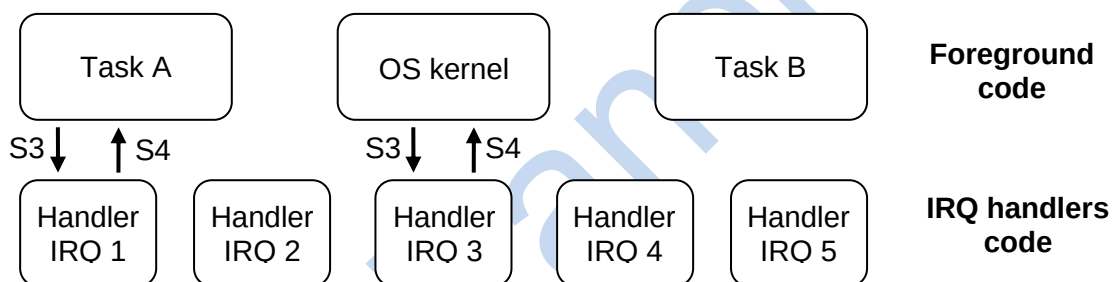
Here a fundamental remark has to be made. If one can find a considerable amount of literature dedicated to processing, both on the hardware and software point of view, very little is available when it comes to I/O's, despite the fact that modern hand-held devices, data servers, IoT devices, TV sets, routers and so on are much more I/O systems than computing systems. We are going to focus here on I/O's, including the much neglected subject of Interrupts Request handling.

The Challenge of Activities Switching

Unlike the human brain, a processor can go from an activity to another one at a theoretical null cost: One machine instruction may belong to one activity and the next instruction to another activity, thanks to hardware blocks known as “interrupt requests handling”. Some care has to be taken in order to avoid jumping to an incorrect address and for that reason activity switching is always an Operating System devoted operation. Most Operating Systems distinguish two categories of activities switches. The first category includes OS kernel to user task (S1) and user task to OS kernel (S2).



The second category of activities switches is foreground code to interrupt request handler (S3) and interrupt request handler to foreground code (S4).



We depicted S1 and S2 with horizontal arrows and S3 and S4 with vertical arrows because in all existing OS architectures those two categories of activity switches are implicitly “orthogonal”, uncorrelated. The OS function that switches uses tasks (S1, S2), aka the “scheduler”, is absolutely not aware of the IRQ dispatcher/manager, that handles (S3, S4). A second noteworthy characteristic of existing OS architectures is the fact that switch S1 and S2 are rather costly in term of needed CPU whilst S3 and S4 are not.

At that point of the text, the writer has to take a deep breath because being the carrier of a very bad news is never easy. This “orthogonality” is a consequence of multi-users operating systems, a “user” being a human being, that have absolutely uncorrelated activities. But with consumer electronic products, with devices such as TV sets, smart-phones or routers, there is only one user at a time (or none) and all the device internal activities are all correlated. To be more specific, each internal software activity, considering the whole software, so OS and applicative tasks, is a suite, a sequence of consecutive elementary pieces of code that are executed at different execution levels. This implies that, in order to achieve the best overall performances, each elementary piece of code, whatever its execution level is, must immediately follow its preceding with no delay or latency. So, the bad news is that we should get rid of what is usually named a “scheduler” and find a better way to handle activity switches: this conceptually exists and is usually named “Finite State Machines”. <Important>

The reader may be shocked by such a statement. A scheduler is usually considered to be the “heart” of an operating system up to the point that many small OS's are reduced to a sole scheduler. The dream of many developer's is to code a scheduler. So, a first mandatory point is to demonstrate that issues raised by “scheduling” are so deep and fundamental that there is no way to “save” the scheduler. And the second point is that until now, no theoretical research has been done about OS architecture using FSM. The present paper will cover both those points.

Regarding the scheduler, the main issue is that the count of machine instructions it executes for each switch is always prohibitive. We will demonstrate it by describing typical “scenarios” with timings numerical values even this may be difficult to follow. The FSM OS architecture, even it took decades to be discovered, can be easily depicted with some carefully chosen diagrams.

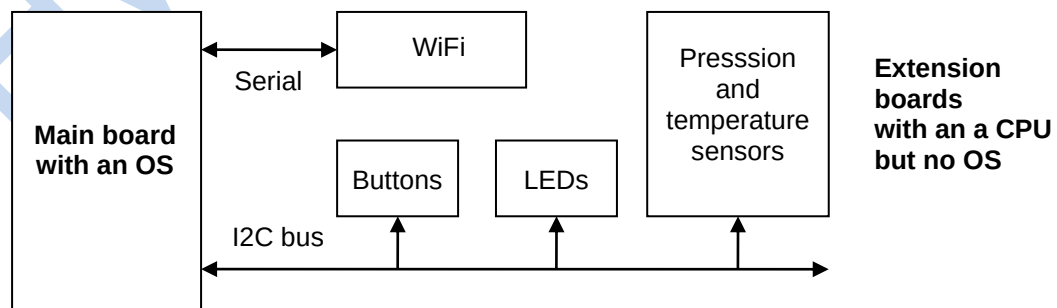
Latency is the issue and not processing time

A very simplistic and common assumption about computers is that increasing processing power will solve all the performances issues: it is not true when it comes to I/O processing and activities switching. Here we are going to explain the reason why with few words:

- 1) Firstly, all the devices like a disk, an Ethernet controller, an UART, an I2C bus controller have no correlation with the CPU speed. A 5400 rpm. hard disk has its own performances, a 9600 bauds asynchronous line can transmit 1 000 characters per second whatever the CPU processing power is
- 2) As a consequence, the only way to reach the maximum level of performance of a disk or an asynchronous line is to reduce its "idle" time. In other words, when the hardware detects that one transfer has been completed, the software has to start the next transfer as soon as possible, especially when the transfer size becomes small. With an UART (the controller of one asynchronous line) the transfer size is one byte only. When the UART FIFO size is only one byte, the software has a time window of the duration of the sending of one byte, so 1 millisecond at 9600 bauds, to provide to the UART hardware the next byte to be sent. It has to be noted that the number of instructions the CPU has to execute is very few, so an execution duration should be far under 1 millisecond, typically 1 microsecond.
- 3) So, the processing time to start the next transfer is most of the time negligible and far below the transfer time. So, increasing the processing power will just reduce a duration that is negligible. Reducing the latency between the end of one transfer and the start of the next one is the only thing that matters.

At this point, one could argue that any processing power increase will decrease in a linear manner the latency between the end of one transfer and the start of the next one, and so that in the end CPU power is the only point that matters. But this is an absolutely incorrect way of thinking as soon as we have any software mechanism that does processing time quantization. If for example we have a 10 milliseconds time quantum, independent from the CPU speed, then we would have an average latency of 5 milliseconds whatever the processing power is. And also, increasing processing power by tenths means increasing the electrical consumption by tenths just because of an ineffective OS architecture and indeed this should be avoided.

The consequences of latencies introduced by inadequate Operating system architecture have a huge impact on many aspects but most visible one is a spectacular increase of hardware cost and electrical consumption, with side effects such as oversized batteries that are however quickly emptied, simply because of the multiplication of big or small CPUs and then consequently of the needed RAM. We could give a multitude of examples but here we will limit to a simple one. Small IOT's boards, such as Arduino or STM32xxxx, are coming with a lot of Operating Systems of different architectures (Linux, RTOS's, simple I/O monitor with no scheduler, simple scheduler with lightweight tasks...) But whatever the chosen OS, very basic things are working rather poorly: Debouncing a pushbutton, having a led blinking at a slow rate, receiving infrared RCU messages, using an I2C EEPROM, ... Those issues are such widely encountered that myriad of tiny extension boards is available, each with an additional CPU and no OS, in order to be able to correctly manage a sole I/O link or kind of device: push-buttons, accelerometer, temperature sensor, blinking LEDs. Also, in case of telecommunication modems, such as WiFi or Ethernet, the extension board CPU hosts the telecommunication stack, including TCP or MQTT. All the tiny extension boards are connected to the main board using UART's or I2C.



So, despite the fact the main board CPU has enough processing power, all the needed IO pins and a very sophisticated IRQ subsystem too, we have 4 more additional CPUs just because the main board OS does not provide appropriate activities switching capabilities, inducing too longer latencies.

Time-line analysis and Latencies identification

The Time-line Oriented Analysis and Design Principle

At application level, software codes are always designed as if they were “alone” to be executed by the computer. One main function of an Operating System is to be responsible of the management of this “as if” model: Whatever the number of user applications the computer is running, as long as they do not share user-defined common resources such as files, the Operating System has to ensure that each execution flow is independent from the other ones. Usually, this is named “task design” or “task model”. We do not know any objective reasons to complicate this model that is ideally suited for applicative software.

Sis

However, in the past some attempts (First Apple Macintosh computers) have been made to let the applicative software provide some limited “cooperation” with the Operating System: A “yield_cpu” function had to be called by applicative software when it could be unscheduled. Also, memory allocated areas were using “handles” (two successive indirections) so that the memory manager could easily “move” dynamically allocated memory blocks in central memory. The result is that the “bomb” icon that was displayed whenever applicative software had a fatal bug that crashed the OS was considered as an Operating System issue, and Apple had to return to a more conventional approach, providing slightly lower performances but a safer behavior.

But when it comes to functions that are universally considered as part of any Operating System, such as file systems, telecommunication protocols, and some network services, we may ask whether implementing those as tasks, threads or kernel internal functions using “task-like” synchronization primitives such as waiting queues, semaphores or context switches is an effective approach in terms of performances, code reliability and security.

Simple benchmarks on “task oriented” designed file systems show that the continuous recording of two simultaneous 2 MB/sec video streams using a 5,400 rpm hard disk is impossible, despite the fact the physical “burst” capacity of the disk is up to 160 MB/sec (but with no disk arm move). So the “task oriented design” clearly provides for performances that are far away from the asymptotical hardware capacities. We do insist on the fact that it is not an issue tied to any specific implementation, this result is roughly the same one whatever the considered implementation is. On the other hand, recording five 2 MB/sec video streams, while reading five of others, so 10 files with a total 20 MB/sec is something easy to achieve using the time-line oriented design method. So we have reached a ten times higher total throughput. Other benchmarks using communication controllers are also demonstrating huge data rate gains.

The time-line analysis and design method foundation is the following one. When it comes to I/O processing, the initial “T0” is a high level software request (High level OS or applicative software) and then we have long sequences of hardware interrupts and software functions interactions. We will represent those as left to right time-lines. This is an orthogonal representation and approach compared to the “task-like” method since we do not consider individually processing steps or layers such as Ethernet, IP, TCP or File systems, the link of all of those is what we are interested in. This model allows for design, for a clear analysis of performance issues, but also for external measurements using oscilloscopes coupled with protocol analyzers. The software has to be “instrumented”, has to change GPIO states on crucial events such as the deposit of one I/O request (The initial “T0”), entering the Interrupt handler, exiting the Interrupt handler...

Scheduling and I/O Performances – Latencies issues

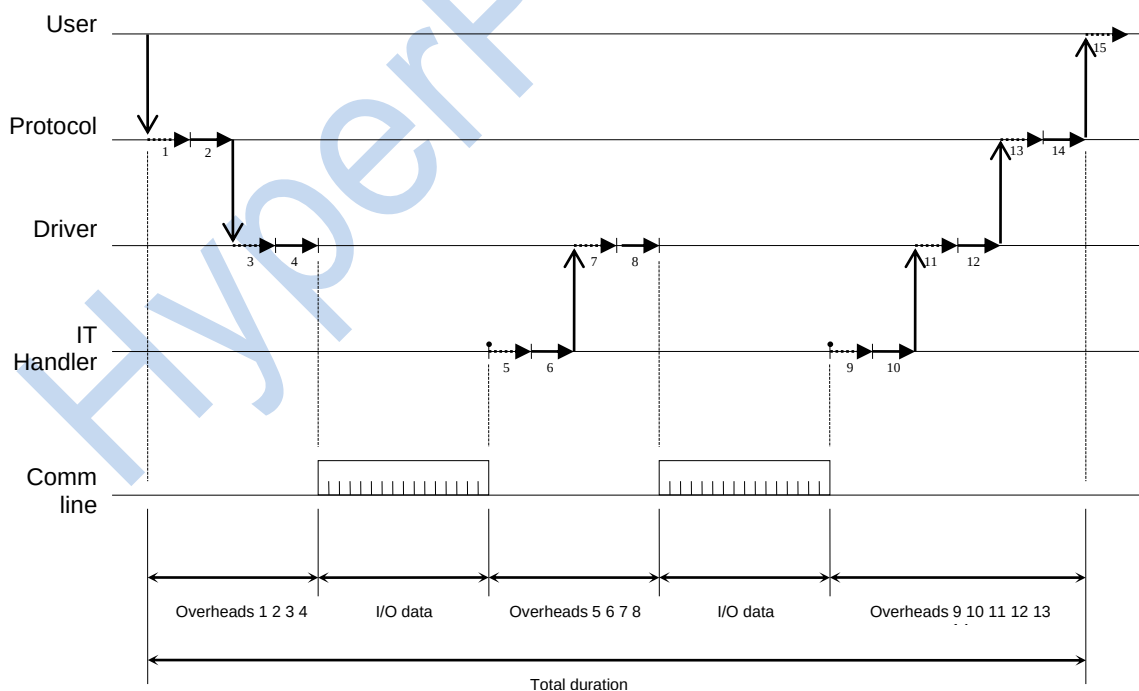
We focus on consumer electronic products, like phones, tablets, home routers, TV sets or IOTs. The three general following remarks can be made:

- They are used by a single human being. We do not use the term "user" because confusion can be made with a "Linux user" that may only be an execution context with attached rights.
- Processing power has become inexpensive and is often oversized. Some smartphones are coming out with 8 processors running at 1,300 MHz, so having 10,000 times the processing power of the first computers. An IOT typically contains one core running at 32 KHz up to 200 MHz.
- Those products are fitted with RAM capacities that may reach several Gigabytes, so 1,000 times the available memory of the first computers. On the other hand, IOT's RAM is only tenths of Kbytes.

However, many products are using OS's whose scheduling policies have been designed under the constraints of multiple human beings simultaneously connected, shortage of CPU and shortage of RAM memory. For consumer electronic products, those constraints have disappeared more than 20 years ago. Regarding OS architecture, the most obvious consequences of those three remarks are respectively:

- A piece of software is not upset because it is waiting for the CPU longer than "someone else"
- Due to the tremendous increase of processing power, there no longer exists any hardware for which interrupt processing has to be deferred. For I/O handling "time-slicing" only creates disturbances, both because of the OS overhead but also because of the increase of complexity of non-transactional event processing, as software can be unscheduled at any time. This is even true with IOT's.
- Virtual memory sub-systems are only creating disturbances with no added value as a counterpart.

We will use a simple scenario to illustrate a list of crucial points tied to scheduling and I/O processing. An application sends one data packet. The protocol splits it in two frames and issues a request to send the two frames. The driver gives the first then the second frame to the transmitting hardware, two end-of transmission interrupts are then raised. The hardware is not fitted with a FIFO for requests. This can be depicted as follows:



We have three activities called "user", "protocol" and "driver". We deliberately use the term of "activities" and not "tasks". We have an interrupt handler that does not defer end-of-transmission interrupt processing. The vertical arrows are calls to one synchronization primitive, it may be a semaphore or an event. On the "Communication line" we have depicted overheads and I/O activities. The horizontal arrows are switching durations and useful processing durations. Here is a description of what is happening:

- The "user" issues a request to the "protocol" to send a data packet. There is a switching of activity delay (1) then the processing time (2) of the protocol. The protocol builds two frames and issues to the driver one request to send the two frames.
- We have a switch of activity duration (3) and then the processing time (4) needed by the driver. It sends the first frame. On the communication line, transmission begins.
- When first frame transmission is completed, the hardware raises an interrupt. We have the interrupt latency (5) then the CPU time used by the interrupt handler (6). The driver is notified of the end of transmission of the first frame.
- We have a switch of activity duration (7) and then the processing time (8) of the driver. It sends the second frame. On the communication line, transmission activity starts again.
- When the second frame transmission is done, a second interrupt is raised by the hardware. We have the interrupt latency (9) then the processing time (10) of the interrupt handler, that sends a notification to the driver.
- We have a switch of activity (11) and the driver does some processing (12). It notifies the protocol.
- We have a switching time (13) and then the processing done by the protocol (14). It notifies the user.

In order to maximize the communication line throughput, one has to suppress or reduce all the durations of line inactivity that are depicted by the horizontal arrows. At that point, three objections may be raised:

- Firstly, the purpose of the OS is to maximize the overall activity and not only just one activity as with our simple example; to that we answer that this argument was meaningful at times when OS's had to face CPU shortage but this has disappeared decades ago. We will not argue if CPU's are powerful enough to simultaneously maximize the throughput of all devices because computations are demonstrating they all are and moreover we physically demonstrated it many years ago. One central argument of this paper is **how a modern OS architecture can make the best use of this new situation.**
- The second objection is the fact that modern hardware has FIFO's for data bytes and eventually for request blocks and so CPU switching and processing activities may occur in parallel of hardware activities, and the communication line will nonetheless be used at its maximum bandwidth. There are at least two main answers to that point. The first is that the total CPU time needed to start an I/O should however be lower than the I/O time itself and that there should not exist polling cycles for end of I/O detection that would be longer than the I/O duration. The second answer is that both telecommunication protocols and file managers are "dialogues", sequences of "reads" and "writes", where the "write" requests cannot be issued before the completion of the previous "read" because the read data is needed to determine the data contents of the following write.
- The third objection is that increase of processing power is sufficient to reduce all those durations to almost nothing without any change of architecture or way of coding software. But even with unlimited available processing power if scheduling is such that the CPU is not quickly unscheduled and continues to do something else, for example computations, then the I/O will not be serviced. One also has to consider that for a given CPU, the less CPU time is used for I/O's, the more time remains for user processing. Some chipset manufacturers told us **their Linux drivers were using more than 60% of the CPU time.**

We have a simple engineering point to give as an answer to a lot of further questions. If we have an OS architecture that allows us, at the lowest development cost, to reduce simultaneously and to nearly zero all of those durations, why should we bother to discuss duration per duration if it is worth doing this or that? So we will continue this explanation by giving a list of all the gains that should be worked on without discussing their relative importance because we were able to work on all of them, but also because the varieties of relative different timings values (serial communications versus file system managers) is such that in the end all the optimizations we made were revealed to be useful in some case.

Looking at the previous basic example, whereas the right oriented horizontal arrows are the overheads, times which have all to be minimized, we can build the following list:

Overhead	Schedulers and Interrupts latencies
1	User to protocol switching time
3	Protocol to driver switching time
5 and 9	Interrupt latency
7 and 11	Driver rescheduling after end of interrupt handler
13	Driver to protocol switching time
15	Protocol to user switching time
Protocol, driver and handler processing times	
2 and 14	Protocol processing
4, 8 and 12	Driver processing
6 and 10	Interrupt handler processing

One can ask why we do here distinguish 1, 3, 7/11, 13 and 15, since they all seem to be "tasks switches". The answer is that very probably with a conventional RTOS "user", "protocol" and "driver" will be tasks, inducing the same switching primitive among them, but the situation may be different depending on the OS. For example, with a Linux OS, "driver" and "protocol" may be located in kernel space and "user" in user space, and so the time (1) may be greater than (3) and (13), but also (1) may be different from (15). We will also remark that the interrupt latency (15) may be extremely different in value depending on the choice that interrupt processing is or not deferred. Also, we should like to point out that if different implementations may lead to very different processing times for protocols (2/14), driver (4/8/12) or handler (6/10), the choice of the **implementation methodology**, if any, can explain even greater differences; the special case "no implementation methodology" being the most common case and the worst one, while the use of FSM will give best results.

Reducing the time of each switch of activity is one point, but reducing the number of those is also crucial, because in the end the overhead is the product of each elementary switch time by the number of those. So, an OS has not only to provide for fast switching primitives, but should be also designed in such a way that the number of switches is reduced to a minimum. We will note that this point is seldom raised, considering that the number of switches is not an architectural concern. But we are not doing philosophy here, we admit that we have to do our best to reduce the product "switching time x number of switches" because it is in the end the overhead we get, also the number of "wasted" CPU instructions. And, last but not least, we know this will give great results just because we actually did it. Here is a list of measures to be taken in order to reduce the overall number of switches:

	Reducing the number of switches
A	Agglomerate telecom stacks and file systems managers
B	Use I/O lists rather than single I/O's.
C	Never use time-slicing
D	Never use blocking I/O primitives
E	Never use waiting queues to store I/O end notifications
F	Never use semaphores
G	Suppress virtual memory

Telecom stacks and file system managers are without any doubt complex functions. The list of documents that one has to read (RFC's) in order to implement a consumer electronic TCP/IP stack is rather long, because of progressive adjunction of new functionalities, some of those being implicit replacement of previously defined mechanisms (e.g. IP address resolution mechanisms). Specifications of any telecom stack are always divided in several more or less independent parts, for example we have TCP, UDP, IP but also physical layers such as the series of IEEE 802 specifications. Here come (A) a fundamental point that is seldom taken in account by OS's design and architecture and even sometimes being considered as out of the scope of the architectural design. If a stack is made of independent specifications, that are functionally independent, that also may be used by applicative software in a independent manner, does it however implies that their implementations have to be **independent pieces of software from the OS point of view**? A naïve answer is "yes indeed". But let's take the example of TCP within a system which is not an IP router. After the IP destination address ARP resolution has been completed, the IP layer can be reduced to a set of subroutines that are called by the TCP code, thus suppressing synchronization system calls and switches because TCP and IP are not executed in separate contexts. This basic optimization is simple and therefore often used. Now, let's imagine that we have a driver that issues one synchronization primitive for a group of sent frames, as in the previous example. This primitive will schedule the "protocol" (switching indicated as 13 on the diagram). What happens if the "protocol" is in fact a stack, made of several layers? Also, what happens when the "user" code issues one packet transmission request that has to be processed by several stacked layers (the sole "protocol" in our diagram)? As soon as the "stack" is implemented as a set of OS separated code entities, whatever they are (tasks, threads or even distinct FSM's) we have a multiplication of activities switches, whatever this means or implies.

So we have here a fundamental design and/or architecture question: is it possible to "agglomerate" telecom stacks in such a way they will be seen by the OS as a single execution entity but nonetheless allow for selective and distinct use if needed (e.g. a user wants to directly use IP but not TCP or UDP). The answer is affirmative, and on top of that code size is greatly reduced, easier to read and understand, protocol processing times (2) are reduced and the telecom stack can be automatically validated. There is no miracle behind that but the use of FSM. We however have to admit that in our case we found a methodology that gave this full result only three years ago after more than twenty years spent in refining again and again FSM based scheduling. A detailed explanation requires some tens of pages, that can be summarized by saying that a full stack is seen by the OS as one unique FSM and the C procedures which are the FSM methods call dedicated FSM engines as subroutines, typically three FSM/transition tables for each protocol layer (connection, send and receive FSM).

It has to be noted that points C, D, E and F do not require any special implementation work when FSMs are used because time-slicing, blocking I/O functions, waiting queues, and semaphores are not needed. Point B is depicted on the previous scenario; the driver receives one request block that contains two frames sending requests. We will finish with other measures that are not tied to scheduling but however we will list those here in order to have a full list of the various measures that will "boost" I/O processing:

	Other measures to reduce number of "wasted" CPU instructions
H	Nest interrupts processing
I	Limit data copies number and sizes
J	Limit data buffering
K	Never use spin locks
L	Use a design methodology
M	Use an implementation methodology
N	Use DMA and other hardware accelerators

H: Interruption preemption and nesting are provided by all modern chipsets including the cheapest ones, typically for small IoT devices. As previously stated, processing power is nowadays sufficient to process all the interrupts without deferring any interrupt handler. But whatever processing power is, interrupts of different devices may always occur within a delay of some clock cycles. Increasing the clock speed will indeed decrease the probability of a second interrupt occurring when the processing of a previous one is not completed, but this will not make the possibility disappear. Such a scenario is taken in account by the hardware because the second interrupt line stays active until the end of processing of the first one, and so the second interrupt handler is started just after the acknowledge of the first one. But complex chipsets always have some hardware "features" (the equivalent of a "bug" in software) that will imply some "software workaround". A very common one is relative to sophisticated DMA engines (having an internal firmware) that are able to parse "on the fly" the transferred data bytes and to raise a dedicated interrupt when a user defined data pattern has been detected in the data stream. The very common bug is that the register containing the offset of the matching data pattern is not frozen upon detection and therefore its value has changed after some instructions of the interrupt handler. In such a case, the interrupt handler has to parse again a part of the RAM in order to find the exact position of the found data pattern. But doing so, the parsing duration may randomly exceed the delay to fill-up the receive hardware FIFO of any other high-speed line. Typically, a 4 bytes FIFO used with a 400 kHz I2C bus is full after 10 µsec. In such a situation, two options are available: using the flow control to stop the I2C incoming flow (known as "pulse clock stretching") or nesting interrupts, the I2C IRQ interrupting the DMA handler.

I: This mainly means that data buffers should be mapped in the different address spaces used by the DMA, the driver, the protocol and the user space. Sometimes, this is not possible and the OS must copy data buffers just to overcome issues related to address spaces "frontiers". This has to be indeed avoided in the search for performance, not because of the CPU time consumption, but because of the DRAM bandwidth maximum limit. A chipset manufacturer (NXP) made the investigation to compare the performance of our OS and a Linux and discovered that when using Linux the DRAM was imposing the limit because of too much data copies to move data between system and user space, and for that sole reason developing a Linux Personal Video Recorder was just not possible with a DDR-1.

K: We can say it is possible to develop an OS without the use of any spin-lock because we did it.

Let's summarize before continuing

Time-line design relies on the fact that every system activity starts with an application request and then we have a sequence on interrupts and software to software signaling messages. We have to add that also the activity start, the T0 initial time, may also be an interrupt request: When a user presses a keyboard key or moves a mouse, an electrical signal is produced and an interrupt is generated. But this does not changes our method nor our target: From Operating System point of view, we receive asynchronous "events", whatever they are, then sequences of other hardware or software "events", and our intent is to try to reduce all the latencies that may exist to almost zero, because we want to be simultaneously as close as possible to the asymptotic limits of all the concurrently running hardware's.

So our first concern has been to identify delays that may exist between an event, a hardware interrupt or a user request, and the corresponding treatment start. We called "latencies" those delays and we intent to reduce all those to 0. Here is a summary of the different latencies kinds we encountered:

Latencies
(1) High level Software Request to low level software start
(2) IT raise to Interrupt handler entry
(3) Exit of IT handler to low level software start
(4) Low level Software end to next low level software start
(5) Low level Software response end to High level Software unblocking

Latencies are the "first order" issue. But with Task Oriented Operating Systems, we have also a second major issue that is the needed time for "context switch". In fact, all the latencies identified in the above list are suffering, are greatly affected by "context switch" durations. A "context switch" is all the operations that are done by an Operating System in order that one execution flow is stopped and another one is started or restarted. With "task" oriented Operating Systems a "context switch" is a "long" operations (RTOS) or a "very long" operation (GPOS) compared to hardware durations and latencies. A common "task" switch may include the following operations:

- Save the registers of the suspended code
- Save the stack pointer of the suspended code
- Update the OS internal state for the suspended code
- Restore the registers of the code to be restarted
- Restore the stack pointer of the code to be restarted
- Restore the address space of the code to be restarted
- Update the OS internal state of the code to be restarted

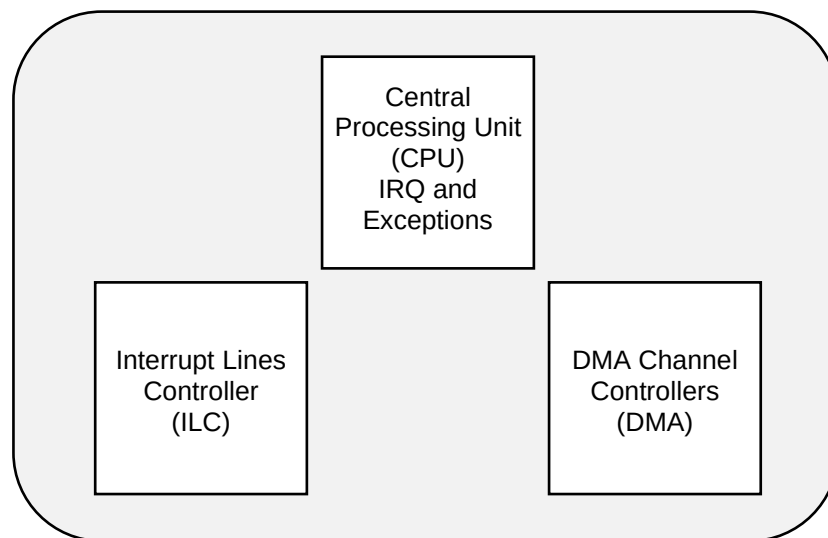
Many Operating System Designers have debates about how to reduce "context switches" duration. But in fact, when it comes to I/O's, the big issue is not the "context switch" duration, but the product "context switch duration" times "number of context switches"..

Our "Time-line" target is to reach the hardware asymptotic limit and therefore we will not discuss about the respective performances of traditional "task context switch" methods because in fact figures are showing that we would be between 10 and 100 time slower even benefiting of the best current performances, to the cost of increased complexity and low software reliability and very low maintainability. So rather than trying to optimize the "task context switch" duration, we will evaluate the possibility just to get rid of it. This means indeed that we get rid of the "task" notion, which may seem absurd at a first glance because we have a lot of concurrent on going activities at the hardware level. We do not want to reach the hardware limit of one sole dedicated device, but simultaneously be as close as possible of all the hardware limits of all the concurrently running devices whatever their number is. But as previously stated, we do not intend to change anything at the applicative level. We limit our perimeter to what is occurring **inside the Operating System** and to what is not "visible" to the applicative software.

Next, we are going to describe hardware capabilities we will need for our Architecture, namely Interrupt Request and DMA hardware. We will explain how start I/O system calls and device IRQ's can be merged. The HyperPanel OS architecture is made of two containers, which are called "I/O" and "application" containers, we will describe the I/O container then the finite state automaton engine that is the heart of it. We will then describe the Application container that is also built around an event loop. We will describe the automaton engine extensions provided by the Applicative container automaton engine. Finally, we will describe the overall two container's Architecture.

The hardware Trinity for no latency, fast IO's and activity switches

Besides the CPU execution unit, the OS Architecture we will describe later on will need as foundations three hardware blocks that are the IRQ and Exception handling logic, that is located inside the CPU, the Interrupt lines controller and the DMA channel controllers. The first two are absolutely mandatory for our architecture and fortunately all modern CPU's, even for the smallest IOT's are including both. The DMA channel controllers are "nice to have", it has to be noted that only the very low end IOT's MCU may not provide it, typically MCU's with 8 bits CPU's.



The Central Processing Unit IRQ and exception block and the Interrupt Lines Controller are often referred to by the "Interrupt Request Sub-system" name, that we will use. As our OS architecture is heavily relying on those three hardware blocks, we detail their behaviors before describing the architecture. We will therefore describe in detail the Interrupt Request hardware. Then we will examine the Direct Memory Access hardware. We will introduce the FSM based I/O container architecture.

Interrupt Request hardware

What is an Interrupt Request?

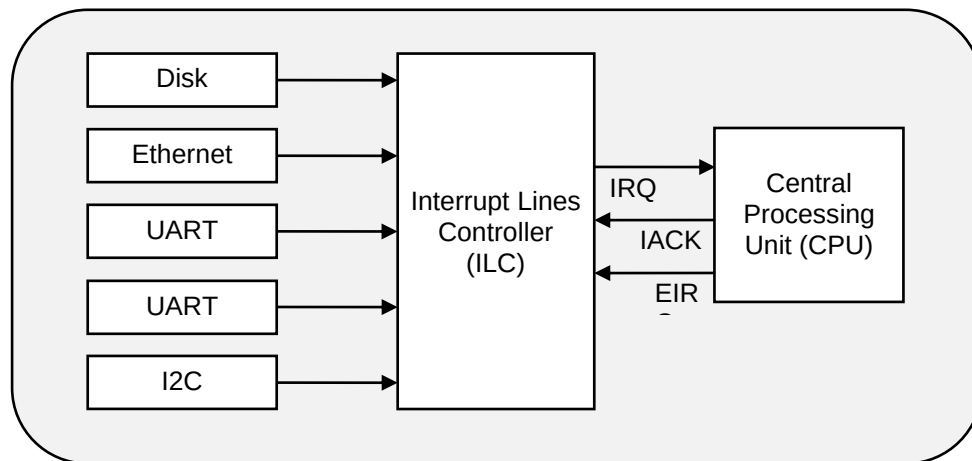
Time quantization of processing time is a main issue when software has to detect an end of transfer. But even when processing time is not quantized, when for example there is only one software task in memory, end of transfer detection is still very ineffective if the software has to "pool" the hardware (periodically check if the transfer is completed).

So as "software only" end of transfer detection is extremely ineffective, hardware designers had introduced "Interrupt Request". An interrupt request for service is an electrical signal generated by a device, like a disk or an UART, that is inputted to the CPU in order to notify that one end of transfer has occurred and that the CPU has to take care of it, to service it.

When a device raises the CPU Interrupt Request input, the CPU fetches in a RAM table the address of the next instruction to be executed. This address is often known as "interrupt vector". This is done by hardware inside the CPU and only takes some CPU cycles, typically between 1 and 3 CPU clock cycles, so some tenths of nano seconds with a 100 MHz CPU clock.

Typical hardware needed for Interrupt Requests support

Fast and efficient Interrupt Request handling has led to the development of dedicated very high performances hardware. A small part of this hardware is within the CPU but the most important one is a block external to the CPU, usually named “Interrupt Controller” or “Interrupt Level Controller”. The hardware of an “Interrupt Request sub-system”, the CPU and Interrupt Level Controller, can typically be depicted as follow, despite the fact that minor differences may exist from hardware to another one:



As previously explained, each device (Disk, Ethernet, UART, I2C bus controller) provides one and only one electrical signal that we call its “Interrupt Request”. Very often those signals are “inactive” when they are in the “high state” and “active” when in the “low state”. On the above schematic only 5 devices are depicted but with “real world” systems the number of those may be higher or much higher, typically from 40 to 60. As a CPU is a complex VLSI, it would not be an effective solution to modify it if some more devices have to be added. So, we have an intermediate block, the “Interrupt lines controller”. We have one line between each device and the Interrupt lines controller, so one wire for each device Interrupt Request signal.

Each time a device sets its interrupt line in the active state, the wire that goes from the ILC to the CPU (IRQ) is set to its active state by the ILC. It has to be noted that the ILC does not contains any software or firmware, so the latency between the two signals becoming active can be considered as 0 (in fact, one pulse clock of the ILC).

When the CPU IRQ input becomes active, the CPU immediately stops executing its current program, at the end of the currently executed machine instruction. Then it loads into the Program Counter (PC) a pre-defined address named the “interrupt handler entry point”. Then the IACK signal is activated by the CPU in order to inform the ILC that it can “release” the IRQ line. Those operations will usually need two or three CPU clock cycle.

So now, the CPU is executing the “interrupt handler” for the device that has requested to be serviced. For example, with an UART, the “interrupt” handler may read one just received byte from a memory mapped register and store that byte in RAM memory, which is extremely short.

At that point, it has to be noted that what is described here is a purely hardware mechanism with an almost null latency. So, it may seem that whatever the OS is, performances will always be the same ones. But in fact, many OS designers are not using this mechanism the way it has it has been intended for because they feel it difficult to implement (see next point “Nested Interrupt Requests”).

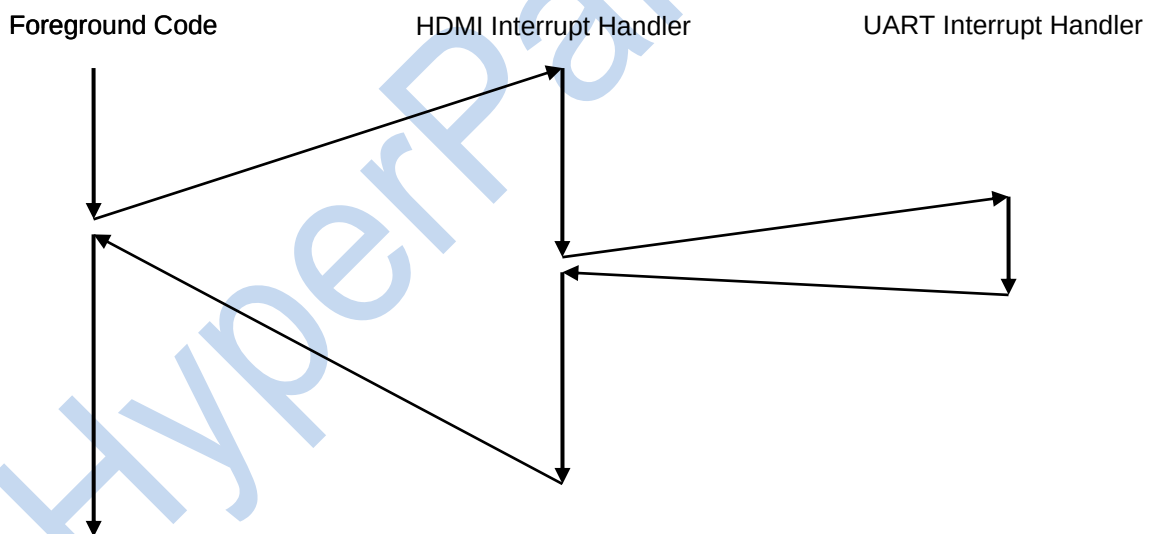
Any interrupt handler has to use a dedicated machine instruction (IRET) in order to notify to the CPU that the processing of the interrupt handler is completed. The reason to use this special instruction is that a dedicated CPU electrical signal EIRQ is activated. The purpose of this signal is to notify to the ILC that an interrupt of lower or equal priority can now be taken in account.

Nested Interrupt Requests

Different hardware devices may have very different behaviors regarding their flows of interrupts. For example, an UART that receives bytes at 9600 bauds will issue one Interrupt Request for each received byte, but the processing of each only needs some machine instructions and not more. An HDMI link will issue one Interrupt request when the HDMI cable is plugged or unplugged and not more but the processing of this one may be complicated. So obviously if the UART IRQ signal becomes active while the HDMI interrupt handler is executing, one has to interrupt the HDMI interrupt handler, start the UART interrupt handler that only needs some machine instructions, then resume the HDMI handler at the point it has been stopped. So, we will not lose any UART incoming byte. More generally, devices with high interrupt rates will always need some instructions and therefore they have to interrupt handlers of devices with low interrupt rates and longer processing. It may happen that a handler that interrupted another one may also be interrupted itself. This is called “nested interrupt requests”.

The ILC is involved in the generation of nested Interrupt Requests. It always includes “priority handling”: The software, during the ILC initialization process, has to provide to the ILC, for each of its device Interrupt Request input, a number that is called a “priority”. Then the ILC uses those numbers when any software interrupt processing is not completed and another device interrupt line becomes active:

- 1) If the priority that has been associated to the new active interrupt line request is lower or equal than the priority of the device interrupt line that is currently serviced then no nesting occurs, processing of the first interrupt continues. When this one is completed, the software executes the IRET instruction. The EIRQ signal is activated by the CPU and the ILC is allowed to generate any new interrupt, so the lower priority interrupt is now taken in account: The ILC sets to the “active state” the IRQ signal, and so the interrupt handler for the second Interrupt Request is started.
- 2) But if the priority that has been associated to the new active interrupt line request is greater than the priority of the device interrupt line that is currently serviced then nesting occurs, processing of the first interrupt is temporally stopped and processing of the second interrupt handler is started. When the IRET instruction of the second handler is executed then execution of the first handler resumes.



How to correctly use Interrupt Requests Hardware

Modern electronics, including the smallest *Systems on a Chip* used for IoT's, all have interrupt controllers allowing nested interrupts. Software can configure the priority of each interrupt, and when two interrupts arrive nearly at the same time, the handling code for the first can be interrupted if the priority of the second is greater. The software must assign the **greatest priorities** to the **shortest and most frequent** handlers and the least priorities to the longest and rarest handlers. That way a long handler will be interrupted by short ones, and no latency is induced in the handling of the frequent interrupt, which leads to the maximum speeds. A short handler cannot be interrupted by a long one, but this is of no importance. Of course, the short handler codes must be carefully optimized, and will not perform slow calls to the system. No interrupt handling can call a blocking function, such as grabbing a semaphore.

Examples of common misuses of Interrupt Requests Hardware

Certain architects consider that the considerable power afforded by the modern processors eliminates the need to make use of the interrupt priorities, and that it's not a problem to call slow system procedures from interrupt handlers even though simple order of magnitude computations prove the contrary. This is the reason why certain operating systems, even when ran on powerful processors lose the last bytes of small radio messages transmitted at 115,200 bauds: if a UART has a 16-byte deep FIFO, it can be filled in 1.5 milliseconds, and the following bytes will be lost if the interrupt handler is not run during this time period. In the case of a communicating operating system, which because of its job, is exposed to sustained high interrupt rates, the interrupt handling must be performed swiftly and cannot be postponed and placed on a run queue, because doing so only increases the amount of ineffective activities in the system, increases the latencies, and worsens the problems due to interrupt misses.

So, as previously described, all the modern MCU's are including Interrupt Request sub-system as described in "Typical hardware needed for Interrupt Requests support" and therefore all operating systems have the possibility to provide zero latency Interrupt Handler start, including the nesting of Interrupt Requests of increasing priorities. But very few OS's are "correctly" implementing the use of the Interrupt Request sub-system hardware. Here are some examples of those "incorrect" uses:

Incorrect use of priorities. Very commonly priorities are not rationally chosen. Typically, the choice is "psychological": The HDMI is "more important" than the UART and so a higher priority is given. As explained before, this is not correct criteria for defining Interrupt Request priorities.

Delayed interrupt processing. This name is given to the incorrect idea that an OS may be temporarily "overloaded" and that delaying the processing of "non-urgent" interrupts is a solution. This is a very simplistic view because when I/O rate has to be sustained the concept of "temporarily" overloaded is by itself just absurd: The system is "constantly" overloaded and in order to correctly handle and maximize simultaneously all the concurrent I/O's then nested Interrupt Request processing has to be implemented. In fact, the concept of "delayed interrupt processing" is an ineffective way to try not to implement the nesting of increasing priority Interrupt Requests.

Drivers implemented as tasks. This is a technique that "partially" differs the processing of the Interrupt Requests: The interrupt does only a part of its job and sends a message to a queue or a semaphore that a task is waiting for. This does not work correctly since the time to re-schedule the task is up to the OS and may be dependent of its processing time quantum. This way of doing is often used with RTOS's.

No use at all of interrupt priorities. A lot of porting's of Linux in embedded system are just affecting all the device interrupt lines priorities to one same unique value in order that nesting of interrupt requests never occurs. As a result, any "short and frequent" Interrupt Request as to wait for the completion of any "long and rare" Interrupt Requests. Those systems will "loose" UART characters and Ethernet frames or graphic vertical (blanking) Interrupts, creating defaults that are externally "visible".

Drivers implemented as "tasklets". This is a Linux concept: As it became "standard practice" not to use nested interrupts, then the drivers are (almost) a user task and there one unique waiting queue of events (Interrupt Requests waiting for processing). This is worse than the previous choice since the "tasklet" activation time is even longer than a RTOS task one.

Driver implemented as user tasks that are polling a socket. This is what can be found in Android. The so called "drivers" are "sleeping" typically for half a second, then they "poll" a UNIX "socket" to check is there is something to be done. This is a "funny" way, using 8 cores running at 1.2 GHz, to provide for lower I/O performances than the IBM PC 5150 of 1981 that only had a single Intel 8088 processor running at 4.77 MHz. But "grabbing" user data is much more easier using such "drivers".

Merging start I/O System calls and device IRQ's

User code is always foreground, using the lowest CPU execution level. Classical Operating system architectures are using the concept that whatever the service user code is requesting to the OS kernel, the user code to OS code switch (S1) is always done in the same way, which is reasonable as long as all the system services are located inside one same kernel. Almost all CPU instruction set architectures are providing a dedicated instruction that user code will systematically use (INT with Intel architectures, SVC with ARM). The new HyperPanel OS architecture is using two kernels or containers, one high level container hosts user tasks and a second low level container that includes all that is related to I/O's, so indeed drivers but also protocols, file system and services such as HTTP servers. This architecture is enabling for a LAN spread Operating System because a user code may request an I/O to the local I/O container but also to a remote I/O container. Indeed, the HyperPanel OS provides system libraries, that are set of subroutines the user will always call and as the result the INT or SCV will never been found in user compiled code but in OS library subroutine codes. So, if system library codes are using one way for always local requests, such as creating a task or a semaphore, but also a second and distinct way for I/O related calls, this will make no difference for the compiled user code.

Within our OS architecture, local only system calls are always subroutines that are using the user stack and address space, no activity switch is done unless the call is explicitly requesting that control must be transferred to another user task.

At a first sight, we can use the CPU "system call" dedicated CPU instruction as an interface to the I/O container. But the HyperPanel OS architecture has been ported to numerous CPU architectures including Intel, Motorola/Freescale, MIPS, ARM, ST, Fujitsu, former DEC, transputers, ... and it appeared that the different "system call" CPU instructions (INT, SVC, TRAP, SYSCALL, ...) behave differently from one instruction set architecture to another one: passing call parameters may vary and relative priority/execution level between "system call" and CPU Interrupt Requests may also vary. Indeed, we can use software encapsulation to hide the differences in passing parameters. But it is not so easy with relative priorities/execution level of "system call" instruction and CPU IRQ's because they never are configurable. So, we had to drop the idea to use the "system call" CPU instruction as the interface between the application container and the I/O container. As the solution to our need does not exists inside the CPU, we have to find it somewhere else. We can use the Interrupt Lines Controller (ILC). All ILC's are able to raise the IRQ output line if a dedicated ILC register is written to by software. So, the foreground user code will proceed as follows:

- Fill a standard data structure that holds the request parameters
- Link this data structure at the end of the I/O container input list
- Writes to the ILC register that raises the IRQ line from the ILC to the CPU

In our architecture, all the I/O container functions are one FSM each, one transition tables and therefore the standard data structure we use is the event structure ("s_evt" data type). On the hardware point of view, the ILC software raised IRQ, like any device input line, have a software defined priority, which means the OS can always select the relative priority of the software raised IRQ and devices raised IRQ's. In our architecture, we always use the lowest ILC priority for the software raised IRQ and all the devices lines will be affected strictly greater priorities in order that the handler for the software raised interrupt will always be interrupted by any physical IRQ line raise. Indeed, this requires that interrupt nesting is implemented. The interrupt handler for the software raised interrupt is simply the I/O container entry point, a C function that is the VMIO FSM engine. As the request event has been linked to the VMIO input list before the ILC register is written, the VMIO, the software raised IRQ handler, will find upon try this event and will immediately treat it (if no other event was previously linked).

So, in one hand we put apart the local only system calls and the I/O related system calls activity switches but, on the other hand, we merged I/O related system calls and device IRQ activity switches. This new and unconventional way is perfectly suited with the use of Finite State machines inside the I/O container.

Direct Memory Access hardware

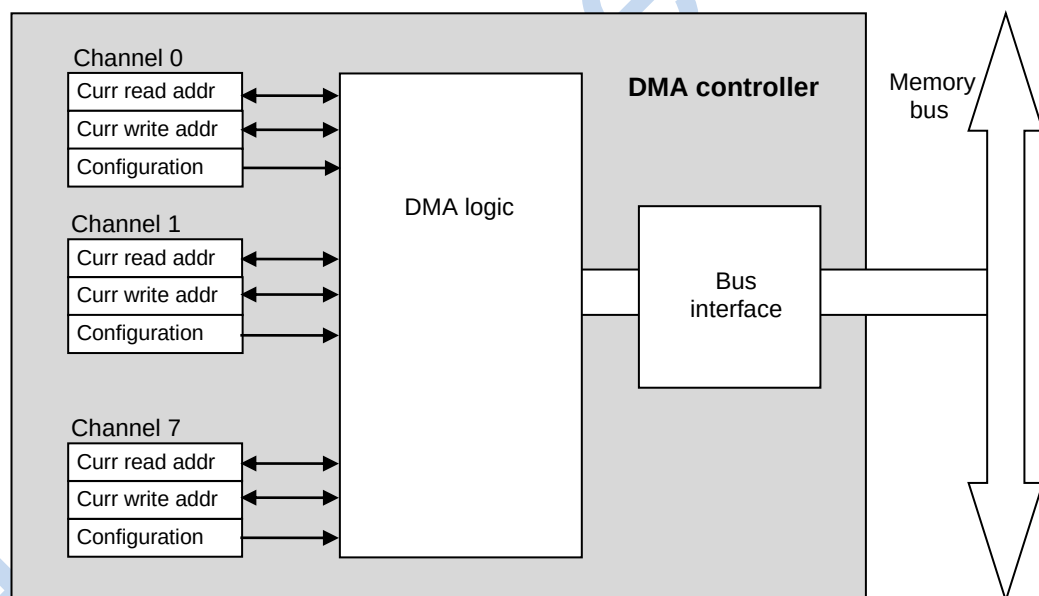
What is a DMA transfer?

A Direct Memory Access (DMA) device is a hardware block able to copy data into one same physical address space, without using a CPU. Thus, CPU's do not have to process time consuming data copies. A DMA transfer is one data copy executed by a DMA device.

Inside a same physical address space, one can find RAM memory, but also ROM memory and registers or memory mapped devices. So, we may have the following different use cases for a DMA:

- When copying from RAM or ROM to RAM, source data bytes (or words) have to be read at consecutive increasing addresses and they have to be written at consecutive increasing addresses. The DMA device must have two current "read" and "write" addresses that it must automatically increment.
- A DMA may be used to "inject" a stream of bytes (or words) that are located in RAM to a device input register, for example an UART or disk data input register. In that case, the DMA has to automatically increment a current read address but the destination address is always the same.
- Conversely, a DMA may be used to read a stream of bytes from a fixed address, an input data register address and write the data in RAM, the DMA has to automatically increment a current write address.

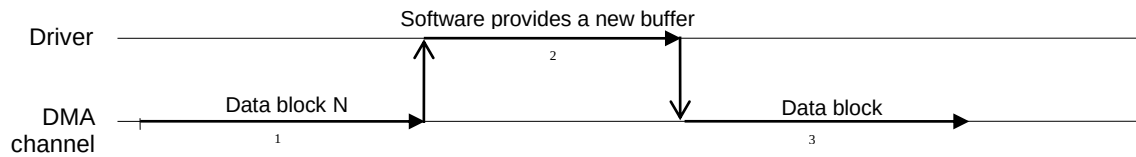
Usually, a DMA device is able to concurrently process several copies, in such a case, the DMA is said to have several "channels".



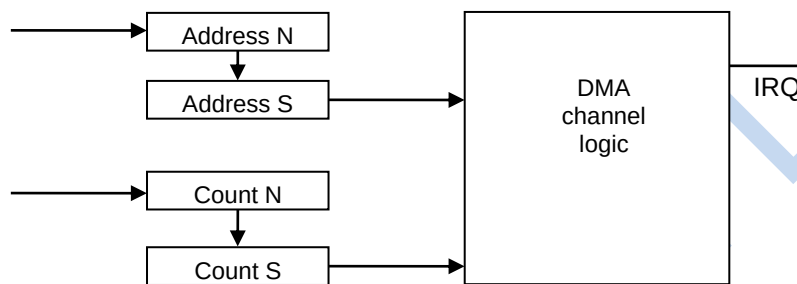
A channel is a set of registers and other associated hardware that is able to handle one transfer. In that case, the name "DMA controller" is used instead of DMA; a "DMA controller" will provides more than one "DMA channel". Within the DMA Controller, we have one set of dedicated registers for each channel. We have at least a "Current Read Address" register, a "Current Write Address" register, but also at least one "Configuration" register that will indicate if the "current read address" and the "current write address" have or not to be incremented after each byte or word copy.

Streaming, Hardware circular buffers and slot buffers

High speed streaming, e.g. continuous data flows such as sample flows or tuner outputs may need special DMA hardware features if no byte has to be lost, for both input data streams but also for clocked output data streams. A specific issue is raised by the fact that as soon as any DMA channel is designed in order to handle data blocks, bytes may be lost during the short period of time that is needed by software to "give" to the hardware a new buffer after the end of transfer IRQ has been raised:

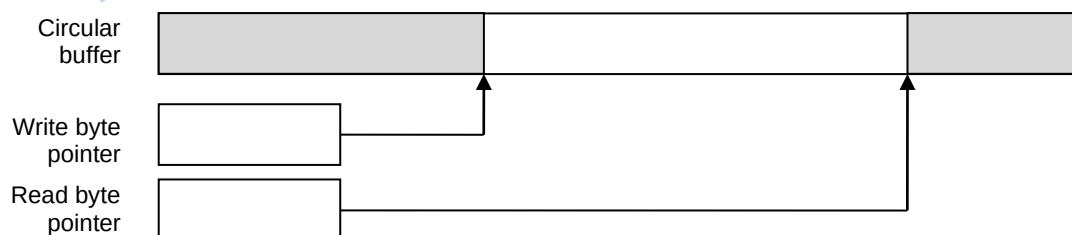


A first solution is to use FIFO's between the DMA and the device that continuously sends or receives data. But hardware FIFO's are expensive. They also are, from a hardware point of view, difficult to interface in a flexible manner: A FIFO is always tied to one specific device, such as an HDMI transmitter. A second and better solution is to have double buffered registers for each DMA channel. Sometimes, "double buffered" registers are also named "double banked" registers. If we take the example of a DMA channel that is receiving data from a register and writing it to memory, we only needed a double buffered data address register and a double buffered data count register. The register that is software addressable is usually named the "normal" register (N). The second register has no address and is only handled by the hardware, it is usually named the "shadow" register (S).

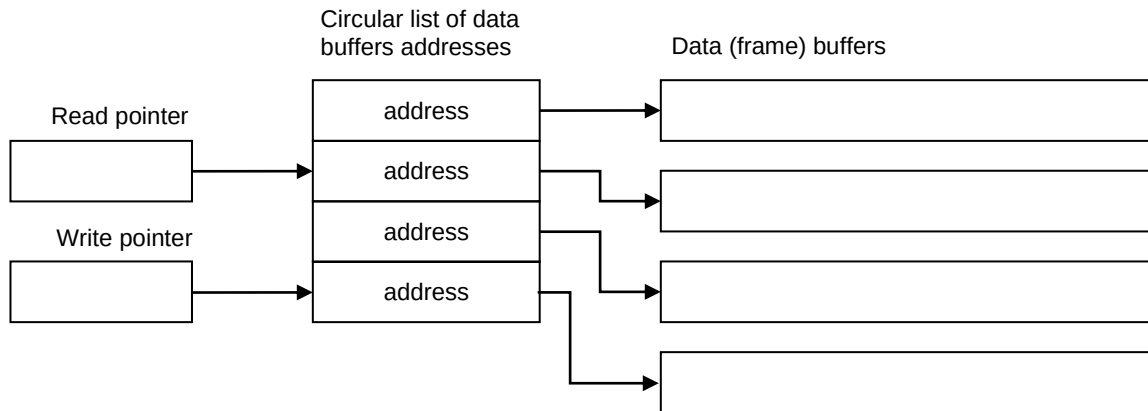


Exactly when the last current block byte has been transferred to RAM memory, the DMA channel logic simultaneously triggers the copy from the "N" registers contents to the "S" registers, without any software intervention. In such a case, the IRQ is not raised because one transfer is completed but because new values may be written to the "normal" registers. So, when the DMA logic is started, a first IRQ is raised and the software writes to the N registers a first (address, count) pair. The first data transfer is started and immediately the IRQ is raised again and the software has to provide, to write to the N registers a second (address, count) pair. Then the first transfer goes on. When the first transfer completes, the hardware copied the N registers to the S registers, so the second (address, count) pair and it raises the IRQ. The software must provide, write to N registers the third (address, count) pair, whilst the second transfer is going on.

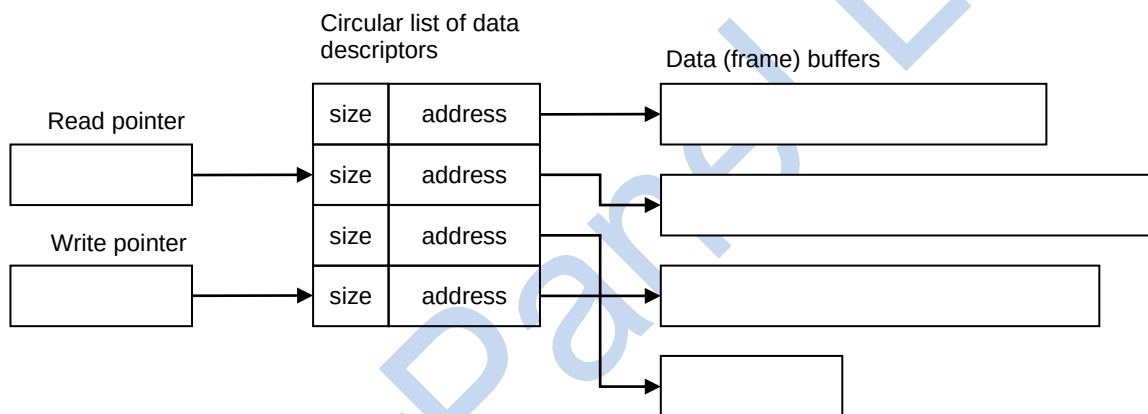
So, register's "double buffer buffering/banking" seems to be the perfect solution, since it is very inexpensive and very flexible, because the mechanism is a DMA channel feature and no more a device feature as with FIFO's. The maximum software latency for providing a new data buffer since the IRQ has been raised is one data block duration, which seems "comfortable". But let's take the example of an ethernet link. The Ethernet frame (maximum) size is 1522 bytes, so $1522 * 8 = 12\,176$ bits, which means 12 micro-seconds at 1Gbits/sec and 120 micro-seconds at 100Mbits/sec. As a matter of fact, such values are not "comfortable" at all for many OS's. So, the hardware designers have to provide for "more". We are going to describe here two families of solutions, that we will call "hardware circular buffers" and "hardware slot buffers". A hardware circular buffer never loses any received byte and never delays any transmitted byte for the simple reason that it never needs to be restarted by any software, when buffer end is reached the hardware automatically "wraps" the hardware write or read pointer. The software will be triggered by IRQ in order to get or put data in the circular buffer RAM area. As the buffer size is software defined, the slower the OS is, the bigger the circular buffer will be.



If we receive fixed size blocks instead of byte streams, a circular ring of fixed sizes data "slot" is more convenient (for example, with ethernet frames). The hardware manages a circular list of data buffers of same sizes, each time a buffer has been filled up the hardware is able to fetch with no software help the next buffer address.



With a DMA channel that is used to receive data, the data buffers will be of same (maximum) sizes. But when sending data, data buffers may have different sizes, as with data sent using an Ethernet controller. In such a case, the circular list of buffer addresses must become a circular list of "data descriptors", each descriptor including a data address, a buffer size and eventually other additional data items:



The software has to allocate the data buffers, give to the hardware the list of descriptors and then start the DMA channel. Each time the hardware completes the treatment for one descriptor, it will raise an IRQ, enabling the software to associate a new buffer to the just "completed" descriptor. A "descriptor" is often also named a "slot" and a slot may be identified with an integer number ranging from 0 to "n-1", if "n" is the count of slots.

Additional DMA functions

As depicted on the previous diagrams, any DMA controller includes a DMA logic block that receives the data byte per byte or word per word and sends them to the DMA output. One may have the idea to add inside the DMA logic additional functions that will process the flow of bytes or words:

- A DMA may be used to parse the data stream in order to detect some data patterns. For example, the MPEG standard states that 3 consecutive bytes (0x00,0x00,0x1) are a block delimiter (start code). So, a DMA that transfers any audio, video or teletext stream may be used to detect the start codes and notify their location in the data stream.
- A DMA may modify the data stream "on the fly". For example, scrambling or unscrambling the data stream. A DMA device able to scramble/unscramble AES or TDES is something very common. Also, some DMA are able to convert color spaces, 3 R,G,B values are converted in 3 Y Cb Cr values. In these cases, the number of bytes remains the same. Also, "tap filtering" can be done, in order to implement digital image filtering at high speeds.
- The number of outputted bytes may be lower or greater that the number of inputted bytes. Image sub-sampling or over-sampling can be done by 2D DMA's. Very often, tap filtering is also used in order to reduce sub or over sampling visible artefacts.

-
- A DMA may also be used to protect sensible data from robbery. In such a case, there is only one input data register in a device, and then the "secured device" will include an internal DMA to read the encrypted sensible data.

Hardware filters are not DMA's. However, they are often used very closely with DMA channels and so we will shortly explain here what is hardware filtering. DMA channels may be used with data byte streams but also with data blocks. When data blocks are received, such as Ethernet frames (1500 bytes) or MPEG2 packets (204 bytes), a fundamental operation, aka "filtering" may be necessary in order to decide "what to do" with the frame or packet. Filtering one data frame or packet means that a few contiguous data bytes are masked and then compared with values that are contained in a table that is named the "filter table". Each table entry is a "filter" and contains two bytes strings of the same size, the first string is the mask and the second one the value. A filter may contain additional data, such as a "key", a value that is returned when a filter "matches" an incoming data block.

Filtering is a very simple operation that can easily implemented by software, with a loop that will scan the filter table. The loop will stop with the first "match". But with high data rate, we may have performance issues. A single HD TV program may need 24MBit/sec of throughput, so 8MBytes/sec or approximatively 15 000 packets per second. If the table holds 8 PIDs and if we consider that 4 comparisons will be made in average, we will have to execute 60 000 filter comparisons per second. Some MCU's may include hardware filtering devices that will unload the CP: Digital Television MCU's are providing PID hardware filtering whilst Ethernet switching MCU's are coming with hardware ethernet addresses filters.

How to correctly use DMA Hardware

In one hand, DMA controllers are able of numerous functions with extraordinary levels of performances, we previously gave some examples of such DMA controllers or DMA related functions. But on the other hand, if we put aside the standard "data move" functions, additional available functions are heavily depending on the MCU that hosts the DMA controllers. Also, the count of DMA controllers, the count of channels of each controller may vary from one MCU version to a very close other version. Many Operating system designers may consider handling this variety has to be left up to the applicative software. But experience shows that application developers, especially in embedded systems, are experiencing the greatest difficulties even with one single hardware and also that doing a software copy is always preferred because it is easier to code. So, in the end, DMA are very seldomly used despite the tremendous gains they offer. We consider that the purpose of an Operating System designer is to solve the complicated problems a developer is not capable of, and points like "it is not an OS job to do that" are dogmatic and meaningless. Our purpose is that the user code is the same one whatever DMA functions are available or not, that also the OS code is left unchanged, whilst DMA functions are "automatically" used each time this is possible. This can be done as follows:

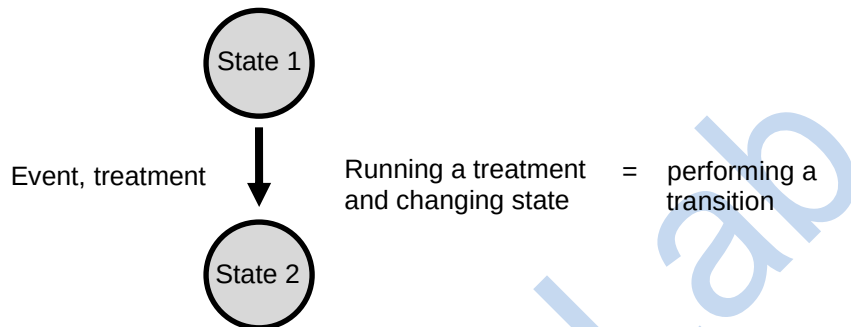
- For each DMA controller and for each DMA channel, the OS contains constant data that describes the capabilities. This data is only read by the generic DMA driver.
- The DMA system driver provides allocation procedures that will fail if there is no or no more DMA channel that is compliant with a list of requested DMA functions.
- Each OS provided driver, such as ASY or ETHERNET, calls the system DMA driver allocation function. It has to be noted that one same DMA channel may be allocated several times as it can be shared amongst several users. Then, each time the driver receives a request, it compares the request parameters and the channels capabilities and decides if a DMA channel may be used or not.

For example, the HyperPanel OS ASY driver does not use DMA if any flow control, XON/XOFF or RTS/CTS has been requested by the applicative software. The application never explicitly asks to use or not DMA. By doing so, DMA channels are systematically used whenever possible and the benefits are huge in terms of data throughput. Also, if the hardware does not provide for enough DMA capacities, the applicative software is always still fully and correctly working.

FSM based I/O container architecture

What is a Finite State Machine?

Writing software as a finite state machine involves defining a list of states and a list of events. From every state, events can be received that drive transitions (from one state to another) during which a treatment will be run. A Finite State Machine is also named an “automaton” and we will often use that shorter name.



A finite state machine can be represented as a graph. Each of the graph's vertex is a state of the automaton. The arrows represent the transitions, an arrow points from the starting state to the outcome state. Every arrow is associated to two parameters which are the event which cause the transition, and the treatment which is run during this transition. Another way to represent the automaton is a transition table. There is one column for every state, and one row for every event. The current state number selects the column. When an event occurs, it selects the line. The intersection of a column and a line is a cell containing two distinct elements: the treatment to run, and the outcome state. Every automaton has a “state variable” which contains the number value of the current state. At a given point an automaton is in exactly one state, and the state variable contains this state value.

Why use Finite State Machines?

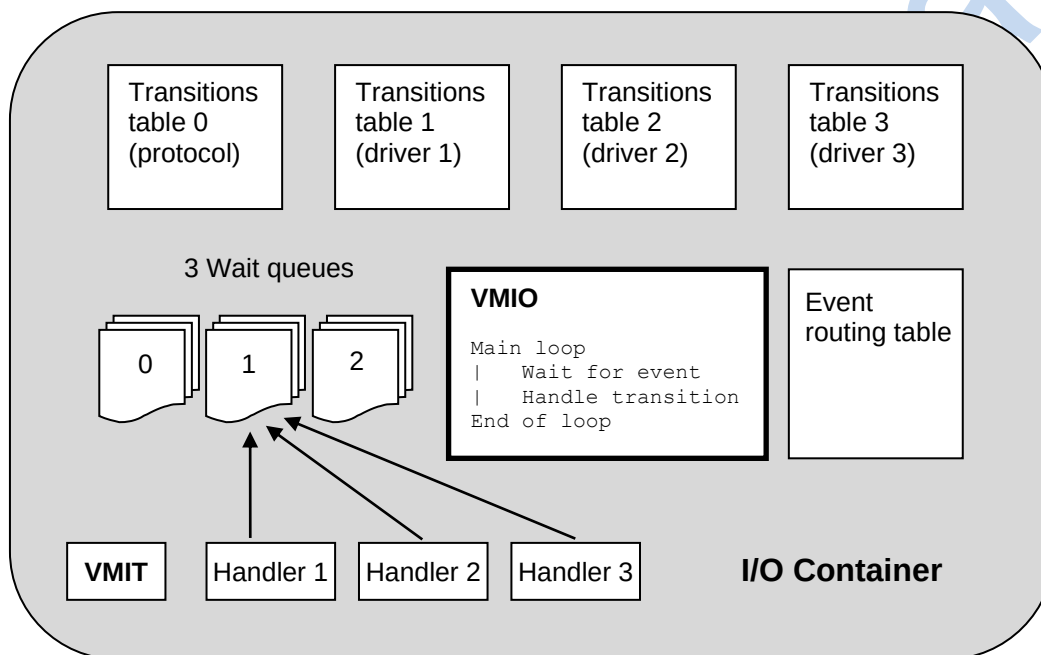
The code which makes an automaton run, which uses its transitions table is often called an “automaton engine”. The events are small messages whose two most important fields are the event code, and the “context number” also called “logical way number”. A treatment is a single C procedure which is called by the engine. There are many benefits to writing a software process as an automaton and to using an automaton engine, including:

- Defining an automaton forces the use of proper methodology to analyze a problem. Indeed, the transition table forces to explicitly plan the (state, event) couples which are possible, and those that are not. Every time an event occurs in a state where it was not expected, the engine detects the anomaly without requiring more application code. This guarantees that hazards are always detected. Finally, an automaton handler is by nature short and simple, which greatly simplifies the understanding and maintenance of the code.
- An automaton engine allows multitasking inside the container, regardless of whether the underlying operating system supports it, including when there is no OS (bare metal), which is another benefit. Going from one software function to another happens solely within an executable container, without involving the underlying scheduler. This provides on one hand much faster context switching than what real-time operating systems provide, but more importantly this time is independent of the system, be it Unix or a “real-time” system. It is easy to add the means to monitor the transitions within the code of the engine, without changing the handler code.
- The notion of “logical way” of an automaton spares code and affords greater readability, because reentrancy is structural, each event always containing a context number, or logical way number.
- Every event handling is transactional, in the sense that the handler run to completion without being interrupted. This greatly simplifies the synchronization between the different activities. In particular, the need for semaphore **disappears**. This point is essential from the point of view of the operational safety of the input/output container, which we strive to make extremely high.

- With the upsides of using an automaton engine listed, let us also draw a list of the major downsides. First, all of the container's functions must be written as automatons in order to benefit from the profits. Second, care must be taken to ensure that no treatment is slow, which would block all the other functions. Third, experience shows that few developers are able to design a proper automaton graph.
- To sum up, an automaton engine is an efficient and elegant solution to implement protocols, to build multi-access servers or intelligent terminals. It is the ideal tool to create input/output containers. It is not to be used for high-level applications, such as scientific computing or accounting applications.

How an I/O container's FSM engine works

An I/O container receives request events from the application container on one side, and end of physical input/output notification events on the other, which are dropped off by interrupt handlers. The notification events are a consequence of the request events, and they must be handled first in order for flow control to work properly. We therefore use two wait queues of different priorities, the middle priority (1) being given to the one for the end of input/output events, and the lesser priority being given to the request events (0). Queue (2) is reserved.



The VMIO automaton engine for the input/output container is made of a loop called "Main loop". In this example, we have chosen to include four transition tables numbered 0 to 3. We have made table 0 be for a protocol, and tables 1 through 3 be for drivers. For each of these three drivers, there is an interrupt handler, each drops off events to queue 1 (middle priority) to "its" driver automaton. The queue 0 receives request events from the outside. The container works as follows, each of the following three points corresponding to consecutive times:

- A request event is dropped off in the wait queue 0. Let's assume that the event is a transmit request meant for the protocol, that is to say automaton 0. Immediately, the VMIO processes the transition: the address of the C procedure and the value of the next state are extracted from the protocol's transition table for the couple (current state, event code), the procedure is called as a sub-procedure, and the state variable is updated to the value found in the transition table. Let's assume further that the transmit requests implies sending two consecutive frames and that these frames are sent using driver 2. The protocol automaton 0 drops an event in the queue 1 to the driver automaton 2 to transmit the first frame. The VMIO main loop immediately detects this new event and draws the transition of automaton 2, whose handler will start transmitting the frame. Then, because both queues are empty, the engine blocks waiting for a new event.
- When all the bytes of the first frame have been sent, the interrupt handler 2 drops an event in wait queue 1 to automaton 2 to notify the driver of the end of transmission. This unblocks the automaton engine which executes the transition of automaton 2, and the handler drops an end of transmission event in queue 1 to protocol automaton 0. This drives a further transition of automaton 0 which drops off an event requesting the start of transmission of the second frame. The engine draws a third transition for

automaton 2 whose handler starts sending the second frame. With both queues empty, the engine once again comes to a halt.

- When all the bytes of the second frame have been transmitted, the interrupt handler 2 drops an end of transmission event in queue 1 for the driver automaton 2, which unblocks the engine. The automaton's handler drops an event in queue 1 for the protocol automaton 0. A new transition is executed, and an response event is sent to a function outside of the input/output container. Both queues are empty, and the engines come to a stop once again.

In this explanation, we failed to mention how the logical ways are used. In the case of a connected protocol, a way will be for a single connection. In the case of a driver, a logical way will be for a single peripheral. For instance, if the electronics have two asynchronous serial ports, then the asynchronous driver will have two logical ways. Of course, each way has its own state variable and its own context.

Why all latencies had magically disappeared?

Our target is the asymptotic hardware performance for all the concurrently running devices. Our methodology is the Time-line Design Principle. Our implementation is based on Finite State Machine's, we have chosen to organize in a unique monolithic container a nested interrupt manager named VMIT, a FSM engine/monitor and all the driver/protocol/service transition tables. But how all the 5 types of previously identified type of latencies have all magically disappeared? And also, why we do not have any equivalent for the "task switching" time? We recall here the 5 types of latencies that we have previously identified:

Latencies
(1) High level Software Request to low level software start
(2) IT raise to Interrupt handler entry
(3) Exit of IT handler to low level software start
(4) Low level Software end to next low level software start
(5) Low level Software response end to High level Software unblocking

The sub-OS named VMIT is the one that manages the hardware we previously named "Interrupt Requests Hardware" as depicted in "Typical hardware needed for Interrupt Requests support". It fully implements nesting of increasing priorities Interrupt Requests. The VMIT requires that the ILC provides 8 priority levels or more (This is even available with the smallest IoT MCUs). The VMIO receives software events requests from the "upper" layers (Wait queue 0) and interrupts notifications from "lower" interrupt handler's (Wait queue 1). Step by step behavior is as follows:

1. Assuming we only have a single core to execute all the code, the high level request is an event that is directly written in the VMIO wait queue 0. At that point, here comes our first challenge, we have to reduce to 0 the latency between this event deposit and the low level software start (1).
2. Here is the first "miracle": The procedure that writes the request event in Queue 0 also writes into one of the ILC register in order to rise by software a low priority Interrupt Request. As we are currently in foreground code this software raised Interrupt Request is taken in account at the next CPU clock cycle (We did not yet exited from the high level event deposit procedure). The low priority Interrupt Request handler is in fact the VMIO FSM Engine entry point. Here, we have a "reduced" task-like context switch because we save application registers and stack pointer current value, and we install a new stack for the VMIO. The VMIO processes immediately the request event, it calls as a subroutine one C procedure whose address is extracted from the protocol transition table 0. This procedure formats a first frame and writes a frame sending request event in Queue 1 for driver 2.
3. Here lays our second challenge, we have to reduce down to 0 the latency between this event deposit and the start of the "driver 2" processing (4). Here is the second "miracle": we still are in the VMIO FSM Engine monitor that immediately extracts the just written event and then call the "driver 2" FSM treatment (procedure). This procedure writes in the device hardware register and physical transmission of the first frame starts.
4. The treatment procedure of driver 2 returns, we are back in the VMIO FSM Engine loop. Queue 0 and 1 are currently empty, the application registers and stack pointer are restored and the EIRQ signal is activated, the Instruction pointer for the interrupt applicative software is restored and we are back in the application code, still in the code of the event request deposit function.

The hardware is transmitting the first frame. CPU may be allocated to another application since the requesting application is currently blocked. We have to remark that previous steps 1 to 4 have been done as a unique transaction, that in a same atomic period of time we have executed application code, then protocol code and finally driver code with absolutely no latency between those and only two “task-like” context switches. When the first frame transmission is completed the hardware raises an End Of Transmission interrupt:

5. The corresponding handler is immediately started (0 cycle wait, but for the intrinsic CPU latency, typically 2 to 3 CPU clock cycle) if no other interrupt is currently being processed. If we already have a lower priority interrupt being processed then this one is stopped and nesting occurs. We achieved a null latency for the IT raise to interrupt handler entry (2).
6. The interrupt handler always fully processes the interrupt, in other words deferred Interrupt Request processing is never used. Therefore for each end of data block transfer the interrupt handler directly writes a software event in queue 1 of the VMIO.
7. We have now to achieve zero latency between the IT handler ending to the low level software start (3). The subroutine that writes the events also writes in a register of the ILC in order to activate (by software) an interrupt of lower priority than all the devices Interrupt Request Priorities. If the lowest ILC priority is 0, then priority 0 is only used with this “software generated” Interrupt Request and all the devices Request Interrupts are using priorities from 1 to 7.
8. At the very moment the IRET execution for the interrupt handler is executed then the EIRQ line is asserted. The ILC immediately “discovers” (0 cycle latency) that the low priority “software raised” Interrupt Request is active and immediately (0 cycle) the ILC puts the IRQ line in its active state again.
9. The interrupt handler for the software raised low priority interrupt is still the entry point of the VMIO FSM engine. This means that the latency between the exit of the interrupt handler and the entry of the corresponding “high level” driver (3) has been reduced to 0: one cannot do better. Using a driver written as an RTOS task will imply a switch of context and therefore latency. One can claim that “task switch” has been highly optimized but within the I/O container there is no context switch (but for the register and stack) between the interrupt handler and the high level driver. In other words, it’s not possible to be faster in any way; indeed, whenever register banking is available, we use it with interrupt handlers.
10. When the high level driver (one C procedure extracted from the FSM transition table) has been executed it will, for example send an event to a protocol. Again the writing of this event will re-activate the software raised interrupt. So, when the driver’s C procedure completes, the VMIO entry point (the handler of the software raised interrupt) is immediately called (2 or 3 CPU cycle). This means that the latency between the end of execution of the driver and the start of execution of the protocol the data is intended to (4) is again 0.

When the second frame transmission is completed the hardware raises an End Of Transmission interrupt:

11. The IT handler is entered and an event is written in queue 1 for the driver FSM. The handler writes in the ILC controller register that raise the Interrupt whose Interrupt Handler is the VMIO FSM engine entry point.
12. When the interrupt handler IRET instruction is executed, the EIRQ signal is asserted and immediately the ILC raises again the IRQ signal because of the software raised interrupt, one more time (3) is null.
13. The driver treatment is executed and this one send an event to the protocol FSM
14. We do not exit from the FSM VMIO engine, we extract the event and execute the protocol treatment, we have (4) zero latency. The protocol treatment writes in a high level queue the end of transmission notification event. When we exit from the FSM VMIO the high level OS, in our case another FSM monitor, immediately extracts the response event and restarts with no delay (indeed if no other task is running) and latency from low level software response end to High Level Software unblocking (5) is null.

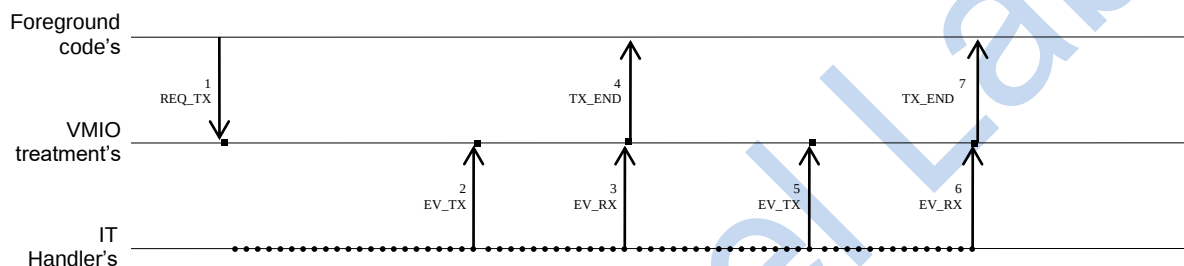
The VMIO uses a unique stack, a unique space address for all the treatments of its automats, thus **considerably** reducing “task-like” context switches.

Additional benefits of the VMIO Monolithic FSM Monitor

VMIO Monolithic FSM Monitor makes foreground code smooth

The fundamental power of VMIO Monolithic monitor relies in the fact that treatments belonging to different “layers” of software, so drivers, protocols and services are by construction and without any effort of development automatically grouped in transactions that are never “giving” hand to the “high level” OS until all the events have been processed, so until both queues 0 and 1 are empty. Within such a transaction, no context switches are done, stack is not switched nor registers nor space address.

At that point, one could argue that the “time-line” oriented design, or its FSM implementation, shows a weakness since, with a mono core system, we never give hand to the high level as long as there is anything to be done at the low level. But let’s examine the resulting time-line:



Each character end of transmission interrupt is depicted as a small dot that also figures the needed processing time that is almost nothing (a few micro seconds). VMIO treatments, depicted as small squares, are also very short (10-1000 microseconds). Therefore, the CPU processing time that is “stolen” to the foreground code is made of “microscopic” time slices at a very high frequency. So at the high level, everything goes smoothly. The more complex functions (protocols, file systems, network services) are run by the VMIO FSM engine, the more those are becoming ultra-fast as previously demonstrated but also, at the foreground level, smoothness becomes the rule. In particular, especially when using small CPU with no graphic accelerator, the UI’s behaviors are remarkably smooth.

Internal I/O container piping of high speed data streams

The VMIO has a pipe mechanism, which allows a unidirectional data transfer from FSM to FSM. A pipe is configured and started using request events, and then the client doesn't need to intervene anymore for the data to flow from FSM to FSM. Thus, the various data flows occur without any context switching, and in particular no copy and no address space change. If for instance the application configures a pipe ETHERNET => TCP => HTTP => FAT, a high speed download feature is realized.

In this example, the TCP protocol, the HTTP service and the FAT file system are executed within the monolithic monitor, and not within the VMK RTOS, which runs in the app container (see below, the HyperPanel spread OS architecture). This is another major trait of the architecture we introduce. Some have noted that the RTOS's are too slow for demanding I/O tasks and have tried to develop faster RTOS's by reducing the context switching time. This is pointless, as it reduces the functionality of the RTOS, without even truly reaching the goal of a sufficient speed-up. By having both a time-line input/output monitor and separately an RTOS, we can both perform I/O faster than an RTOS would, while also having a feature-rich RTOS if needed at foreground level.

Details of the I/O container FSM engine

The different concepts related to finite state automaton engines in regard to their use in the case of an input/output container are further expanded upon in the points:

- States, state variables, and contexts
- Transition tables – empty cells
- Events
- Event treatments
- Conditions
- Timers
- Event putting, sending and routing

States, State Variables, and Contexts

An automaton is a transition graph. The states are the vertices of the graph, and the transitions are the edges between the vertices. It is common practice to number the vertices from 0 to N, and to refer to each state, to each vertex by its number. Each logical way of an automaton has a state variable, an integer containing the current state number of the logical way. It is useful to store the state variables for a single automaton in a one-dimensional array, because the logical way number can be used as an index in that array. In addition to the state variables which are exclusively read and written by the automaton engine, the event treatments of an automaton are using different persistent sets of data for every logical way. The data used by a logical way is called its “context”. Usually a C structure is defined containing all of the fields for a context. An array of such structures called a “context table” can then be defined; the logical way number is an index in that array.

Transition Table – Empty Cells

On a piece of paper, an automaton is often represented as a graph. There are circles for the graph's vertices, which correspond to the various states. The transitions from one state to another are represented by arrows. Above each arrow, we have both the name of the event which causes the transition, and that of the treatment to run. An automaton can also be represented by a two dimensions array, called transition table. This table has one column by state and one line by possible event. Each cell of the table contains two elements which are a reference to the event treatment and the number of the outcome state. When the product of the number of states by the number of events is not too high, we use a “complete” transition table, that is a two-dimensional table. When the size of the transition table grows, experience shows that most of the cells are empty. This expresses the fact that there is not a transition from every state to every state: the graph density is low. In this case, the two-dimensional table can be “compressed” as two one-dimensional tables, or two lists. The first list is a triplet list (event code, treatment reference, next state). It is sorted by state: all triplets for state 0, those for state 1, then those for state 2, etc. The second list is an index giving for every state the index of its first transition in the first list. In the following example, the transition table contains three states with respectively 3, 2, and 4 transitions:

EVT Code	Treatment	Next State	
+	+	+	+
			<----- 0
+	+	+	+
			+----- 3
+	+	+	+-----
			+--- 5
+	+	+	+
			<---+ +----- 9
+	+	+	+
			+-----
+	+	+	+
			<-----+
+	+	+	
+	+	+	
+	+	+	
+	+	+	
			<-----+

The first element of the index table is always 0. The index table contains one more element than the number of states which give the index immediately following the last transition of the last state.

When an event is received in a state where it is not expected, we have an “empty cell”. Indeed, in the case of a complete transition table, the array cell is empty, containing no treatment and no next state. Encountering a case of an empty cell signals a defect in the design of the transition table. An automaton engine is normally able to automatically detect these abnormal conditions, these empty cells.

Events

An event is described by a set of values whose number and nature depends on the automaton engine used. However, no matter the engine, an event always has at least three fields:

- The reference to the recipient automaton. Indeed, an engine always has multiple transition tables, and the events meant for different automata can be stored in the same internal wait queue. This reference is usually a number.
- The reference to a logical way, of the context of the recipient of the event. This field is usually an integer which is an index in the state table and the context table.
- The event's code. This field is usually an integer which is an index inside a full two dimensions transition table.

In the case of an engine used by input/output container, a request event should also include at least the following other fields:

- A reference to the destination node. The application container which originates the request will not necessarily be hosted on the same machine as the machine with the input/output container.
- A reference to the node making the request, which will be used by the input/output container to return the response event.
- A reference to the automaton making the request.
- A reference to the logical way making the request.
- A data address, used for instance to in the case of a transmission request.

The purpose of an automaton engine in an input/output container being to obtain the best performance possible, an event will always be a fixed size message made of the fixed size fields and always in the same order. The size of a message, of an event is typically 32 bytes. We use a C structure type named “`s_evt`”.

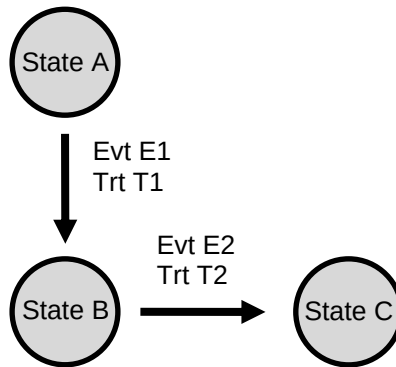
Event treatments

Every transition, every arrow of the graph is associated to a single event treatment which is always executed when the transition is triggered. According to the automaton engine used, the handling can take different forms. It can be an interpreted script, or native code. Of course, in the case of an automaton engine meant for an input/output container, a treatment can only be native code, typically a compiled C procedure.

In the case of transitions where the only processing required is a change of state, an empty treatment can be used, that is to say code with no instructions, an empty C procedure for instance.

Conditions

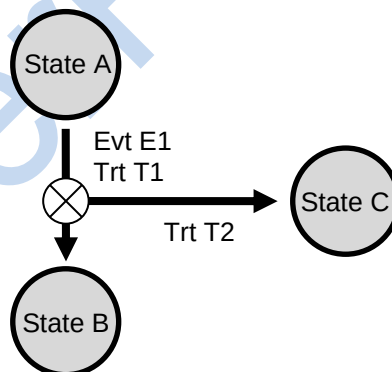
In the base model previously described, the automata evolve according to the events they receive from the outside, and from other automata. This explanation deserves further clarification, and “conditions” come in the picture. In a transition table, the next state number is a constant, in other words, the pointers to vertices of the graph are statically and definitely defined. This can become a problem in certain cases, for instance when a handler detects an anomaly or an end of a loop. From state A, event E1 is received and triggers treatment T1. After having run T1, the automaton normally goes to state B. But T1 contains tests, for example on the values of parameters associated to the event. Sometimes, an anomalous condition has occurred, and the next state should be C rather than B, and additional treatment has to be performed: T2 needs to be called. To achieve this, it is enough that when T1 detects an error, it sends the automaton an event E2. In this case, once the call to T1 is completed, we transition to state B. The FSM engine immediately finds E2 in the wait queue, and runs a new transition, which will take the automaton to state C, by executing T2.



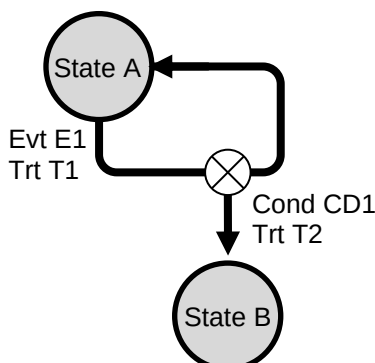
Apparently, this solution seems suitable, because in case of error, we will have run T2, and we end up in state C. But there are two downsides:

- First of all, in case of error, going to state B is fictitious. Actually, no time is spent in that state which is only crossed. The readability, the understandability is improved if the transition is directly from A to C.
- More importantly, we have done **two** transitions, two runs of the automaton engine loop, and that can have devastating consequences. It is possible that the handler T1, in addition to sending E2, has also sent another event to **another** automaton before E2. In this case the other automaton will handle that event **before** the transition from B to C, and before T2 is called. We want transitions to be transactional, that is not interrupted by another event handler, in order to do without semaphore. When T1 detects an error, T2 must be part of the same transaction, that is T2 must be run right after T1, without any other handler between them.

The second downside is a major issue, so we cannot use an additional event and transition. We introduce the concept of "condition". All treatments may "raise" a condition, a condition being an integer. Just after the end of the treatment, if a condition was "raised", the FSM engine doesn't change the current state, and uses this condition as a new event code, and another transition occurs. Final state is that of the second transition. It's important to realize that condition is an internal mechanism of the engine, and that no event is added to any wait-queues. On a graph, the presence of a condition is usually depicted by cross along the transition, along the arrow. In the following treatment T1 can raise a condition. In that case T2 is executed by the engine just after the execution of T1. Final state is not B, but C.



In addition to the detection of errors, the mechanism of condition raising can also be used to terminate loop as in the following example:



From state A, we expect to receive the event E1 100 times. The first 99 times, Trt1 must be run, and the automaton should stay in state A. The 100th and last time, Trt2 should be run as well, and the automaton should go to state B. To run this loop, we can use a variable in the context as a loop counter. The treatment Trt1 increments the global variable and raises the condition CD1 when the counter reaches 100. When the condition is raised that way, the automaton engine runs Trt2 and goes to state B. This terminates the loop.

Timers

An input/output container runs real-time functions which sometime require timers. Typically, when a point-to-point protocol sends a frame, it sets a timer. The remote end is supposed to acknowledge the frame before the timer fires, that is reaches zero. When the acknowledgment is received, the timer is cleared, while if the remote end doesn't answer in time, the timer reaches zero and the frame is sent once more. With an automaton engine, it is logical that the end of the timer be indicated using an event. The VMIO automaton engine must provide this service, and it must provide at least two procedures respectively allowing an event treatment to set and clear timers. For every active timer, the VMIO will need to keep at least the following information:

- A unique reference to this timer, attributed when the timer is started and used to clear the timer.
- A firing date, for example expressed in ticks, that is in number of milliseconds since power on.
- The code of the firing event.
- The number of the automaton receiving the firing event.
- The logical way number receiving the firing event.

Event Putting, Sending and Routing

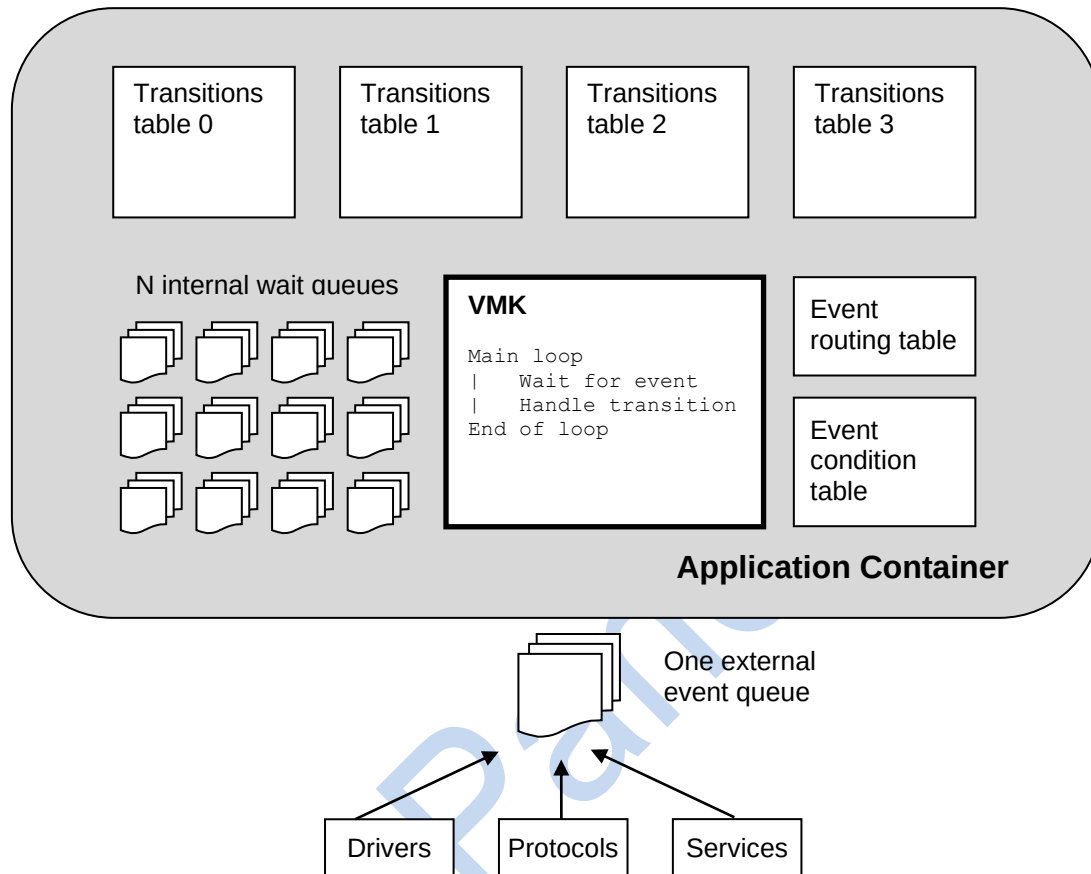
We will end this part by describing the most fundamental aspect when using an automaton engine, which is to say the mechanisms to send events. Our architecture needs three different mechanisms for the IO container:

- Events are “put” by writing a single event in the local wait queue 1. This is used by interrupt handlers to send up an event to the driver which handles it, driver which is always written as an automaton.
- Events are “sent” to transmit a single response event to an automaton which can either be in the same input/output container, or in an application container on the same machine, or in an application container in another machine. This mechanism will first look at the node number and automaton number of the recipient, in order to find the target container and to perform the appropriate actions.
- We call event “routing” the mechanism used by a driver, protocol or service automaton when an event it creates must be sent to one or more automatons which must first have declared the desire to receive a copy of such events. Each client creates a “route” which specifies the parameters of the desired events, such as the automaton number and way number of the event creating automaton, as well as the code of the event. Each route specifies the automaton number and logical way number of the receiver.

Nothing prevents the same event from being put, sent and routed. For instance, pressing on a keyboard key, or an infrared remote control can both be sent to an application container, as well as be routed to one or more drivers of the same input/output container, for example to make a LED blink at every key press.

Building an application container using FSM and events

The HyperPanel OS application container is using an FSM engine and its architecture can be illustrated as follows:



The three sources of events are drivers, protocols and services, which are located in the I/O container previously described. The Application container hosts applicative FSM's and tasks, it works as follows:

- 1) An event is placed in the external event wait queue by a driver, protocol or service.
- 2) The wait for an event being over, the main loop continues by determining the number of the driver which placed the event in the queue, and calling the automaton engine procedure with the event as input parameter. One of the fields of the event identifies the source driver of the event.
- 3) For each of the drivers, protocols or services, the FSM engine has a procedure to "bubble up" an event. Using the input driver number, the engine calls the procedure for the driver, protocol or service which acted. The "bubble-up" procedure extracts the event from the FIFO, and copies it to one or more of the N internal wait queues of the automaton engine. In order to do so, a routing table indicates for every event which of the N internal wait queues the event is copied to.
- 4) The engine checks its internal wait queues from the highest priority to the lowest. This is how the notion of priority between internal queues is implemented. As soon as it finds an event, the engine extracts it from the internal queue and calls the procedure which will handle the transition of the destination automaton of the event.
- 5) Once the treatment is finished, the FSM engine restarts checking its internal wait queues. It is possible for an event treatment to have created an event for one of the internal queues. Thus, the internal queues can contain both "external" events (from drivers) and "internal" events (from automaton treatment).
 - a) If at least one event is found in one of the internal wait queues, the engine goes back to step 4.
 - b) If at least one event is found in the external wait queue, the engine goes back to step 3.
 - c) If no event is found in either the internal or the external queues, the main loop resumes waiting for an event in the external wait queue.

The different basic concepts related to finite state automaton engines have already been introduced previously and are all applicable to the application container FSM engine, those are:

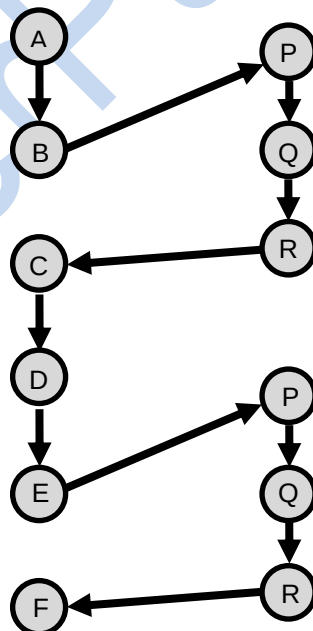
- States, state variables, and contexts
- Transition tables – empty cells
- Events
- Event treatments
- Conditions
- Timers
- Event putting, sending and routing

Additional functions are also available from the VMK, which is the name we gave to the application container FSM engine. Those are:

- Hierarchical FSM's
- Event conditional writing
- Internal and external event queues
- Lists of awaited events
- Tasks
- Semaphores
- Time slicing scheduler

Hierarchical FSM's

A transition table, as described in "Transition Tables - Empty Cells" may be as large as needed and therefore this is "conceptually" sufficient to implement any FSM (Finite State Machine) of any size and complexity. But however, this may not be handy in all cases. It is rather common that identical sequences of states may be duplicated in large transition tables. This is not an issue in terms of performances nor memory footprint since a transition only needs typically 12 bytes of storage. But designing a transition table is always a difficult job and if this one is too large the design may become extremely complex. So, as with programs that are using "subroutines", in order to "break down" a design and to get smaller and simpler distinct pieces, the same thing can be done with transition tables. Rather than designing a unique large transition table, one can design several smaller and simpler pieces.



This automaton has 12 different states and if we use a single transition table the sequence of states would be A B P1 Q1 R1 C D E P2 Q2 R2 F. If we break that in two parts, the first part states are A B C D E F and the second part states are P Q R, thus a total of 6 + 3 = 9 states instead of 12. This kind of organization is commonly known as "Hierarchical FSM's".

Event conditional writing

Some events come from the "outside world", they come from the I/O container, see diagram in "Building an application container using FSM and events". In some instances, events expire, that is to say they can become uninteresting, because they become too old. For example, receiving an MPEG frame just before changing the filter which had acquired it. The sequence of actions is the following:

- The application requests a change in the demultiplexer filter, which is done by calling the "drv_setval" procedure. The effects of this procedure are not instantaneous. An event is written to the DMX driver. This event is stored in a "driver" wait queue.
- Before extracting the "driver" event from the queue, a section matching the "old" filter state is received. The demultiplexer driver writes an event in the external event wait queue.
- The demultiplex driver receives the event for the request to change the filter.
- The VMOS automaton engine extracts the event from the external event wait queue, and copies it to one of the internal wait queues of the VMOS
- Then, the automaton engine wakes up, and activates one of the automatons of the VMOS, which might be precisely the one to have requested the filter change.

In the end, the VMK automaton which requested the filter change receives a notification event for the reception of a section **after** having made the request to change the filter, and the received section was for the previous state of the filter. This is inconvenient. The fact that a request, which is carried over an event, can thus "cross" a notification event of a state or of data from "before" the request is a phenomenon which can always happen whenever an engine sends a command event to an "external" event source, such as in this example, the driver which is not handled by this engine. To avoid these "parasitic" events, there are two possibilities:

- Let the automatons "deal with" the obsolete events.
- Introduce in the engine a "session" mechanism for all external even sources, which will let the engine itself prune the obsolete events from the drivers, by **not** writing them to **any** internal wait queue.

In the second case, it is necessary to have a table of event conditions. For every external event source, the table contains the criterion of expiration of the events from that source. Thus, every time the VMK engine extracts an event from the external event queue, it checks this table, and is capable of discarding the "unwanted" obsolete events. Our HyperPanel OS architecture is using that mechanism, that is fully unloading the applicative software of this management: experience shows that dealing with old or unwanted events is never rigorously done by applicative software and we consider that all what is difficult to manage or implement must be transparently done by the OS code. To sum up, the VMK API procedure that requests the update of driver tunings increments the device associated session number when needed, and all the events that are received later on bearing a session number that is not equal to the current session number are discarded by the OS.

Internal and external event queues

As shown in the diagram from "Building an application container using FSM and events", the FSM engine has one "external" wait queue, and a set of N "internal" wait queues. The "external" wait queue receives events from the outside world, from the drivers, protocols and services, from the I/O container. The main loop of the FSM engine extracts the events one after the other. Each event is entirely handled before the next event is extracted from the external wait queue.

Just after having extracted an event from the external wait queue, the FSM engine writes the event in one or more internal event queues. There is one copy per "route" (see "Event Routing Table") which matches the event. Then the FSM engine extracts the highest priority event, that is the event at the head of the queue with the highest priority, and performs the transition. The execution of the treatment for this event might create and write events in the internal wait queues. The engine continues "running" as long as at least one internal queue is not empty. When all internal queues are empty, the engine places itself on hold until a new event arrives in the external wait queue. In fact, if the MCU is able to do that, the CPU clock speed is reduced, until the next interrupt is received. In the ARM architectures, we execute the "Wait For Interrupt" (WFI) machine instruction.

List of awaited events

An automaton treatment can request to only receive events from a set of expected events. This effectively filters the events that the FSM engine extracts for that logical way. The VMK engine places event-level locks on the events already in the queue, and those that are put after the filter is set in place. To specify the filter on the events, an automaton allocates a structure called list of events to wait for, or event wait list, or simply event list. This list contains a set of expected event patterns, specified by 3 integers: a type and two values "val1", and "val2". There are two types:

- `WAIT_CODERES`, where the event is accepted if and only if the event's "code" field is equal to the "val1", and "reserve" is equal to "val2".
- `WAIT_CODEINT`, where the event is accepted if and only if the event's "code" field is the range [val1; val2].

Once the event list is allocated and setted, it can be given to the VMK engine to filter the incoming events. This mechanism may be used by any VMK FSM, even though it is primarily intended to be used by "tasks", that are special cases of VMK FSM's. After being created, a wait list is associated to one and one only internal queue. Each time an event is added in the internal waiting queue, the OS compares it to the wait list and a lock is set if the event does not match the list, if it is not an awaited event. The main loop of the VMK FSM engine extracts the first unlocked event from a queue, in other words the oldest unlocked event, if there is one. It then unlocks all the older events sent to the same logical way which were locked. Finally, it performs the automaton transition.

Tasks

To implement any software as an event loop requires some upfront planning. Integrating existing applicative codes requires the support for tasks. Also, we faced in the past with mental toughness about the fact that an OS may not provide for tasks. The most elegant way to integrate "tasks" codes to an automaton engine is to transform them, without their knowledge, to automatons. This means that they get assigned a transition table, which describes the "macro states" of the code, of this "user task". This OS provided transition table is called "task_trans", has the FSM number `AUT_TASK`, and is as follows:

```
static CONST S_trans task_trans[] = /* Transition table */
{
    /* State 0: TASK_UNUSED. The logical way of the automaton is free .....*/
    #define L_TASK_UNUSED 1

    KILL_TASK , kill_task , 0 , TASK_UNUSED , /* The task is done. */

    /* State 1: TASK_CREATED. "create_task()" has run .....*/
    #define L_TASK_CREATED L_TASK_UNUSED+1

    ALL_EVENT , trien , 0 , TASK_CREATED , /* Evt are discarded */

    /* State 2: TASK_STARTED "start_task" has run .....*/
    #define L_TASK_STARTED L_TASK_CREATED+2

    RUN_TASK , run_task , 0 , TASK_WAITING, /* Start task event */
    ALL_EVENT , trien , 0 , TASK_STARTED , /* Any other event is discarded */

    /* State 3 : TASK_WAITING .....*/
    #define L_TASK_WAITING L_TASK_STARTED+1

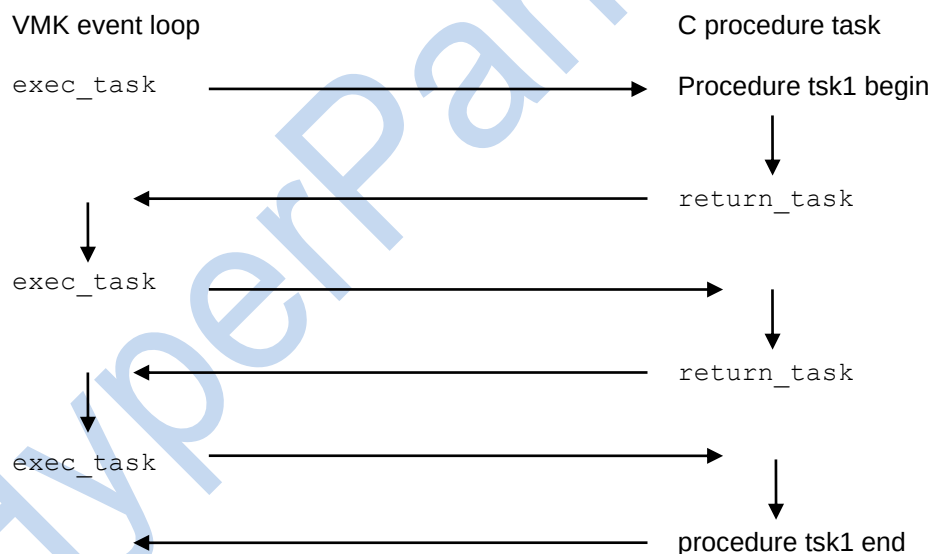
    ALL_EVENT , run_task , 0 , TASK_WAITING , /* All events waited for */

}
```

Each "user task" is associated with a single logical way of the `AUT_TASK` automaton and a single task context. The user task code is one C function, task entry point is the procedure first instruction. The task context is made up of both a generic C structure and of a hardware dependent structure. When the task is created, using the "create_task" function, the C function address is stored in the generic C structure. The hardware dependent structure is opaque and accessed only using two hardware dependent procedures called "exec_task" and "return_task". The opaque data contains the "CPU context", that is the values of the CPU registers, and a CPU stack. The OS provides procedures dedicated to managing the user tasks, that are used by user codes:

<code>create_task</code>	Allocate a user task context
<code>delete_task</code>	Free a user task context
<code>start_task</code>	Start running a user task
<code>suspend_task</code>	Place current user task in suspended mode
<code>exit_task</code>	Exit current user task
<code>get_task_id</code>	Get the identifier of the user task
<code>waitevt_task</code>	Wait until one event from a list is received
<code>readevt_task</code>	Non-blocking read from a wait queue
<code>sendevt_task</code>	Write an event in any task's wait queue
<code>get_task_list</code>	Returns the list of existing tasks

Upon start-up, the application container needs only one stack, used for the VMK event loop, its size is typically 256 bytes. Each time a task is created, using "create_task", a new stack must be allocated, its size is user chosen. So, if we have N created tasks, there are N + 1 stacks. In order to transfer control to a task, the VMK event loop calls "exec_task". The task may call blocking OS functions that in turn will call "return_task". The blocking end is always notified by an event, when this event is received the VMK event loop continues the task with a new call to "exec_task". When the code of the C function reaches its end, the task has completed. A task may also request its end using the "exit_task" procedure.



The only call to the "exec_task" procedure is located within the VMK OS subroutine that executes on FSM transition. The "exec_task" procedure executes an automaton treatment, after having replaced the VMK's stack by the stack of the relevant task. The task code is executed, until it calls "return_task" or reaches the C procedure end. The "exec_task" procedure is therefore called:

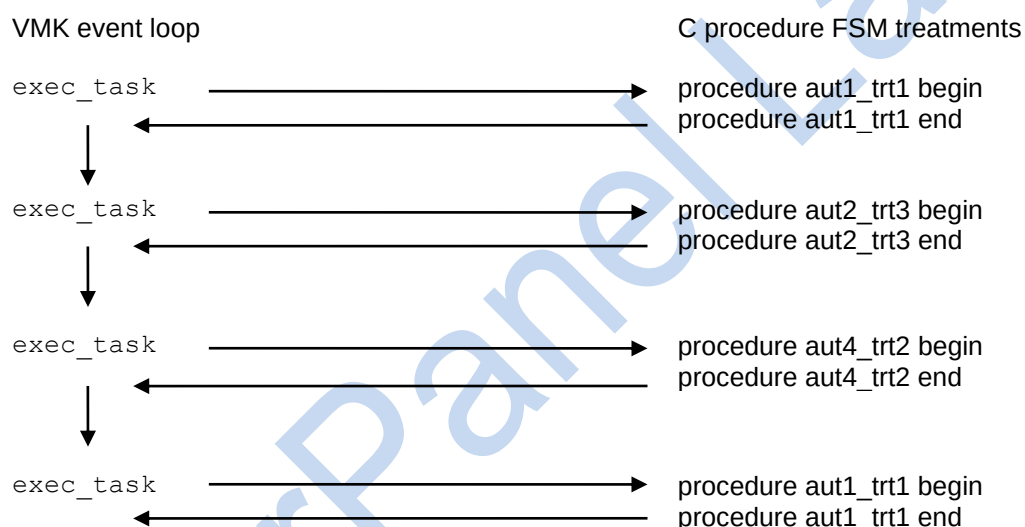
- To start the execution of the treatment. In that case, the task stack is empty. The "exec_task" procedure finds the address of the procedure to run in the "proc" field of the generic task context.
- To resume the treatment where it was suspended before coming to its end, before the automaton C procedure returned. In that case, the task stack is not empty. The treatment address, of the suspended C procedure is still in the "proc" field of the generic context, but it is not useful, as the data stored in the low-level/hardware dependent context is enough to resume execution where it had stopped.

There are two distinct reasons for returning from the C procedure task to the VMK event loop:

- The procedure stops on its own, as it calls one of the "suspend_task", "waitevt_task" or "get_semaphore" procedures. In that case the task stack is not empty upon return.
- The procedure returned. In that case the task stack is empty.

The procedure "return_task" is called by "exit_task()", "waitevt_task()", and "get_semaphore()". It returns to the VMK event loop when the stack of the task is not empty. It replaces the stack of the current task by that of the VMK. The call to "exec_task()" will therefore return to the VMK event loop code. This procedure shall ALWAYS be called after a corresponding call to "exec_task".

The same VMK event loop handles both applicative tasks, each being associated to one logical way of the AUT_TASK system FSM, but also other system or applicative FSM's. Each treatment of each automaton is a "one shot task" that is run once, using always the same couple of task contexts, so generic task context number 0 and hardware dependent context number 0. This implies that only one same stack is needed for all the logical ways of all the FSM,s/automatons.



With any "standard" automaton, so not being AUT_TASK, waits are always wait for events and are always located in the VMK event loop and never in any FSM/automaton treatment therefore no treatment will ever call "suspend_task", "waitevt_task" or "get_semaphore", in other words "return_task" is never called and any treatment always comes to its end. On the other hand, in the case of a task, the VMK event loop code is not aware of the fact that when it calls "exec_task" the executed code is the one for a starting task or for a resumed task. Also, the VMK event loop is no aware of the fact it gets back control because "return_task" has been called or because the C function end has been reached. In the end, the VMK event loop just executes block of codes, whatever the block is a full C function or only a slice of one C function.

A simplified pseudo code for the VMK main loop is given below. This simplified version does not take in account "conditions" nor "hierarchical FSM's" nor "scheduling". We firstly wait for the next event. The incoming event is a C structure that holds, amongst others, the "aut" target automaton number, the "logway" target logical way number and the "code" event code. We extract those event fields that we store in "numauto", "logway" and "code". Then we derive the "numtask" context task number from "numauto": If we are a task, so if "numauto" is AUT_TASK, then we use task number "logway". Else, we are a "standard" automaton/FSM and we systematically use, as explained above, context task number 0. Indeed, "logway" should never be 0 with a task. The VMK has a "task_ctx[]" table of generic task contexts, and we set the "taskctx" pointer to the address of "task_ctx[numtask]". We call "state" the state variable for logical way "logway" of FSM "numauto" and we compute the "tra" address of the transition to be fired (address of an entry of the transition table). This transition depends on the FSM number, that selects one transition table, and from the "state" current state and the "code" incoming event code. The "tra" entry holds the "tra->proc" treatment address and also the next state number "tra->nextstate". We copy the "tra->proc" treatment (C function) address to the "taskctx->proc" generic task context field and then we call the "exec_task" low-level function (hardware dependent) that will retrieve the treatment address from the generic task context. Then the logical way state variable is updated with the "tra->nextstate" new state number.

VMK

```

Main event loop
|   Wait for event
|
|   numauto  = event->aut
|   logway   = event->logway
|   code     = event->code
|
|   numtask  = numauto == AUT_TASK ? logway : 0
|   taskctx  = task_ctx + numtask
|
|   Compute "tra" for (numauto, state, code)
|
|   taskctx->proc = tra->proc
|   exec_task(numtask)
|
|   new state for (numauto, logway) = tra->nextstate
End of loop

```

Three Important remarks have to be made about the very high efficiency of this event loop:

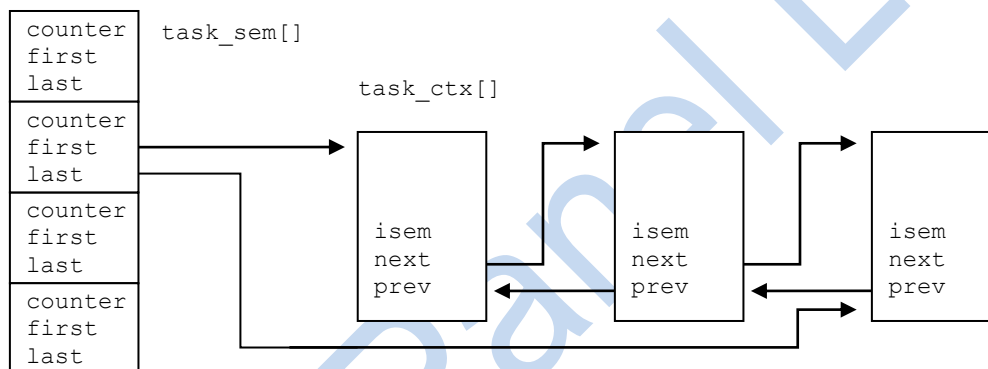
- The "exec_task" function, despite its name, is much more "lighter" than a traditional context task switch: No address space change, no insertion into any linked list of contexts. The function just saves the PC register (CPU Program Counter), saves the VMK stack pointer and sets the SP register (CPU Stack Pointer) to the task stack base address, so only a very few machine instructions. So, our "slow" (high level) activity switch from VMK code to task code is however faster than with classical RTOS's and indeed much faster that it may be with Windows or Linux.
- This same loop executes both sophisticated functions designed and implemented as transition tables and also "simple" code implemented as tasks. Automaton transitions and "slices" of task codes execution are interleaved. Synchronizing a task code and an automaton/FSM is exactly done in the same way automata are together synchronized: the task can send events to any automaton and it can wait for automaton replied events. So, a simple user code may be implemented as VMK tasks whilst other more complex applicative or system VMK function may be strictly and rigorously designed as FSM/automaton.
- As already stated, on a sole stack is needed for task 0, for all the treatments of all VMK automaton/FSM's, because the C procedures are always reaching their end and the stack is always empty after any treatment completed. So, the RAM that is specific to a dedicated logical way is only its context, typically one C structure. If this context is carefully designed, its size may be only some tenths of bytes. We were able, for an IOT MCU with 192 Kbytes of RAM, to build an application with 400 logical ways whilst the most RAM effective RTOS's may not provide for more than 14 tasks.

Semaphores

FSM's treatments never issue any OS blocking call because FSM relies on the principle that all that can be waited for is notified by an event that triggers a next treatment. Also, any access to any shared resource, such as a disk or any other mass storage device, is always done by sending a request event to a dedicated FSM that has the unique access to the device, typically a file manager. Therefore, semaphores are useless within an OS build on FSM's. However, existing codes may use semaphores and for that sole reason the HyperPanel OS provides semaphores and a set of API procedures, that indeed are only intended for use by "tasks" and not by FSM/automaton treatments:

```
create_semaphore  Allocate and initialize a semaphore
delete_semaphore  Free a semaphore
put_semaphore     Increment a semaphore's counter
get_semaphore     Take a semaphore
```

All the contexts of the tasks that are waiting for one same semaphore are queued in the semaphore wait queue, that is in fact a linked list of task contexts. A semaphore is a C structure that basically contains three fields, the "counter" semaphore counter, the "first" number of the first task waiting for the semaphore and the "last" number of the last task waiting for the semaphore:



The semaphore structures are the elements of one "task_sem[]" table. The task contexts, although shown as distinct elements on the diagram, are all elements of the same "task_ctx[]" table. A task identifier contains an index, a task number, which is simply its index in the "task_ctx[]" table. Similarly, a semaphore identifier contains an index in the "task_sem[]" table.

The "put_semaphore" procedure increments the counter, no matter the wait queue is empty or not. It writes a single event to the first queue task, if queue is not empty. In the case where one or more "put_semaphore" calls are successively made on a same semaphore, a sole TASK_SIGNAL event is put, when counter goes from 0 to 1.

The "get_semaphore" procedure, when getting a busy semaphore, performs the following three actions:

- Decrement the counter by 1.
- Extract the task context from the semaphore's wait queue.
- If the counter is still strictly positive after decrement, and if further the queue is not empty, a TASK_SIGNAL event is put for the task that is the new head of link.

The "kill_task" procedure checks if the destroyed task was not waiting for the semaphore. If it was:

- The counter value remains untouched;
- The task context is removed from the semaphore's wait queue.
- If by chance, the task was first in the list, the counter is strictly positive and the queue is not empty, a new TASK_SIGNAL event is put for the new first task in the queue.

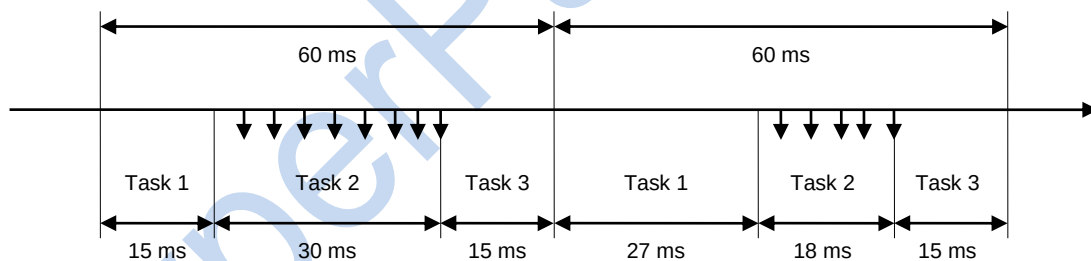
To sum up, a "put_semaphore" only writes a single TASK_SIGNAL event to the task blocked waiting on the semaphore the longest, even if multiple semaphore tokens are given, and multiple tasks are waiting. The task, when it resumes, writes a new TASK_SIGNAL event if need be. This approach does require handling the case where the first task in the queue gets destroyed before the TASK_SIGNAL is handled. A killed task cannot send an event. The "killer", the "kill_task" procedure to be precise, is in charge of sending the new TASK_SIGNAL event instead.

Time slicing Scheduler

Automatons/FSM's do not need time slicing, in fact one of the main benefits of using an automaton engine is that the automaton engine is the activity switcher and switching performances are better than any classical scheduler. As long as no treatment has a "long" duration, time slicing can be avoided with the fundamental consequence of a considerable logical benefit: semaphores are useless because each automaton treatment is a transaction and whenever any resource, such as a disk, has to be shared then all requests are sent to one same FSM that queues and manages all the service requests. So, time-slicing is never needed with FSM designed software, whatever FSM are "system" or "user". When the OS manages tasks that "frequently" do I/O's, time-slicing is still not needed because activity switch may occur at each new IO requests. But there is a very special but noteworthy case where we have an architectural problem. Many open source Operating systems are suffering of both very poor IO performances and security flaws. A radical but simple solution is to strip the open source OS kernel from all its drivers, protocols and network services and redirect requests to a VMIO I/O container. The hosted kernel is seen by the VMK as one VMK task and indeed the hosted kernel tasks are not visible to the VMK.

An issue has to be solved: Once CPU control has been given to the hosted OS kernel, its scheduler never returns and so VMK and VMK tasks are no more executed. A solution is to use a hardware timer to retrieve CPU control, but this not very simple if the timer duration is not correlated to the internal hosted kernel activity. A better and simpler solution is to very slightly modify the hosted OS scheduler in order that it often calls the HyperPanel OS "suspend_task" function, that "decides" to continue the hosted kernel or to return to the VMK kernel, using the low-level "return_task" function. This tiny hosted OS scheduler modification is very simple, we just add one call to "suspend_task" but indeed this cannot be implemented if the hosted scheduler source code is not available.

The VMK time-slicer defines cycles of a constant duration, 60 millisecond each with the example that follows. At each cycle start, the VMK allocates to each running task an allowed CPU time and those values are stored in the task contexts. In the next example, we have 3 ready to run tasks and the timer slicer stores 15, 30 and 15 before the first 60 milli-second cycle and 27, 18 and 15 before the second 60 milli-second cycle. On the diagram, Task 2 is supposed to be the hosted OS, its scheduler repeatedly calls "suspend_task". Those calls are depicted with vertical arrows. At each call, "suspend_task" checks if the allowed CPU time has elapsed and when this occurs then "suspend_task" calls "return_task" that returns control to the VMK main loop and the VMK main loop then gives control to Task 3.



The VMK time-slicer function is to compute, at each cycle start, the allocated time quantum for each ready to run tasks. For this computation, two classes of tasks, "real-time" or "standard" are defined. Any task that executes short treatments triggered by events will be "real-time" whilst a task that is CPU consuming will be a "standard" one. The VMOS or an external RTOS will typically be "real-time" whilst a Linux kernel will be "standard".

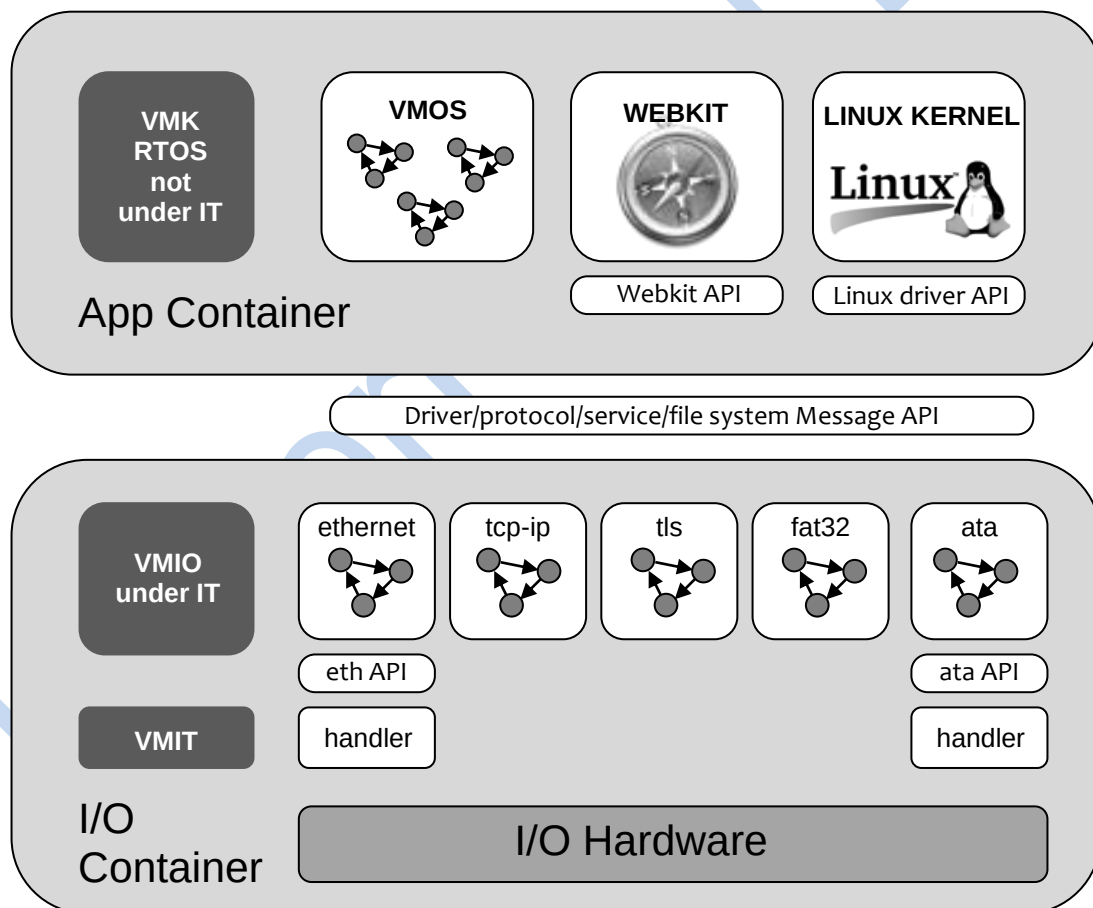
An essential characteristic of the VMIO/VMK/VMOS combination is that it offers a much better response time as seen from the outside than those gotten with other "real-time" operating systems, and of course even better so compared to non "real-time" embedded operating systems, such as UNIX, Linux, or Windows. A key point for the good response times come from the choice that handling new events is less urgent than that of old events. As soon as we have an operating system dedicated to real-time, in which all the application are defined as automatons, it becomes clear that all the treatments are direct or indirect treatments of I/O container events. When we consider additionally that the operating system is not meant to run "long" treatments, such as in scientific computation, we realize that it is better to finish handling the current events before starting with new ones. We will therefore strive to finish executing the "real-time" C procedures that were interrupted before going to extract new events from the internal wait queues. However, the higher priority granted to the event treatments over the event acquisition should not be extended to C procedures which loop, as is done in the Linux kernel, or the non-looping processing would be starved by the looping processing. This is the reason why the VMK therefore distinguishes between two categories of tasks, "real-time" and "standard".

The VMK's main loop organizes the execution of the different tasks according to the four "essential rules" found below:

- Finish the suspended treatments of tasks marked as "real-time" first. Events won't be dequeued from the internal wait queues as long as there remains at least one "real-time" treatment C procedure which was interrupted.
- Then, empty the internal wait queues which are associated with "real-time" tasks.
- If the two above conditions are both met, the VMK can run the external wait queue reading procedure "waitevt_fae". The read event triggers the execution of an automaton treatment, a new C procedure, which is not necessarily from "real-time" task.
- Ensure that the interrupted non real-time tasks have regularly access to CPU time.

The HyperPanel spread OS two containers architecture

The HyperPanel OS architecture is made of two separated containers, an Application container that contains applicative software and an I/O container that contains all the drivers, protocols, services and file systems. Below is the example of a Home Gateway OS architecture.



The application container embeds a first application operating system VMOS which contains interpreters built as finite state automatons. The VMOS executes the *middleware* of the *Media Home Gateway*. Second, there is a HTML renderer, for which an abstraction layer was implemented, which the input/output requests of the renderer are converted in events deposited to the I/O container. Third is a Linux kernel stripped of all of its drivers, protocols, services, and file systems. An abstraction layer converts all Linux input/output requests to request event deposits.

The VMIO input/output monitor is a finite state automaton engine, which executes automata that are independent of the hardware. The drivers, the protocols, the services, and the file systems are all implemented as automaton transition tables. There is no notion of task at this level, nor of semaphores which makes going from one function to the next one a latency-free operation, which in turn allows for high bit rates even with multiple concurrent flows. In order to achieve zero-latency, the VMIO uses a single stack for itself and all its automata, a single address space and a single set of registers. Every driver, protocol, service or file system component is made of a transition table and of a set of C procedures, which are all transactional treatments for transitions. These C procedures, these automaton transition treatments, when they need to send commands to a device under their control, will directly call, without any context switching, the C procedures which implement the *Hardware Abstraction Layer* for the device. There is a one to one correspondence between high-level drivers and the specifications of the low-level drivers. All of the code for the VMIO, including the VMK, is written in C, and is strictly portable without any modification or conditional compilation. The VMIO provides a pipe mechanism, which allows a unidirectional data transfer from automaton to automaton. A pipe is configured and started using request events, and then the client doesn't need to intervene anymore for the data to flow from automaton to automaton. Thus, the various data flows occur without any context switching, and in particular no copy and no address space change. If for instance the application configures a pipe ETHERNET => TCP => HTTP => FAT, high speed download feature is realized.

In this architecture, the TCP protocol, the HTTP service and the FAT file system are executed within the monolithic VMIO monitor, and not within the VMK, which runs in the app container. This is a major trait of the HyperPanel OS architecture. Some have noted that RTOS's are too slow for demanding I/O tasks and have tried to develop faster RTOS's by reducing the context switching time. This is pointless, as it reduces the functionality of the RTOS, without even truly reaching the goal of a sufficient speed-up. By having both an input/output monitor and separately an RTOS (the VMK), we can both perform I/O faster than an RTOS would, while also having a feature-rich RTOS if needed.

The sub-OS named VMIT is the one that manages the "Interrupt Requests Hardware" as depicted in "Typical hardware needed for Interrupt Requests support". It fully implements nesting of increasing priorities Interrupt Requests. The VMIT requires that the ILC provides at least 8 priority levels (This is even available with the smallest IoT's MCU). When any interrupt is raised then:

- 1) The corresponding handler is immediately started (0 cycle wait, but for the intrinsic CPU latency, typically 2 to 3 CPU clock cycle) if no interrupt is currently being processed. If we have already a lower priority interrupt being processed then this one is stopped and nesting occurs.
- 2) The interrupt handler always fully processes the interrupt, in other words deferred Interrupt Request processing is never used. Therefore, for each end of data block transfer the interrupt handler directly writes a software event in input queue number 1 of the VMIO. The VMIO is an FSM engine. The subroutine that writes the events also writes in a register of the ILC in order to activate (by software) an interrupt of lower priority than all the devices Interrupt Request Priorities. If the lowest ILC priority is 0, then priority 0 is only used with this "software generated" Interrupt Request and all the devices Request Interrupts are using priorities from 1 to 7.
- 3) At the very moment the IRET execution for the interrupt handler is executed then the EIRQ line is asserted. The ILC immediately "discovers" (0 cycle latency) that the "software raised" Interrupt Request is active and immediately (0 cycle) the ILC puts the IRQ line in its active state.
- 4) The interrupt handler for the software raised low priority interrupt is in fact the VMIO entry point. This means that **the latency between the exit of the interrupt handler and the entry of the corresponding "high level" driver is reduced to 0**: One cannot do better. Using a driver as an RTOS task would imply a switch of context and therefore a latency. One can claim that "task switch" has been highly optimized but within the I/O container there is no context switch (but for the stack) between the interrupt handler and the high level driver. In other words, there is no possibility to be faster in any way.
- 5) When high level driver (one C procedure, extracted from the FSM transition table) has been executed it will, for example send an event to a protocol. Again, the writing of this event will re-activate the software raised interrupt. So, when the driver's C procedure completes, the VMIO entry point (the handler of the software raised interrupt) is immediately called (2 or 3 CPU cycle). This means that the latency between the end of execution of the driver and the start of execution of the protocol the data is intended to is also 0. This is absolutely unique in the world of Operating Systems and again there is no way to be faster.

An application container indeed sends requests to the local I/O container but it can also simultaneously send requests to others remote I/O container, using a LAN. As a result, the HyperPanel OS architecture can be truly considered as a network spread Operating System Architecture.

Conclusions

The I/O container makes full use of “Interrupts Request Hardware’s” including an 8 level’s nesting Request Interrupt scheme which is something that very few OS’s are implementing. But rather than “optimizing” existing architecture’s, a fundamental and deep conceptual work has also been done: We took the time (some decades...) to study if one could reach the ultimate hardware I/O performances (the asymptote). This means that, whatever the kind of I/O hardware we manage, the HyperPanel architecture will always and systematically provides better performances, because we reduced to 0 absolutely **all** the software latencies and **not only** the “interrupt to interrupt handler” latency but **also** the “interrupt handler to driver latency” **and** the “driver to protocol/file” system latency. On one hand, suppressing all latencies assures that I/O hardware is never idle and, on the other hand, using DMA increases maximum throughputs, so great efforts have been also made in order to use DMA transfers whenever possible.

“... and once again, as with the ENIAC, computation rate is not the performance-limiting factor, rather it is still the communication, the I/O, the setup for the computation. It seems that communication science may be at the heart of the problem after all.” (E. Davidson, 1999)

Our new OS Architecture is the long awaited answer to the issue raised in 1948 (referencing the 1948 article *The Eniac*) and correct Request Interrupt Handling is one of its key points, thanks to the VMIT and the VMIO.

The foreground code is wrapped in a second container we call the applicative container. This container is built around an event loop we call VMK. This event loop executes traditional tasks but also system and user FSM. As an application container may simultaneously request services to the local but also to remote I/O container, the OS architecture can claim to be a LAN spread Operating System Architecture.

The finite state machine engine is the core component of both the input/output container and the application container. At the end of this article, it makes sense to ask whether the claimed performances level is achievable, both in terms of speed and reliability. In other words, is the input/output problem that A.W. Burks was still calling out in 1999 truly solved?

In order to validate what was described here, the two containers architecture has been implemented for *Media Home Gateways*. The role of these machines is to serve video and audio flows as well as still pictures to all the screens in a home: TVs, tablets, and smartphones. The I/O container integrates more than 60 automaton: drivers, protocols or services. It can read or write up to 10 simultaneous HD video flows, starts streaming a video in 2 tenths of a second, handles home automation protocols, receives 4 TV multiplexes, and streams 4 TV programs over WiFi. All of this at the same time, using a single 440 MHz 32 bits processor, which runs both an input/output container, but also an application container which contains Webkit, a CPU hungry HTML renderer.

These performances go well beyond the claims of the designers of the used SoCs. The input/output container can be used with any type of application container, including those hosting an open source Operating System, RTOS or Linux, giving these systems a high level of input/output performances, a reliability and security level out of the ordinary. We will never have the chance or the honor of presenting these results to A.W. Burks (1915-2008), but we hope that designer of future operating systems will make the best use of this architecture.

ANNEX Linux and HyperPanel Media Home Gateway comparison

The HyperPanel OS is claiming unrivaled level of performance. To demonstrate those, we ported the HyperPanel OS on a digital TV set-top box that we intentionally designed using one very low-end digital TV chipset (a single 440 MHz CPU core) to which we added a 5400 rpm hard disk, a WiFi access-point, and 4 tuners. The resulting product is able to simultaneously receive four 58 Mbit/s DVB broadcasted streams, one 25 MBit/s IP stream, read/write ten 12 Mbit/s video streams, broadcast via WiFi three 12 Mbit/s video streams. And while all of this I/O activity is going on, the display of the UI remains fully smooth and responsive.

The question how this level of performance compares to what can be provided by others OS's, typically Linux, is often raised. But we have here an unexpected difficulty: we were not able to find any other set-top box running Linux and using the same widely spread 440 MHz CPU (STi7105) we could compare to. The most advanced STi7015/Linux set-top box we found is fitted with one 5400 rpm hard disk, a WiFi client and 2 tuners. This product is able to receive only **one** 58 Mbit/s stream **or** one 10 Mbit/s IP stream, while reading or writing only **one** 12 Mbit/s video stream. Video broadcast via WiFi is not supported. UI is slow and sometimes choppy.

Not having anything we can compare the OS to is rather frustrating. In that case, one is fully justified to ask for detailed descriptions about how things run inside the HyperPanel OS, how it can provide such higher performance. The HyperPanel OS is the result of 30 years of research in the fields of OS architecture and effective implementation of Finite State Machines processing, and the complete and detailed explanation takes several thousands of pages. The purpose of this annex is to provide however a very deep description of the inner workings of the HyperPanel OS, concentrating only on points that are the most important ones when it comes to performance and energy savings. We have chosen to describe here in detail three scenarios which are:

- Serial communication controller use
- Hard disk management for video recording
- Access and display of one HTML document

Description of Studied OS Architectures

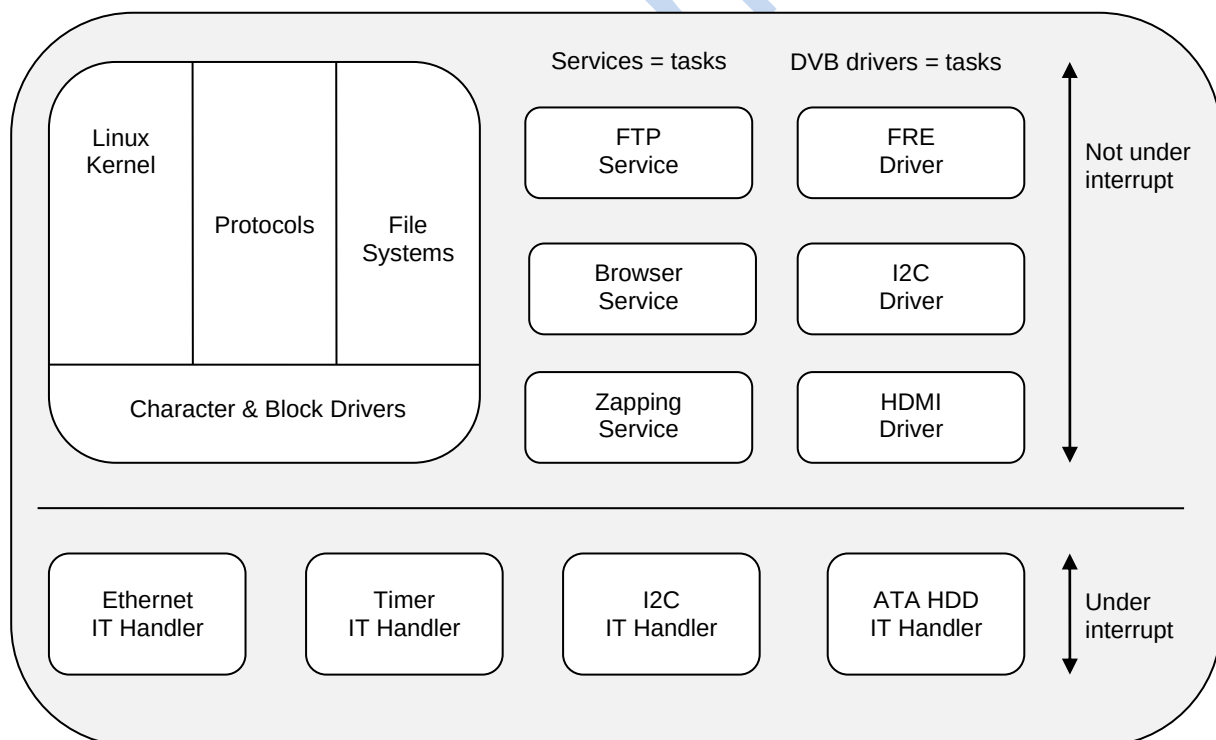
Studied OS Architectures

In the present article, we are going to compare two OS's architectures. The first one is a standard Linux architecture whilst the second one is a two-container OS architecture, which is the one used by the HyperPanel OS. Indeed the two architectures have been simplified, keeping only the architectural features we need for the sake of our explanations.

Linux Set-top Box Architecture

Chipset manufacturers are used to develop and provide to their customers what they call "drivers", those being dedicated to their own hardware. Under this name, one can find two different kinds of software:

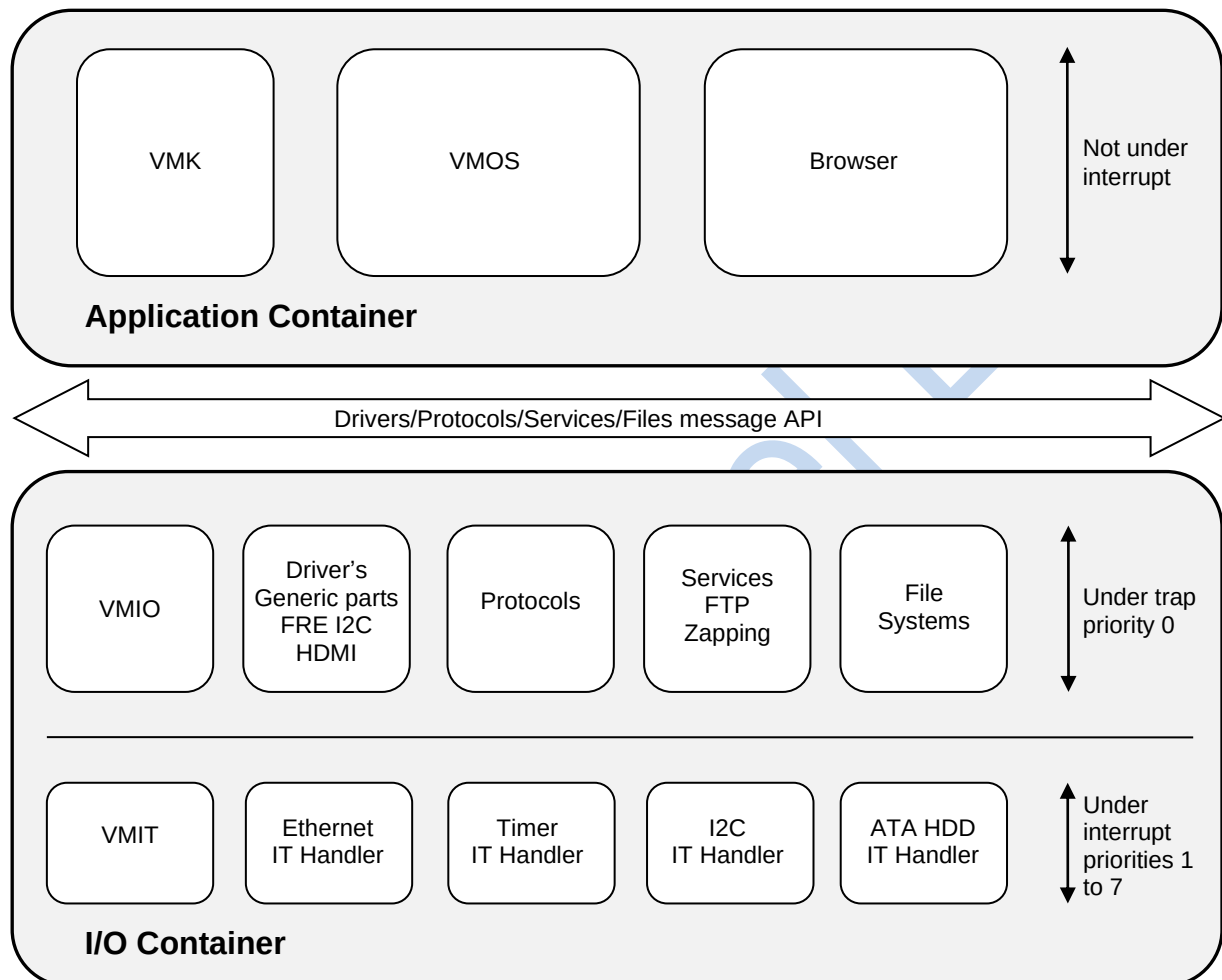
- When the Linux Kernel includes protocols (TCP or UDP) and file systems (EXT, FAT32) then the manufacturer must provide a Linux driver for the networks adapters (Ethernet) and for the disk controller (ATA). This means that the driver code is statically linked to the kernel code, or loaded dynamically into kernel space by system initialization scripts (.ko files).
- When one peripheral device is normally used in conjunction with a protocol or a control logic that does not exist in the Linux kernel provided functions, manufacturers are providing Linux tasks that are each the aggregation of the protocol/control logic and hardware dependent parts. They also call those parts "drivers". We will not discuss here whether the use of the term "driver" is suitable in that case, nor if this way of mixing hardware dependent and hardware independent code is justified or not.



The above Linux Architecture is used by all digital TV chipset manufacturers. The front-end "driver", the I2C "driver" and the HDMI "driver" are user space tasks containing peripheral device hardware dependent code and implementations of protocols and control logics that are to the contrary hardware independent.

Two-Container OS Set-top Box Architecture

The HyperPanel OS is split in two containers, application and I/O containers. All drivers, protocols and file systems are located inside the I/O container. All drivers have an especially defined and described Hardware Abstraction Layer and so each driver has a generic part that is hardware independent code (Front-End, I2C, HDMI) and is always implemented as an FSM (automaton).



The VMIO is an automaton engine. It receives events. Each event contains the reference of its target automaton. The VMIO extracts from the selected automaton transition table the address of one C procedure that is the processing code of this event and calls that procedure as a subroutine. So, there is no context switch at all. If the C procedure writes any event into the VMIO waiting queue then immediately after the end of the procedure the VMIO will call one C procedure of the automaton the event has been sent to, using the same stack and the same address space. The interrupts are never masked, meaning that any hardware interrupt will interrupt any VMIO C procedure with a null latency. Interrupts processing can be nested, so each interrupt level has a dedicated stack, but address space is still the same. When any interrupt handler has something to notify then it writes an event in the VMIO waiting queue and also raises by software the trap of priority 0. As interrupt handlers have always a higher priority (1 to 7) the IT handler execution continues. At interrupt handler termination, when the interrupt is acknowledged then the priority comparator immediately raises again the main CPU interrupt line because the trap request is active. The handler of trap 0 is the VMIO entry point and so the event is immediately taken in account with no delay (one CPU clock pulse).

Some security experts may consider that such a monolithic architecture is "weak" regarding the security concerns. But using FSM enables both for tremendous code reduction and full test coverage realization and this eliminates bugs and failures so drastically that, in the end, even a "nano-kernel" implementation is always a less secure solution.

First Scenario: Serial Communication Controller Use

Scenario Synopsis

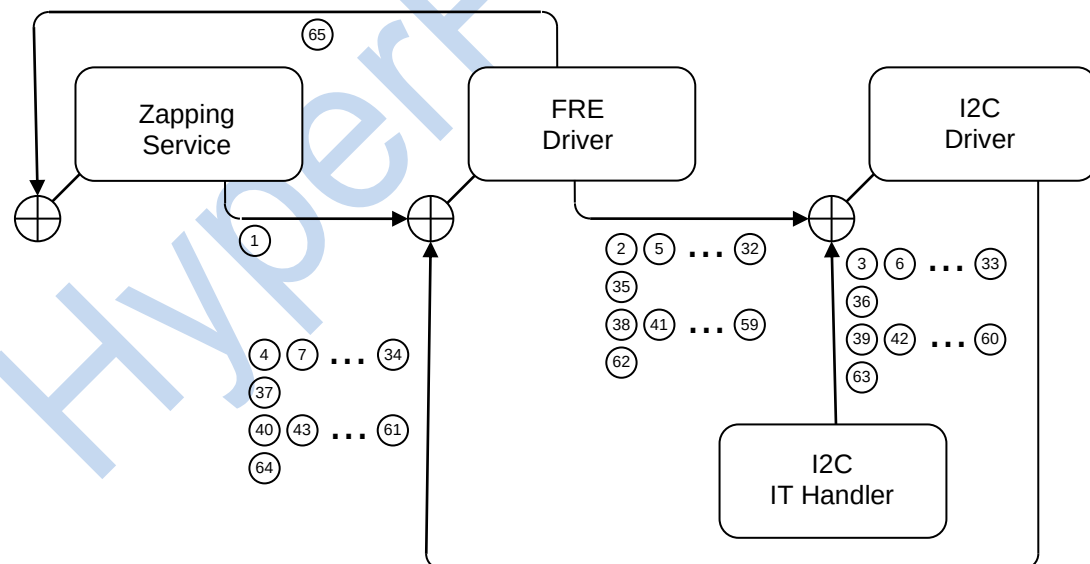
Serial communications have always been massively used by computers and still are. Asynchronous RS232, synchronous RS232, USB, Ethernet, WiFi and I2C are serial communication links. The I2C bus is a 3 wires bus that enables to interconnect several masters and a group of slaves, typically one CPU acting as a master to several slave devices. Slave devices can be EEPROM's, TV tuners and demodulators, front panels controller. The VGA 15 pins cable contains an I2C bus, and so does the HDMI cable. The extremely simple first synopsis is the "opening" of one TV Front-End, which will setup one tuner and one demodulator. The slave tuner and the demodulator are connected to the master main CPU via an I2C bus. We suppose here that the front-end driver has to:

- Send 11 parameters to the tuner, so 11 couples of bytes (register number, register value)
- Read sequentially 13 bytes from the tuner
- Send 8 parameters to the demodulator, so 8 couples of bytes (register number, register value)
- Read sequentially 14 bytes from the demodulator

Firstly, we are making the assumption that in both cases, Linux and then our two-container OS, drivers are doing the same sequence of I/O's, as described just before. Our intent is to compare the switching activities that are done.

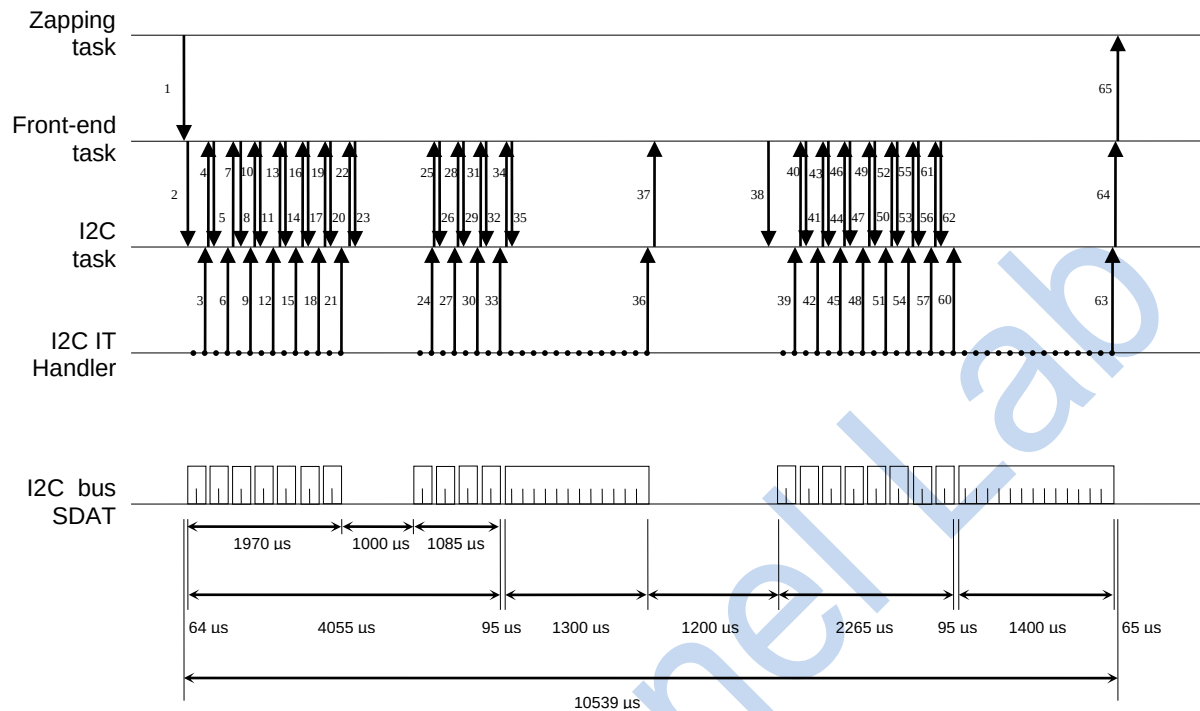
Linux Case

The scenario involves three Linux tasks: the Zapping service task, the Front-End driver task (FRE) and the I2C driver task (I2C). Each task has a dedicated semaphore, and is waiting on it for a POST when it has nothing to do. Each time a POST is done the task that is waiting on the corresponding semaphore goes to the running state or is moved to the CPU waiting queue. The chosen "open driver" scenario will need 65 POST's and so 65 task switching are done. The following diagram depicts the three involved tasks, the associated semaphores and the I2C interrupt handler. All the POST's are depicted as circles containing the sequence number.



- | | | | |
|---------------------------|----------------|--------------------------|----------------|
| Open request | (1) | | |
| 11 writes of 2 bytes each | (2) ... (34) | 8 writes of 2 bytes each | (38) ... (61) |
| 1 read of 13 bytes | (35) (36) (37) | 1 read of 14 bytes | (62) (63) (64) |
| | | End of open notification | (65) |

The following diagram depicts the time ordering of the POST's (numbered vertical arrows). It also shows the interrupt occurrences (points on the I2C IT Handler line) and what one can see on the I2C bus, using for example an I2C bus analyzer (I2C bus SDAT line):

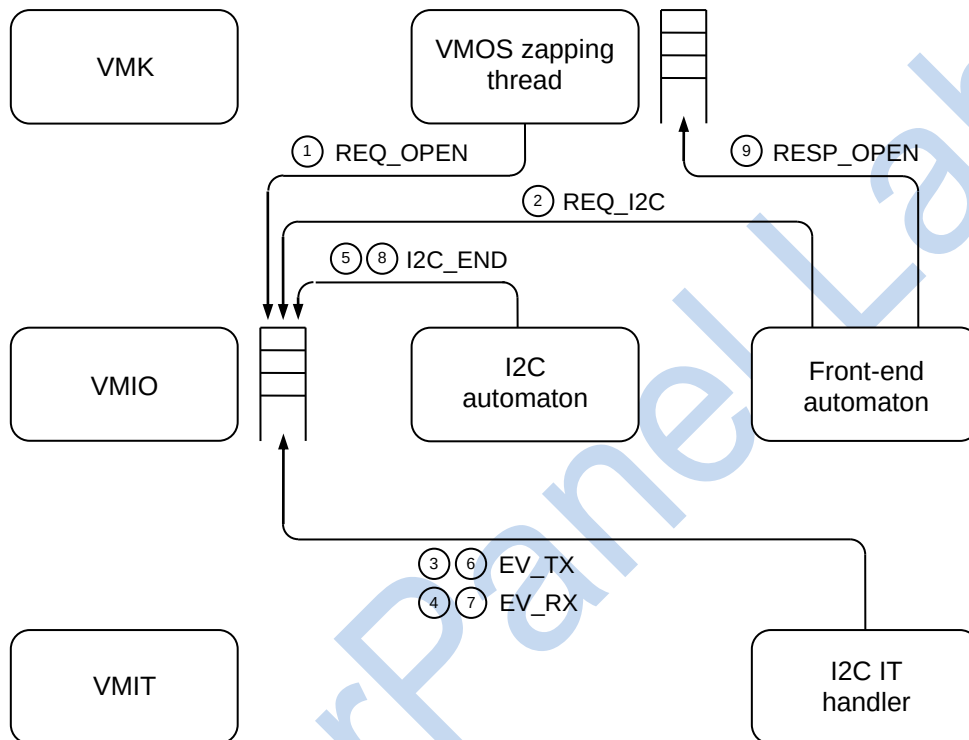


Because of the acknowledge bit, each transmitted byte needs 9 pulses of the I2C clock (SCLK). If this clock frequency is 90 kHz, then 100 µsec are needed for each byte. In this scenario, $22+13+16+14 = 65$ bytes are transmitted and so the I2C bus is busy during 6 500 µsec (we do not take here in account I2C start and stop patterns). We will assume that our CPU executes 250 M of instructions per second. We will assume that the processing of the POST request will require around 250 CPU instructions, so 1 µsec. When a task has completed its job, it will execute a GET on its semaphore in order to become unscheduled and we assume that this has a cost of 250 CPU instructions, so 1 µsec. In the case that another next task has to be started, the added cost to the GET request is 7500 CPU instructions more, so 30 µsec.

- The zapping task issues a POST (1) to the semaphore of the Front-end task to notify the "open driver" request and then blocks itself by issuing a GET on its own semaphore. The Front-end task is scheduled. This needs $1 + 1 + 30 = 32$ µsec. The Front-End task issues a POST (2) to the semaphore of I2C task and then blocks itself with a GET on its semaphore. The I2C is scheduled ($1 + 1 + 30 = 32$ µsec). We spent here $32 + 32 = 64$ µsec of CPU time.
- The I2C task requests the sending of 1 byte. 100 µsec later the first byte is sent, and the first interrupt occurs; the interrupt handler sends the second byte. 100 µsec later the second byte is sent and the second interrupt occurs. This took 200 µsec with almost no CPU time.
- The interrupt handler has no more byte to send and it issues a POST (3) that schedules the I2C ($1 + 30 = 31$ µsec). The I2C does a POST (4) to the Front-end semaphore and then issues a GET. The Front-end is scheduled ($1 + 1 + 30 = 32$ µsec). The Front-end issues a POST (5) to the I2C and then a GET. The I2C is scheduled ($1 + 1 + 30 = 32$ µsec) and the transmission of the third byte starts and this happens $31 + 32 + 32 = 95$ µsec after the end of transmission of the second byte.
- We suppose that 7 couples of bytes are transmitted, we have 95 µsec of CPU between each couple, for a duration of $200 \times 7 + 95 \times 6 = 1970$ µsec. We suppose that other tasks have to be scheduled and that there is no activity on the I2C bus during 1000 µsec. Then the remaining 4 couples are sent, so $200 \times 4 + 3 \times 95 = 1085$ µsec. We have 95 µsec of CPU before the beginning of the 13 bytes read starts, and this will take $13 \times 100 = 1300$ µsec. (The handler does the job alone.) We suppose that other tasks have to be scheduled for 1200 µsec, then the write of 8 couples takes $8 \times 200 + 7 \times 95 = 2265$ µsec, then 95 µsec of CPU, then the read of 14 bytes takes $14 \times 100 = 1400$ µsec. Then we have 1 µsec (63), then $30 + 1 + 1$ µsec (schedule I2C, (64) and GET), then $30 + 1 + 1$ (schedule FRE, (65) and GET).

Two-Container OS Case

The scenario involves one VMOS thread, which is the Zapping thread. We use two automatons of the I/O container, the I2C automaton and the Front-end automaton, both of them are run by the VMIO, which is the automaton engine of the I/O container. We have also the I2C controller interrupt handler. We have one event waiting queue for the Zapping thread (each VMOS thread has its own queue) and one event waiting queue for the VMIO (whatever the number of managed automatons is). The chosen "open driver" scenario will need 9 events. The following diagram depicts the VMOS zapping thread and its waiting queue, the two VMIO automatons and the VMIO event waiting queue. All the events are depicted as circles containing the sequence number.

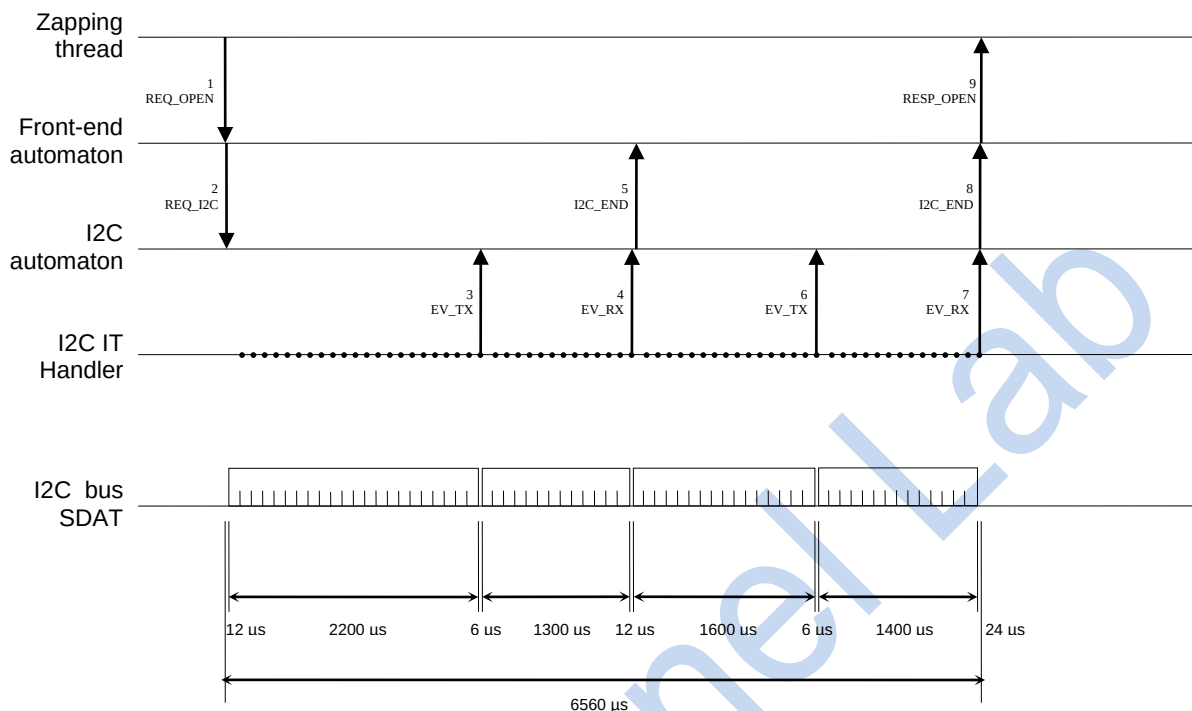


Open request	①	End of 16 bytes write	⑥
Multiple I/O request	②	End of 14 bytes read	⑦
End of 22 bytes write	③	Notification for FRE	⑧
End of 13 bytes read	④	End of open notification	⑨
Notification for FRE	⑤		

The I2C driver is able to handle multiple I/O requests. In our case, (2) conveys 6 elementary requests:

- Send 22 bytes
- Receive 13 bytes
- Send an I2C_END event to FRE
- Send 16 bytes
- Receive 14 bytes
- Send an I2C_END event to FRE

The following diagram depicts the time ordering of the events written into the VMOS queue and the VMIO queue (numbered vertical arrows). It also shows the interrupts occurrences (dots on the I2C IT Handler line) and what one can observe on the I2C bus (I2C bus SDAT):



As previously, each transmitted byte needs 9 pulses of the I2C clock (SCLK), so 100 μsec per byte. 65 bytes are transmitted and the I2C bus is busy during 6 500 μsec. We assume that our CPU executes 250 M of instructions per second. We assume that writing an event in a queue requires 500 CPU instructions, so 2 μsec. Writing an event in the VMIO waiting queue raises the trap and the next executed machine instruction is trap handler which is the VMIO entry point. The extraction of the event from the VMIO waiting queue, then the search in the transition table and the call as a subroutine of the C procedure that is extracted from the transition table require 1000 CPU instructions, so 4 μsec.

- The zapping thread puts a REQ_OPEN (1) event in the VMIO queue and raises the trap (2 μsec). The VMIO extracts it and calls one C procedure of the FRE automaton (4 μsec). This one writes a REQ_I2C event (2) in the VMIO queue (2 μsec). The VMIO extract it and calls one C procedure of the I2C automaton (4 μsec). The I2C automaton sends the first byte of 22. A total of 12 μsec of CPU time has been used.
- We have 22 interrupts corresponding to the end of transmission of the bytes, so $22 \times 100 = 2200$ μsec. For the 21 first interrupts, the handler writes the next byte into the hardware transmitting register and this needs a negligible amount of CPU time.
- For the 22nd interrupt, the handler sends a EV_TX event (3) to the I2C automaton (2 μsec). The VMIO extracts this event and calls one C procedure of the I2C automaton (4 μsec). This procedure asks for a 13 bytes read. A total of 6 μsec of CPU time has been used.
- We have 13 receive interrupts, so $13 \times 100 = 1300$ μsec. For the 12 first ones, the handler just starts the SCLK signal (9 pulses of SCLK) and this needs a negligible amount of CPU time.
- For the 13th interrupt, the handler sends one EV_RX event (4) to the I2C automaton (2 μsec). The VMIO extracts it and calls one C procedure of the I2C automaton (4 μsec). This procedure sends one I2C_END event (5) to the FRE (2 μsec) and then starts the sending of the 16 bytes frame. The VMIO extracts the event and calls one C procedure of the FRE automaton (4 μsec). A total of 12 μsec of CPU time has been used.
- Then we send 16 bytes and this will take $16 \times 100 = 1600$ μsec with almost no CPU use. Then we use 6 μsec of CPU because of EV_TX (6). Then we read 14 bytes and this needs $14 \times 100 = 1400$ μsec. The 3 last events are using a total of 24 μsec of CPU time.

Discussion

The previous evaluated results can be summarized as follows:

	Linux	Two-container OS
Hardware activity duration	6 500 μ s	6 500 μ s
CPU usage	1 840 μ s	60 μ s
Total activity duration	10 540 μ s	6 560 μ s
Synchronization system calls	65	9

We want here to firstly point out the two following main points:

- The I/O performance of the two-container OS is in that case very near to the hardware limit. The total duration is 6 560 μ sec instead of 6 500 μ sec, meaning 99% use of the physical bandwidth. With the Linux implementation, we have 10 540 μ sec instead of 6 500 μ sec, so 62% use of the available bandwidth. We mean here that the comparison between two OS's on the same hardware should not be done by comparing directly the two throughputs values, but rather the respective percentages use of the physical available bandwidth.
- The two CPU usage values are very different, 1 840 μ sec and 60 μ sec. The main explanation for this is that the two-container architecture simultaneously and dramatically reduces both the number of Synchronization system calls and the duration of each and so the product value of those.

Those evaluated results have been in the past compared to measures. We used an evaluation board that was delivered with a complete set of Linux software, all the source code being available. We ported the HyperPanel OS on that same board and so we were able to make measures with both the OS's. It turned out that:

- Regarding the HyperPanel OS, CPU usage was around 80 μ sec. In our previous computations, we had considered that the CPU usage of the I2C handler is negligible, and also for the C procedures of the automations and this is not true. But regarding the total duration, measured value was around 6530 μ sec, despite a higher CPU usage and the presence of start and stop patterns. The reason for this is that the computations did not take in account the existence of hardware FIFO inside the I2C controller, and so CPU processing is done in parallel while the I2C controller is receiving or sending.
- For the chipset manufacturer provided Linux software, the total duration as measured with the I2C analyzer was much more than the expected value (around 120 msec, and also changing from test to test). This is detailed just below.

A short study of the I2C captured messages and small investigations into the Linux and drivers source code did provide logical explanations about the 120 msec duration instead of the 11 msec expected value:

- The I2C interrupt handler did not process the I2C controller interrupts. Instead, the Linux "tasklet" mechanism was used which is not a reasonable choice when there are 10 000 interrupts per second.
- Due to the previous choice, the I2C clock speed had to be slowed down (half) in order to avoid I2C controller FIFO's overruns.
- The number of I2C messages was far above what one can expect. The study of the contents of the I2C messages showed a surprising way to cope with bit-fields. When one same byte contains more than one value, the drivers are systematically reading the byte and then writing it back, and this for each bit-field. So if a byte contains 3 values, this byte is read 3 times and written 3 times.
- Due to the way of programming, using many sub-routines, the same parameter can be written many times with the same value, which is useless. But software engineers can quickly forget that the subroutines they did not write can also do Input/output, and they are not aware of the useless time costs.

At this point, one can argue that the two last points are not correlated to the OS. If we reduce an OS to its kernel this is true. But in the end chipset manufacturers do provide such drivers, and those are used. We consider that correctly designed drivers and protocols have to be provided along with an OS and we will not discuss whether they can be or not semantically considered as OS components.

Second Scenario: Hard Disk Management for Video Recording

Scenario Synopsis

Digital set-top boxes with video recording capabilities are using mechanical Hard Disk Drives (HDD) rather than NAND Flash because of a cheaper cost per MByte. Typically, video recorders are using 500 or 1000 GBytes Hard Disks spinning at 5400 rpm. Those have a typical transfer rate of 150 MB/sec and a mean access type of 1 msec, so 1000 accesses per second (arm moves, not including read/write sector duration). When the HDD is an external USB drive, this one may be connected to a PC and thus the file system should be FAT32 or NTFS. As there is no official specification of NTFS, FAT32 is systematically used for such use.

The fact that file system layout is FAT32 absolutely does not mean that all the OSs using it will have the same levels of performance; in fact huge differences of performance can be found. Long story short, the root cause for those differences is the number of moves per second of the arm of the HDD that are requested. Indeed, the fewer arm moves are requested, the higher sustained throughput is. On top of that, fewer arm moves also means a dramatic increase of the drive life duration (~ x5) and makes a fan useless.

Typical FAT32 Implementation

Typical FAT32 implementations are optimized for disk space savings. Disk clusters have a size of 4 kB, so 8 disk sectors. The File Allocation Table (aka. FAT) is updated for each allocated cluster and so the creation of each new data cluster implies at least two disk accesses (write FAT, write data cluster), a maximum of 500 new clusters/second, and so a maximum record rate of 2 MByte per second. If the File System management code is located in the kernel space of a Linux or Windows OS, one will randomly fall down under this value; this means that the recording of only one Live High Definition program (~2 MB/sec) will fail. The record of one Simple Definition program (~ 1 MB/sec) will succeed, but one record and one play-back simultaneously will not be possible. We have here a level of performance of $2/150 = 1.3\%$ of the maximum hardware limit. If 64 kB disk clusters are used, then 500 new clusters/second would mean $500 \times 64 = 32000$ KB/sec = 31.25 MB/sec and so a level of performance of $31.25 / 150 = 21\%$ of the maximum hardware limit. And we did not take in account either the disk spinning times or read/write durations but just the arm move durations.

VMIO FAT32 implementation

The VMIO has a dedicated FAT32 implementation that has been optimized for several modes, one of those being video record and play-back. It is using 64 KB clusters, which is the maximum FAT32 allowed size. The 64 kB FAT32 clusters are always allocated by groups of 16 contiguous clusters that we call "mega-clusters". The "mega-cluster" size is 1MByte (2048 contiguous sectors), so approximately 1 second of Standard Definition Video, or half a second of High Definition video. The file system implementation also provides for a temporary/permanent option at file creation/opening:

- For a temporary file, the File Allocation Table is kept in RAM memory. Therefore, there is no disk access for the FAT update. After the arm has been moved to the target data cylinder, then one have to wait for the disk to rotate until the first destination sector. This may need up to $60/5400 = 11$ msec, we will take 6 msec as a mean value, so we have to wait for a total of $1 + 6 = 7$ msec before the beginning of the writing of the 2048 consecutive sectors and this will take $1/150 = 0.007$ second. So 7 msec of "loss" for 7 msec of effective write. For each HD stream the drive is used $(7 + 7) \times 2 = 28$ msec per second, thus enabling for $1000 / 28 = 35$ simultaneous HD streams recordings, this corresponds to $70 / 150 = 46\%$ of the maximum hardware limit.
- For a permanent record file, the File Allocation Table is not kept in RAM and it is updated on the disk two times per second whatever the number of records is. Each FAT update requires most of the time only 2 disk accesses, read and then write and so we have 4 disks accesses per second. If we ignore the time to read and write the small group of contiguous FAT sectors that have to be updated, this will take $7 \times 4 = 28$ msec. We will then need $2 \times (7 + 7) = 28$ msec per second more for each recorded HD stream. So if we record N HD stream, we will need $28 \times (N + 1)$ msec each second and the effective recording bandwidth will be $N \times 2$ MByte/sec. We are still able to record 34 HD streams, this corresponds to $68 / 150 = 45\%$ of the maximum hardware limit.

Discussion

The fact that software that has been optimized for a specific use gives increased performance is not surprising. But the previous video recording example has been chosen in order to illustrate the consequences of some fundamental choices that have to be made when implementing any file system software and/or designing the file system structure itself. We will firstly give the list of those four points then we will, for each of them, explain what choices have been made for the HyperPanel OS and their consequences:

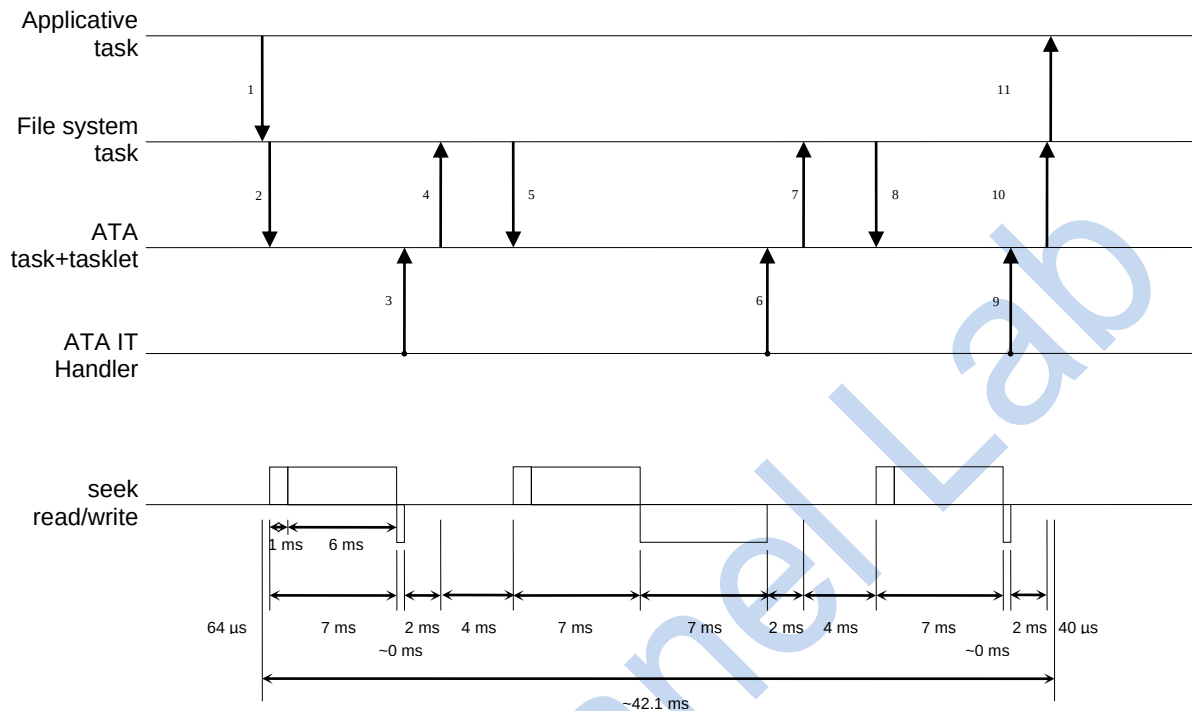
- Decide whether parameters are tied to a whole file system or to each elementary file. For example the "temporary" attribute may be tied to a whole file system, so all the files of that file system are lost in case of power loss, or it can be tied file per file. The first choice will simplify the work of designing the file system and the administration tasks in the case of a multi-user machine whilst the second will simplify the application development and will raise the overall level of performance; partitioning a disk has a heavy performance cost because of increased arm moves.
- Decide whether parameter semantics are "user" or "internal file system structure" oriented. Attributes such as "video", "database", "small files" can be easily understood and so correctly used whilst attributes such as "number of copies of super-blocks", "inodes_size" or "reserved_blocks_percentage" are almost never used or not correctly because they are not easily understood.
- Choose switching methods that are used for file system management. Maximum file reading and writing rates are indeed depending on the file system structure but more heavily on the delay between the end of execution of the drive interrupt handler and the code of the file system management.
- Choose an implementation methodology. The use of classical sequential programming will greatly reduce the complexity of real-time logic that can be safely implemented. For example, correctly managing several opened files using different disk space allocations methods is almost impossible using an "if (flag)" way of programming. A more formal approach using a set of FSM will enable a more reliable implementation of such a complex management.

The VMIO file systems management modules are using parameters tied to files rather than to a whole file system. The logic of the file system manager is more complex because attributes are not the same ones for all the files, because those parameters are dynamically specified (at file creation) rather than statically defined (at file system creation, like with "mkfs"). A correct behavior cannot be reached if the logics are not formalized and if the resulting implementation is not automatically self-validated through very detailed unitary test sets. The benefits of this increased complexity in logic and of the extra work needed to create the unitary tests are best throughput performance and increased system MTBF (Mean Time Before Failure), both being consequences of the drastic reduction of the number of arm moves of the disk drive as described previously.

The use of parameters whose values are selected by application developers have impacts on the overall performance that are frequently under-estimated; we mean that very simple but crucial information that developers will give to the OS may have a great impact. It is very often stated that a file system must not be aware of the nature of the data its files are containing. We claim that the OS should not be aware of the values of the bytes the files contain, that it should always provide the same elementary functions (read/write/seek) for each all files, but that it can use user provided simple information in order to (only) perform optimizations. In our case, as parameters are provided on a file by file basis, they are passed by the user to our "open file" primitive. As with many other OSs, the "open file" primitive has a 32 bits integer parameter that is a set of 32 "mode flags" that will have a "visible effect" from the application point of view, like "temporary" or "non-blocking" modes. The difference is that HyperPanel OS filenames always end with a 3 characters filename extension (like MS/DOS) because we use it to select an optimizing algorithm accordingly. For example, any file name using ".txt" will be considered as being a "small file" whilst ".vid" are "video files". At a first glance, this may seem to be a rather simplistic idea. But the fact that we have, at the OS level, a simple and naïve but **correct** information that enables to **systematically** select an optimization strategy is much more effective than trying to optimize with no knowledge at all or with information that is not systematically provided and correct.

The reason why reducing switching times is crucial for file system management needs some preliminary explanation. Mainly due to the UNIX "pipe" abstraction, it is often considered that buffering data and/or write requests has no influence on maximum data throughput. This is not true at a file system management level and we have to clearly explain the reason why. As soon as a cluster/sector allocation table is not wholly copied in RAM memory, then disks accesses have to be made in order to determine the location of the next cluster/sector that will be used for writing. The user data write cannot be started or put in any waiting queue before the disk read of the allocation table has not be completed.

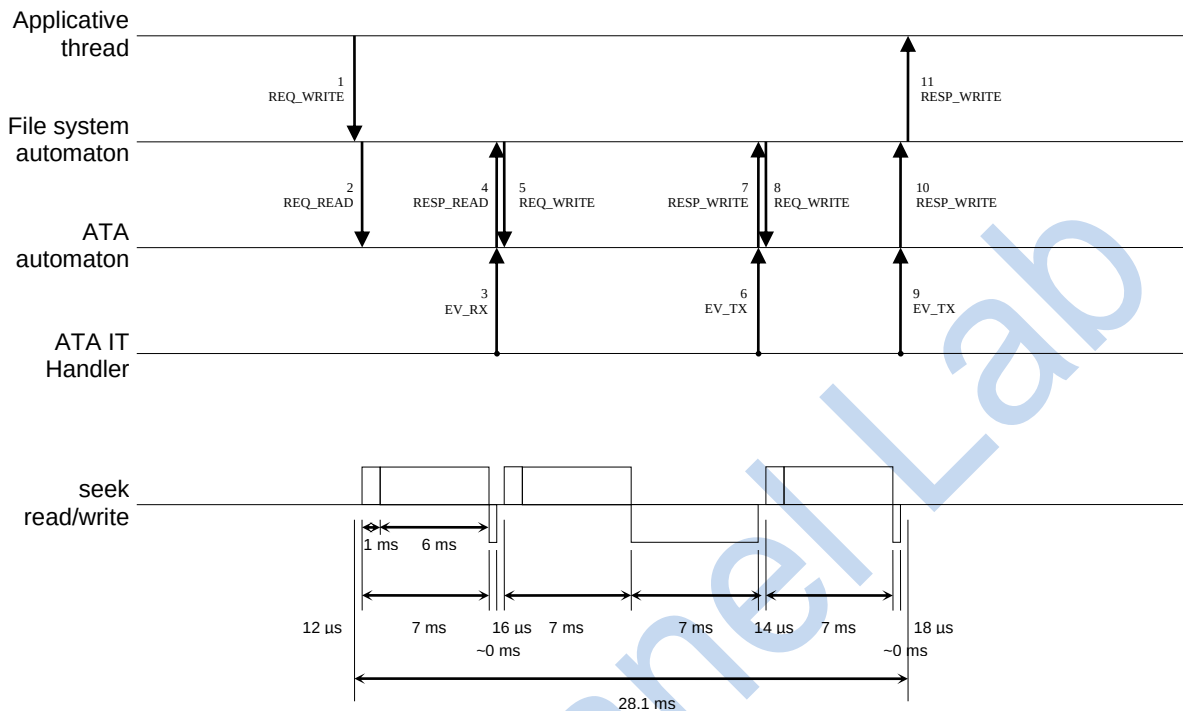
The following diagram shows one elementary file write transaction: read a FAT area (a few contiguous sectors), write one mega-cluster (1MByte of contiguous sectors) and write back the FAT area. We depict the time ordering of the system calls "post_sem", "tasklet_schedule" and "schedule_work" (numbered vertical arrows). Also, the drive's interrupts (dots on the ATA IT Handler line) and drive activity are shown:



As with the I2C previous example, we will suppose that a POST requires 1 μsec of CPU, a GET requires also 1 μsec, and that restarting a task needs 30 μsec, so 32 μsec for those three operations.

- The applicative task issues a POST (1) to the semaphore of the File System task to notify the "write file" request and then blocks itself by issuing a GET on its own semaphore. The File system task is scheduled. This needs $1 + 1 + 30 = 32$ μsec. The File System task issues a POST (2) to the semaphore of the ATA task and then blocks itself with a GET on its semaphore. The ATA is scheduled ($1 + 1 + 30 = 32$ μsec). We spent here $32 + 32 = 64$ μsec of CPU time.
- The ATA task issues two drive commands in order to read a few contiguous sectors of the FAT. The first command is "seek", it will move the arm of the drive (1 msec) then will wait for the disk to rotate (11 msec maximum, average 6 msec). The second command is the reading of the few FAT contiguous sectors, we will here consider as negligible its duration.
- The ATA interrupt handler does a "tasklet_schedule" (3) for the ATA tasklet. We will suppose the ATA tasklet procedure is called 2 msec later by the kernel. This procedure calls "schedule_work" (4) for the File system and its execution begins 4 msec later. A new POST (5) is done to the ATA task, that issues a "seek" ($1 + 6 = 7$ msec) then a write of 2048 contiguous sectors (7 msec).
- The ATA interrupt handler does a "tasklet_schedule" (6) for the ATA tasklet. The ATA tasklet procedure is called 2 msec later by the kernel. This procedure calls "schedule_work" (7) for the File system task that begins execution 4 msec later. A new POST (8) is done to the ATA task, that issues a "seek" ($1 + 6 = 7$ msec) then a write of the FAT region to be updated (negligible).
- The ATA interrupt handler does a "tasklet_schedule" (9) to the ATA tasklet, its procedure is called 2 msec later by the kernel. This one issues a POST (10) to the File system task and the latter one issues a POST (11) to the Applicative task.

The following diagram shows the same elementary file write transaction using the HyperpPanel OS two-container architecture. We depict the time ordering of the events written into the VMOS queue and the VMIO queue (numbered vertical arrows). It also shows the interrupts (dots on the ATA IT Handler line) and drive activity is shown:



As with the I2C previous example, we will suppose that a PUT_EVENT requires 2 μsec of CPU, an event read and then the search into the transition table requires also 4 μsec.

- The applicative thread sends a REQ_WRITE (1) event in the VMIO queue and raises the trap (2 μsec). The VMIO extracts it and calls one C procedure of the File System automaton (4 μsec). This procedure writes a REQ_READ request event (2) in the VMIO queue (2 μsec). The VMIO extract it and calls one C procedure of the ATA automaton (4 μsec). The ATA automaton issues a block of two drive commands: a seek command (1 + 6 = 7 msec) and a read command (negligible duration).
- The disk drive raises an interrupt when the read of the FAT sectors has been completed and the interrupt handler sends an EV_RX event (3) in the VMIO waiting queue and activates the trap. The next machine instruction that is executed after the "return from interrupt" that end the interrupt handler is the trap vector, in our case the trap vector is the VMIO entry point and so immediately (around 4 μsec) a C procedure of the ATA automaton is executed (called as a subroutine by the VMIO, with no context switch at all). This C procedure sends a RESP_READ event (4) in the VMIO waiting queue. Immediately the VMIO extracts it, calls a C procedure of the File System automaton (2 + 4 μsec). This procedure sends a REQ_WRITE event (5) in the VMIO waiting queue. The VMIO extracts this event, calls the ad hoc procedure (2 + 4 μsec) of the ATA automaton and the latter issues a block of two disk drive commands: one seek (1 + 6 = 7 msec) and one write of 2048 sectors (7 msec). Just after the transmission of the command block to the driver (so when their processing begins) the VMIO terminates, the trap is deactivated and CPU goes back to foreground execution, to the application container execution.
- The disk drive raises a second interrupt when the mega-cluster has been written. The interrupt handler sends an EV_TX event (6), the ATA automaton sends a RESP_WRITE event (7), the File System automaton issues a REQ_WRITE (8), the ATA automaton receives it and issues the last command block to write back the modified FAT area. The disk drive will need 1 + 6 = 7 msec to "seek" and then almost nothing to "write".
- The disk drive raises the third interrupt, the interrupt handler sends a EV_TX event (9), the ATA automaton sends a RESP_WRITE (10) and the File System automaton sends a RESP_WRITE (11) to the application container, to the VMOS application thread.

One could ask for the reason why we depicted 2 + 4 msec of scheduling waiting time between the sending of successive disk command blocs on the first diagram and nothing (4 µsec) on the second one. As previously stated, source code reviews of several chipset manufacturers showed that interrupt nesting was **never** used and the tasklet mechanism (one unique queue per CPU) was systematically used, thus deferring all interrupts processing, even with chipsets having more than 40 different interrupt sources and devices with interrupts rates above 10 000/sec.

The idea of deferring interrupt processing comes from very ancient times when CPUs had less than 1 MHz clocks and hardware interrupt nesting did not exist. When 3 or 4 interrupts were occurring in a short time, say less than 1 msec, it was absolutely necessary to queue interrupt processing. Nowadays, the smallest CPUs have 16 MHz clocks and interrupts controllers have 8 to 32 levels of interrupt nesting. Also, chipsets have small quantities of internal SRAM to store the most often executed interrupts handlers. To say things roughly but simply, all modern chipsets are able to process all their interrupts at their maximum rate using 40% or less of the CPU power. In other words, there are no more hardware limitations to justify any kind of interrupt processing deferring. So why operating systems are still proposing mechanisms like the "tasklet" or others, since the necessity to have them has totally disappeared a (very) long time ago? We can find an explanation in the book "Linux device drivers" 3rd edition ISBN-13 978-0-596-00590-0 page xiv: *If you don't want to go deep into the details, you can just skip the most technical sections, and stick to the standard API used by device drivers to seamlessly integrate with the rest of the kernel.* It is true that writing an interrupt handler that can be interrupted by another one is difficult and that the job of correctly handling hardware priorities of interrupt lines is also difficult because it needs an overall understanding of the whole hardware and software system. Use of tasklets and non-nested and deferred interrupt processing is much more "simple" in that respect. But the drawback of this choice is a **tremendous increase of overhead processing**, with the visible result of lower data throughputs. We insist on the fact that tremendous increase of overhead processing does not mean only loss of I/O bandwidth but also **tremendous increase of CPU energy consumption**.

One fundamental reason to split the HyperPanel OS in an application container and an I/O container is to make the best use of the fact that with modern CPU we definitively do not need interrupt deferring at all. In order to be as close as possible to the maximum hardware capabilities two conditions have to be simultaneously fulfilled:

- Shortest as possible switching time
- No delay induced in chains of consecutives events and transitions.

The HyperPanel OS fulfills those two conditions in a way that seems to be the optimum thanks to the use of Finite State Machines (FSM, or automats) inside an I/O container. As a matter of fact we have no latency at all and no switching time from IT handler to ATA driver and from ATA driver to file system.

Third scenario: Access and Display of an HTML Document

Scenario Synopsis

This scenario is the access and display of an HTML file by a digital set-top box. Broadcast video is played on the TV screen and several video recordings are ongoing. The doorbell button is pressed by someone outside, a radio message is sent once, it is received by several receivers, the set-top box being one of those. Upon reception of that message, the set-top box automatically initiates an HTTP GET of a "doorbell" HTML page. This HTML document includes references to "png" image files and to one "js" JavaScript file. The set-top box loads the files. The HTML document is drawn, it includes two frames, one for the TV broadcast video and a second for a JPEG still picture that has been retrieved by the set-top box from an IP camera located near the door. There is a message on the TV screen that tells the end-user that a press on the OK key of the infrared remote will establish a video streaming connection with the IP camera. Suppose the user presses that key. A JavaScript processes that key press, stopping broadcast video playing and then starting IP camera video streaming.

Broadcast Video is played and Recordings are Ongoing

The chip manufacturer (ST Microelectronics) of the chipset of our set-top box (STi7105) was proud to announce that its own Linux distribution (aka. STLinux, more than 400 man-year of work) allowed for one program recording using NAND flash as mass storage. NAND flash devices have better access time than mechanical HDD, but a lower capacity and also a higher cost per GByte. The STLinux is also able to use a HDD to record one program while the box is on stand-by (no video decoding). The "HyperPanel box" that is fitted with the same STi7105 is able to simultaneously play one broadcast video on the TV, record this one on the 5,400 rpm hard disk (Time-shift buffer) and record three other TV programs. The recording of 4 programs and the decoding of one of them means that the following interrupts have to be processed by the CPU:

- MPEG DVB Demultiplexor interrupts. We have four input 42 MBits/s multiplexes (DVB-S). The Demultiplexor is a complex DMA engine that has its own dedicated firmware and CPU. Parsing of video and audio PES packet has to be done in software because the PVR functions needs some data that the firmware does not provide. For each multiplex one TV program is processed and this requires 50 it/sec for video PES parsing, 100 it/sec for audio PES parsing, 64 it/sec for AES encryption (mandatory for PVR, 2 MByte/sec in chunks of 32 KB each), then 60 it/sec for issuing file write requests (2 MB/sec means 10 000 MPEG packets/sec, an interrupt is provided each 178 packets). So we have $50 + 100 + 64 + 60 = 274$ it/sec for each program, so 1 100 it/sec for 4 programs. When a zapping is made to a new multiplex, the EPG data (aka. EIT's) need be acquired. The broadcast cycle is 25 msec and it contains approximately 100 events, so 100 interrupts during 25 msec (4 000 it/sec).
- Video decoder interrupts. The MPEG-2/H.264 reconstructs the frames, 50 frames per second in interlaced mode (1080i), we have 50 interrupts per second corresponding to end of frame decoding.
- Audio decoder. The MPEG-1/AC3/EAC3 audio decoder reconstructs uncompressed audio frame and generates an interrupt at each reconstructed audio frame, so 10 msec for MPEG-2 audio frames. We have 100 interrupts per second for each audio channel, so 200 interrupts per second with stereo.
- Vertical Sync interrupt. Every every 20 msec exactly (50 times per second) the video signal generator issues a VSYNC interrupt and the interrupt handler has to give to the hardware the memory address of the decoded video frame that has to be displayed. This interrupt is crucial because the processing must be completed at all costs 100 µsec at the latest after the VSYNC has been raised.
- I2C HDCP interrupts. Every two seconds a cryptographic handshake is done between the decoder and the TV-set. Due to the relative long size on the I2C bus that is inside the HDMI cable, clock speed is set to 10 KHz, so we have 1 000 bytes end-of-transmissions interrupts per second.

So, we have a continuous rate of $1\,100 + 50 + 200 + 50 = 1\,400$ it/sec, plus two bursts of 1 000 it/sec for the HDCP I2C and 4 000 it/sec for the EIT's (25 msec duration). This is not huge but however this has to be correctly handled, using multi-level nested interrupts because the HDCP I2C interrupts have to be processed before the end of the rather long processing (software parsing) of audio and video PES packets. And there is the VSYNC that has a low rate (50 it/sec) but that has to be processed within 100 µsec.

Here we have a special mention. In order to provide x32 trick mode with all the video recordings, we have to add inside each video file an index for all the I-frame and some other information that is not provided by the hardware, and so each of the four video streams (12-24 MBits/s each, also 1.5-3 Mbytes/s) has to be parsed "on the fly" by the main CPU. This situation led to the last evolution we made some years ago to the VMIO: previously it had only one event waiting queue for all the events wherever they were coming from. Now the VMIO has 3 waiting queue corresponding to 3 different priorities of processing. The lowest priority queue is receiving all the VMIO external events and only these, the highest priority queue receives the VSYNC event only. The 100 µsec maximum delay to process the VSYNC is in any situation absolutely guaranteed.

A Radio Message is received from the Doorbell Button

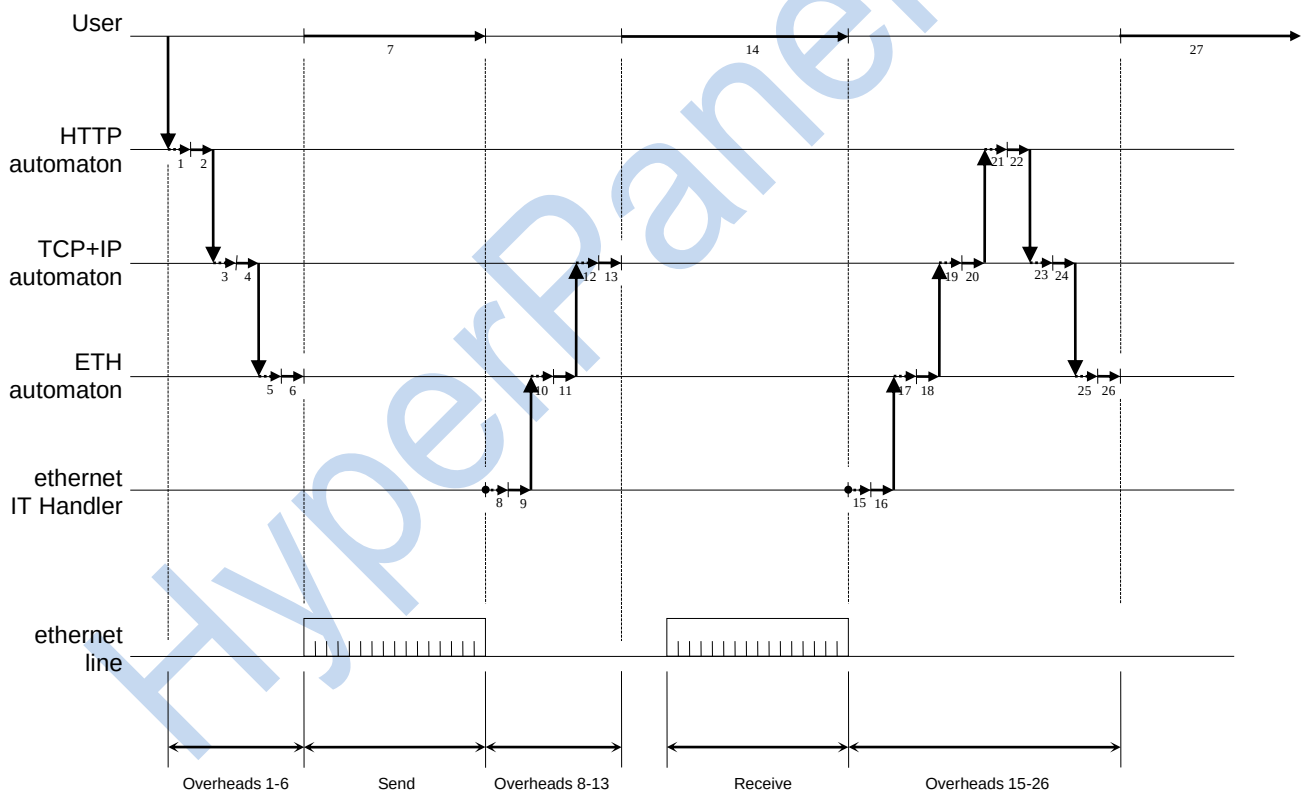
A radio message is sent once by a piezoelectric button that has been pressed. Currently we use a radio receiver that is embedded in a USB dongle. The short radio message is stored in the RAM memory on the dongle before being sent on the USB. There is one USB interrupt. Any OS is able to process that because the USB interrupt can be in this situation deferred for a long time. Using an embedded radio modem whose interface is a serial line at 9,600 bits/sec means that the main CPU receives one interrupt for each character (if FIFO are not used), most OS's are able to process that.

Several HTTP GET Requests are Made

The applicative software of the box downloads a dedicated HTML document when the "button pressed " radio message is received. This implies several HTTP GET requests:

- The HTML file
- One CSS file
- One JavaScript file
- Several PNG image files

The applicative software is located in the application container, both containers being executed by the same and unique CPU, the I/O container having an absolute priority. The VMIO has three waiting queues but here only two of them will be used; the low priority queue will receive the application container requests and the normal priority queue will receive the other events, generated by the handlers and by the VMIO automatons. Each time an interrupt handler writes an event to the normal priority VMIO queue then CPU is given to the VMIT at the next instruction cycle that follows the end of the interrupt handler. Very few CPU instructions are needed to change the stack and there is a jump to the VMIO entry point. The address space is not changed in order to save time. Each time the Application container issues a request to the I/O container a request event (40 bytes) is directly written inside the low priority waiting queue of the VMIO, CPU interrupts being globally disabled during this 40 bytes copy, and the software sets to active a dedicated interrupt line. The consequence of this line becoming active, whose handler is the entry point of the VMIT, is immediate: the next executed CPU instruction is the VMIT first instruction; stack is changed and a jump to the VMIO entry point is done. Now, let's detail the steps needed to get a file using the HTTP protocol:



We have four activities called "user", "HTTP" (service), "TCP+IP" (protocol) and "ETH" (driver). The three "HTTP", "TCP+IP" and "ETH" are FSM (automatons) and are located inside the I/O container, they are run by the VMIO which is an automaton engine. We have an Ethernet interrupt handler that does not defer end-of-transmission interrupt processing. The vertical arrows are calls to the "put event" OS function, each arrow also corresponds to the copy of one event structure (40 bytes) in one VMIO queue (slow or normal). On the "ethernet line" we have depicted I/O activities. The horizontal arrows are switching durations and useful processing durations.

Here is a step by step detailed description of what is happening:

- **Overheads 1-6.** The applicative software, so the Applicative container, calls a VMOS API C procedure in order to request the HTTP GET of the HTML file. This procedure allocates a shared memory buffer, copies the URL string and other needed parameters in the buffer, initialize a request event structure that will contain the address of the parameters buffer. Then CPU interrupts are globally disabled, the 40 bytes event structure is copied into the low priority waiting queue of the VMIO, the software controlled interrupt line is set to the active state. This interrupt is not taken in account immediately since interrupts are globally disabled. At that point we are executing code of the API C procedure that is part of the application container. This procedure now enables the CPU global interrupt. As the software controlled interrupt line is active, the next executed CPU instruction is the entry point of the VMIT. The current stack pointer and some other registers are saved and a dedicated VMIO stack is installed (it is empty) and then a jump to the VMIO entry point is done. The VMIO extracts from the low priority queue the event that has just been written into. The event contains the reference of the HTTP automaton and so one transition of this automaton has to be fired. The VMIO extracts from the HTTP transition table the address of the C procedure which is the processing to be done and the VMIO loop calls this procedure as a subroutine (2), so no context switch is done. The HTTP and TCP are not agglomerated so HTTP sends an event to the TCP automaton in order to request a TCP connection with the remote HTTP server. At the next VMIO loop the event is extracted and a TCP automaton procedure (4) is executed. As TCP and IP are agglomerated, we obtain a buffer containing an IP packet which wraps the TCP connection request message and an event is sent to the ETH driver in order to send the IP packet. At the next VMIO loop this event is extracted and an ETH automaton procedure is executed (6), this one gives the frame to the Ethernet hardware and transmission begins. All the VMIO queues are now empty and so the VMIO acknowledges the software interrupt. The VMIT restores the VMOS stack and terminates the IRQ processing. The next executed instruction is one of the VMOS API procedure. Please note the Ethernet hardware has not finished transmitting the Ethernet frame, all I/O requests are non-blocking.
- **Overheads 8-13.** When the Ethernet hardware has completed the frame transmission it raises an interrupt. The VMIT is entered, VMOS stack and registers are saved, an interrupt stack is installed and the Ethernet handler begins execution (9). It writes an end-of-transmission notification event in the normal VMIO queue and sets the software controlled interrupt line to its active state (one register write). The VMIT restores the register and the stack then the IRQ is acknowledged. But at the next interrupt controller clock cycle the software raised interrupt line is taken in account and the VMIT is immediately entered again, and the just re-installed stack is saved again and the VMIO stack is installed. The VMIO is entered and it extracts the event that has just been written by the Ethernet IT handler, then it calls the appropriate ETH driver processing procedure (11). This procedure writes another termination event to the TCP+IP automaton. Immediately at the next loop the VMIO extracts this event and calls the appropriate TCP+IP procedure (13) which will free the transmitted memory buffer. All the VMIO queues are now empty and so the VMIO acknowledges the software interrupt. The VMIT restores the VMOS stack and terminates the IRQ processing. The next executed instruction is one of the VMOS code (14).
- **Overheads 15-26.** After an undefined amount of time, the remote HTTP server answer is received, it is an acknowledgement of our TCP connection request. We have an interrupt at the end of the incoming Ethernet frame. The VMIT is entered, VMOS stack and registers are saved, an interrupt stack is installed and the Ethernet handler begins execution (16). It puts an end-of-reception notification event in the normal VMIO queue and sets to active state the software controlled interrupt line. The VMIT restores the VMOS stack and registers and acknowledges the IRQ. But at the very next interrupt controller clock cycle the active state of the software controlled line is detected and the VMIT is entered again. The VMOS stack and registers are saved and the VMIO stack is installed and a jump to the VMIO entry point is done. The VMIO extracts the just written event and calls the appropriate ETH driver automaton procedure (18) which puts a new notification event targeted to the TCP+IP automaton. The VMIO extracts the just written event and calls the appropriate TCP+IP protocol automaton procedure (20) which puts a new notification event targeted to the HTTP automaton. The VMIO extracts the just written event and calls the appropriate HTTP service automaton procedure (22) which allocates a memory buffer, writes into it the text of the HTTP GET request and then puts in the normal VMIO queue a write request event targeted to the TCP+IP automaton. The VMIO extracts this just written event and calls the appropriate TCP+IP protocol automaton procedure (24) which puts in the normal VMIO queue a write request event to the ETH driver. The VMIO extracts this event and calls the appropriate ETH driver automaton procedure (26) that "gives" the buffer address to the Ethernet controller hardware. Transmission begins. All the VMIO queues are now empty and so the VMIO acknowledges the software interrupt. The VMIT restores the VMOS stack and terminates the IRQ processing. The next executed instruction is in the VMOS code (27).

We will now evaluate roughly the duration of each elementary operation. We are still assuming that the CPU executes 250 instructions per μsec :

- (1) The synchronization primitive used by the application container to send a request to the I/O container is no more than a copy of 40 bytes and then a write of one hardware register in order to set the controlled by software interrupt line state to active. This requires 500 CPU instructions, so 2 μsec . Then we enter the VMIT, save the stack pointer and other registers, install the VMIO stack, jump to the VMIO; it extracts the event then gets a C procedure from a transition table and calls that procedure. We will not use more than 1000 CPU instructions, so 4 μsec .
- (3, 5, 12, 19, 21, 23, 25) A VMIO automaton writes 40 bytes in the normal waiting queue, the VMIO loop detects that all the queues are not empty, VMIO extracts the event and calls the appropriate automaton C procedure. No stack change has occurred. We will not use more than 500 instructions, so 2 μsec .
- (8, 15) A hardware interrupt is raised. The hardware has a negligible response time under 5 clock cycles (finish current instruction, hardware comparison of IT lines priorities, set CPU IT line active). The VMIT is entered, the stack pointer and other registers are saved, one interrupt stack is installed (one per interrupt level because of possible nested interrupts) and a jump to the interrupt handler is made. We will not use more than 250 CPU instructions, so 1 μsec .
- (9, 16) Handler IT processing. We will count 500 instructions so 2 μsec because the biggest instruction number is in fact the event deposit, The IT handler writes 40 bytes in the normal waiting queue, activates the controlled by software interrupt line.
- (10, 17) The VMIO returns to the VMIT, the VMIT restores the registers and acknowledges the interrupt. Immediately a new interrupt occurs, registers are saved, the VMIO stack is installed, a jump to the VMIO is done, the event is extracted from the normal queue, a C procedure address is extracted from a transition table and it is called as a subroutine by the VMIO (4 μsec).
- (2, 4, 13, 18, 22). We will suppose that (2, 4, 18) request a dynamic memory allocation and that (13, 22) do a free (13 frees allocation from 4 and 22 from 18, the corresponding free of 2 is not done in those first steps). Those operations are "long" and we will consider we need 5 000 instructions so 20 μsec .
- (6, 11, 20, 24, 26) Other automaton procedures, which do not allocate memory, will use 5 μsec of CPU.

Step numb.	Step	μsec
1	User to HTTP service switching time	6
2	HTTP service processing	20
3	HTTP service to TCP protocol switching time	2
4	TCP protocol processing	20
5	TCP protocol to Ethernet driver switching time	2
6	Ethernet driver processing	5
TOTAL		55
8	Interrupt latency	1
9	Ethernet IT handler processing	2
10	IT handler to Ethernet driver switching time	4
11	Ethernet driver processing	5
12	Ethernet driver to TCP protocol switching time	2
13	TCP protocol processing	20
TOTAL		34
15	Interrupt latency	1
16	Ethernet IT handler processing	2
17	IT handler to Ethernet driver switching time	4
18	Ethernet driver processing	20
19	Ethernet driver to TCP protocol switching time	2
20	TCP protocol processing	5
21	TCP protocol to http service switching time	2
22	HTTP service processing	20
23	HTTP service to TCP protocol switching time	2
24	TCP protocol processing	5
25	TCP protocol to ethernet driver switching time	2
26	Ethernet driver processing	5
TOTAL		70

Then the remote server sends the data file. If we assume that the Ethernet MTU is 1 500 bytes, so 12 000 bits, and that we are using a 20 Mbits/sec download rate ADSL line, one frame is received each 600 μ sec (around 1600 frames/sec). The processing that is done is similar to the second pattern of the previous sample, so around 35 μ sec of CPU time. It has to be noted that TCP has an anticipation window, so a frame can be sent before the previous one has been acknowledged, so the 35 μ sec processing time can be overlapped with reception, incoming frames are not blocked by the flow control. In any way, the ADSL line may be used at 100% of its bandwidth (if the remote server is able to sustain that rate). CPU use is around $35/600 = 6\%$.

The first received file contains HTML data. One event is sent by the VMIO to the VMOS for each group of 20 received frames (a link of 20 shared memory buffers), so $1600 / 20 = 80$ events per second. The parsing of the HTML data is done for each received link of buffers. Each time an URL is found (IMG SRC=...) the Webkit HTML parser returns the found URLs which are given to the HTTP service, that is able to handle multiple downloads simultaneously.

The HTML Document is displayed on the TV Screen

The computations that are done by the Webkit code to determine sizes and positions of the elementary graphical objects (strings and bitmaps) do not require I/O's and here only CPU power matters. The OS has no influence at all on that point.

The hardware of a set-top box chipset graphic subsystem provides for 5 graphic layers that are fully independent. This means that the video plane can be used by the video decoding driver which is located inside the I/O container while the on-screen display plane is used from the application container. The OSD comes with a dedicated processor aka. "2D blitter", whose most important functions are the fractional resize, copy and filling of a rectangle of pixels. The WebKit defines its own graphical API and a wrapper has to be provided in order to map Webkit graphical calls to the OS graphical calls. The HyperPanel OS provides a direct to hardware graphical set of primitive. The drawing of a Unicode string of characters, the API "drawstring" procedure does not contain any system call and makes direct use of the 2D blitter. It has to be noted that all the drawing procedures are putting requests in a 2D blitter waiting queue that is read when the blitter interrupt is raised, thus allowing for parallel work of the CPU and the 2D blitter. The blitter interrupt handler and the blitter waiting queue are logically parts of the application container.

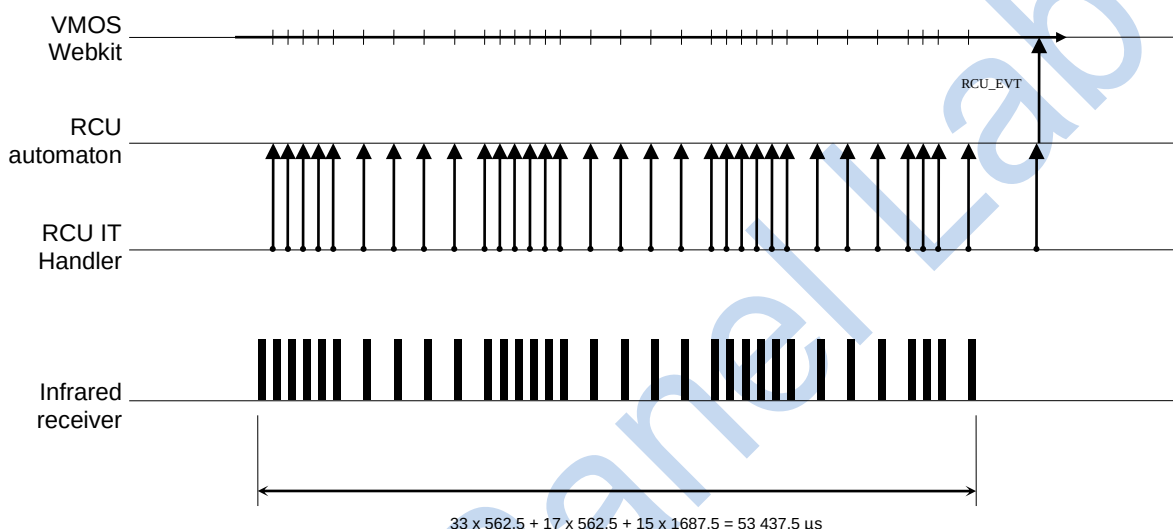
Video is resized and a still camera picture is displayed

The HTML file can embed dedicated HTML 5 tags, or HbbTV tags, which define the video rectangle position and size. Those tags are processed by a Webkit dedicated plugin. This plugin is provided by the HyperPanel OS and is a part of the application container. The video plane is handled by the video decoding driver, which is part of the VMIO container. The plugin sends an event request that is immediately processed by the VMIO because after writing the 40 bytes event in the low priority VMIO waiting queue, the controlled by software interrupt line is set to its active state. The proper C procedure of the video decoding driver is executed and video registers containing position and size are updated. Those will be taken in account by the hardware at next VSYNC interrupt.

A JavaScript is executed by the Webkit. This script requests to HTTP the reading (HTTP GET) of a JPEG picture file. An event is put in the low priority VMIO waiting queue. The VMIO handles the JPEG file download and sends a notification event to the application container. The Webkit asks for the JPEG decoding, in that case this decoding is done with a software function provided by the HyperPanel OS.

The OK key of the Remote Control is Pressed

The user presses the OK key of the Infrared Remote Controller. We will assume NEC 2000 IR protocol is used by the RCU. Each time a key is pressed, a 32 bits message is sent. This message is made of 33 light pulses of 562.5 µsec each. The duration between two adjacent light pulses has two possible values, 562.5 µsec for a logical 0, and 1687.5 µsec for a logical 1. The hardware generates an interrupt after each symbol end (a light burst followed by the OFF period) and two registers are respectively indicating the ON and OFF value in µsec. For the last symbol the OFF value is equal to 0xFFFF (time-out). There is no hardware FIFO for those two registers, so the priority of this interrupt is rather high in order to interrupt other interrupt handlers if needed. When the 32 bits have been received and checked (they contain redundant data) the VMIO sends to the application container a notification event containing the keycode.



The minimum duration between two consecutive interrupts is the one of a logical 0, so $562.5 + 562.5 = 1125\text{ µsec}$. This is a rather "long" time but however some chip manufacturers have chosen to add a 64 positions FIFO because they have to provide their own Linux distribution. We can suppose they are using the Linux "tasklet" way of handling interrupts.

The top line of the previous diagram perfectly illustrates the fact that for the foreground task, here the Webkit, extremely tiny "pieces of time" are stolen. We can generalize that way of operation by saying that the IRQ driven OS architecture "grabs" very small time's quanta very often instead of biggest quanta less often as any RTOS would do. If the user does not release the key, a repeat pattern is sent every 110 milli-second. As a result of it, the same Webkit UI will be perfectly smooth with the HyperPanel OS and choppy with any RTOS or Linux. Therefore, some chipset manufacturers are providing a hardware NEC 2000 IR decoder in order to fix that issue, but indeed the hardware decoder is not able to recover noisy messages like software is able of. Unfortunately, the IR band is extremely noisy indoors and so IR range may be reduced by two.

Streaming from the IP Camera is started

Upon reception of the OK keycode, some JavaScript is executed by the JavaScript interpreter of the Webkit. This script makes use of one plugin function that is dedicated to broadcast video control, here to request to stop the broadcasted video playback. This function sends an event from the Webkit to the VMOS, both the Webkit and the VMOS being located in the application container; those are two tasks under the control of the VMK. This event starts a PanTalk script that requests all the elementary actions that are needed to stop broadcast video. The script sends several request events to the I/O container. The PanTalk script then sends a response event to the Webkit task. Then the JavaScript issues a HTTP GET using the IP camera URL. To do so, the Webkit plugin sends a request event to the HTTP automaton located in the I/O container. As with the HTTP GET processing of an HTML file previously described, the compressed video data buffers are directly piped inside the VMIO to the video decoder driver automaton, so the data stream flows smoothly inside the I/O container **without going through the application container and with no context switch**, thus greatly reducing CPU use.

Summary

One role of an OS is to allocate CPU to user computational activities. Modern devices have new needs regarding Input/Outputs, because of tremendous increases of both interrupts rates and maximum data rates. We concentrated here on the performance of the more and more complex part of an OS which is handling numerous and simultaneous Input/Outputs. The two first scenarios (Serial communication controller use, Hard Disk Management for video recording) allowed us to point out the most crucial points when it comes to maximize I/O throughputs:

- Reduce interrupt latencies
- Nest interrupts processing
- Reduce scheduling latencies
- Reduce the cost of each activity switch and the number of those
- Reduce the number of system calls that are made
- Reduce CPU time wasting
- Reduce data piping overhead, do not copy or move it across address spaces
- When data has to be copied, make maximum use of DMA engines
- Do not use virtual memory
- Do not use semaphores and spin-locks

A typical set-top box drivers Linux implementation shows poor results on all of those points and this has two consequences: low sustained throughputs and choppy behavior as seen per the end-user. In fact, the analysis we made with the two stacks "zapping/front-end/I2C protocol" and "applicative/file system/ATA" could be done in the same way with other I/O software stacks; we will always find those crucial points. Other crucial points that have to be taken in account in order to increase I/O performance are:

- Define APIs that will reduce I/O requests e.g. I/O list and not only single I/O requests
- Define several I/O API's, e.g. for remote use (app to IO container) and local use (IO to IO container)
- Code optimization has not to be limited to the kernel but also to drivers/protocols/services
- Proper formalization methodology enables for less code and also for safe implementation of more complex optimization strategies, as we did with a fast FAT32 file system manager.

Using those points, we can shortly explain performance ranking of several OS architectures that do not rely on the two-container architecture:

- Linux or Windows are qualified as being "non real-time". A "real-time" qualified OS has to provide at least one of the two following kind of guarantees. The first one is that for some dedicated events an absolute response time has to be provided, e.g. the handling of the brakes of a car needs such a kind of a guarantee. The second one has to be provided "all the time but for a set of limited and well-known situations". A banking automat has to give the cash within a defined amount of time. But in some situations, for example when the system is uploading its data, it is acceptable that the system response time is higher. Such a behavior is acceptable for a banking automat and not for the brakes of a car. Linux and Windows do not provide such kind of guarantees and as a consequence it is not possible to implement high data throughputs.
- Interrupt Abstraction, RT Linux being one implementation of this. It relies on the very incorrect assumption that *"...It turns out that in a great many applications, only a small part of the system truly requires hard real-time determinism The non real-time stuff, such as graphic, file management and networking, which Linux already does very well, is handled by Linux"* (Linux for Embedded and Real-time Application, p 172). So RT Linux mainly only reduces interrupt latencies and as stated here file management and protocols handling are still "inside" Linux. RT Linux is not used by chipset manufacturer that are using Linux, because it does not bring a lot of value.
- Conventional RTOS's enable better results because using priorities correctly all main latencies can be reduced: interrupt and scheduling latencies. But they all suffer from high switching costs, especially when a "nano-kernel" is used. They also suffer from too basic I/O APIs. But the main issue is that their functions (protocols, file systems and services) have not been designed using a formal methodology that would reduce bugs and they don't have thousands of people to correct those bugs.
- Dual OS's solutions. They require at least two chipsets, one to run a "high level OS", typically Linux, and a second to run a "low level OS", typically a RTOS. Such an asymmetric multiprocessor architecture may be used within telecom machines. But here the issue is that the RTOS, despite its name, is relatively "slow" compared to a FSM only architecture, the scheduling cost being still relatively high.

The HyperPanel OS two-container architecture locates the services, protocols and file systems in the same address space as the drivers and the interrupt handlers. This provides for a maximum of CPU time savings. Also, the I/O container is only made of interrupt handlers and FSM. As a result, all of the crucial points given before have been addressed. This is the reason why the HyperPanel OS has an unrivaled level of performance. As depicted by the three scenarios we described earlier, one can see that **the overall scheduling of activities is far better** because all activities relative to the physical I/O's (drivers) but also depending on those (protocols, file systems and services) are scheduled with an almost zero delay, thus creating fast and seamless flows of data that never stop, that do not have to cross any kind of frontiers (no address space changes, no data copy); due to flow control mechanisms, the hardware cannot be used at its maximum if any layer of any stack is introducing erratic or systematic delays.

The use of FSM has also induced specific benefits that have to be outlined. The time has to be explicitly taken in account and thus becomes a central concern. A file system manager is no more reduced to a kind of "structure parser" that is responsible of only maintaining a structure, but is also a time dependent machine, and implementation of more complex file handling strategies becomes possible. The fact that many disk accesses cannot be started before completion of access to "management data" is clearly seen when looking at a transition table. By the way, this is the reason why the widely spread idea that General Purpose Operating Systems (aka. GPOS) are able to maximize "overall disk performance" is incorrect. Also "nasty" provided systems mechanisms (semaphores and spin-locks) are not needed any more, thus reducing switching activities and useless waste of CPU time. And, last but not least, use of FSM enables for both reduced code size and less bugs per kilobyte of code, because of full coverage automatic tests, thus **massively reducing the total number of bugs**.

We would like to stress here that the best gains we got are not relying upon code optimization but architecture optimization; using events rather than semaphores, reducing system activities and interactions and providing more sophisticated APIs. The gains we obtained on the CPU consumption use are much higher than if we "only" had working on optimizing C coding. During the three last decades, RAM and CPU performances have been increasing in such a way that deferring interrupt processing became **completely** useless. We mean that not only the sole processing of the interrupt by itself does not need to be deferred, but also the processing of the drivers, and the protocols or file system, and the services that are stacked on top. The HyperPanel OS architecture has taken in account the huge progresses made by the hardware and also the fact that modern CE products will stress the OS with an ever-growing rate of interrupts.

We will finish by identifying the two main benefits of the two-container architecture for the application container and its contents, for example a Web Browser (Webkit).

- CPU use needed to handle the services, the file systems, the protocol drivers is greatly reduced thanks to the architectural choices which are previously detailed. With the Serial Controller scenario we evaluated 60 μ sec instead of 1 800 μ sec. Other estimations and measures made by NXP chipset manufacturer at the CPU level gave a ratio of 20 between HyperPanel OS time and Linux OS time (but the Linux did not have PVR). So when the Hyperpanel OS needs 4% of the CPU time, the Linux needs 80%. We were told that among those 80%, more than half of it was CPU copy of data. So the benefit for the application is obvious. In a general way, the fact that all the protocols and file system have been put "near" the hardware is also enabling for an easy use of all the hardware accelerators available, for example the TLS can make use of the AES hardware ciphering/deciphering DMA engine.
- In addition to being less CPU consuming, the two-container architecture has a nice pattern of CPU use. The I/O container is entered when hardware interrupts are raised. So the I/O container is entered very frequently for very short whiles. For any graphical UI software this is a nice pattern because no graphical order can be deferred more than the maximum duration spent in the I/O container, which is normally around 1 msec. The drawings are thus always very smooth.

Conclusion

We have tried here to explain what the HyperPanel OS can achieve, using "real-word" scenarios that we described in details. The "paper results" we provide here have been obtained making some elisions for the sake of clarity, but we also tried to strengthen those by providing measurements results when we had them. The unrivaled level of performance we are claiming for are demonstrated with existing CE products and the "all-in-one" Media Home Gateway which we deliberately designed with a widespread low-end chipset.

All the efforts we made to solve the Input/Output challenges raised by modern consumer electronics chipsets have been fruitful and the HyperPanel OS is able to reach I/O performances that are extremely near the hardware maximum bandwidth. This result has been demonstrated with many set-top-box products which have the characteristic to simultaneously handle sustained high data rates and also bursts of I/O's.

Hardware Referenced in this Paper

All-in-One

The HyperPanel box we created some years ago (2014) in order to demonstrate the performance of the HyperPanel OS has been named "all-in-one", because it includes all the possible devices one can find in a Media Home Gateway. The case has been opened in order to show those devices. The chipset is one STi7105 running at 440 MHz. We have 256 Mbytes of DDR-2 RAM. The first daughter board on the left features a 10/100 MBits/sec Ethernet switch and a WiFi Access point (Realtek 3530). The second daughter board on the right is a 4 Tuner DVB-S/S2 board with two antenna F connectors. It has a 500 GByte 5 400 rpm 2.5" hard disk drive. We also have a 4 Tuner Terrestrial daughter board that is DVB-T and DVB-T2 compatible (not shown here).



The front panel features one USB connector and the infrared receiver. The rear panel has 2 F connectors (2 satellite dishes), 3 Ethernet RJ45 ports, 2 audio CINCH's, 1 S/PDIF socket, 1 composite video CINCH, 1 USB, 1 HDMI and a power supply connector.

+Le Cube HD Wifi

The most advanced Linux set-top box using a STi7105 chipset we found is "+Le Cube HD Wifi". The operating system is Linux and the chipset manufacturer provides a distribution named STLinux, which includes all the drivers that have been developed by ST Microelectronics.



The official name of that set-top box is "+Le Cube HD Wifi". It has a STi7105 chipset and 512 MBytes of DDR-2 RAM. Its PCB has been designed to have four tuners but only two tuner chips are fitted probably because its STLinux software cannot handle such data throughputs. This box can have an external 320 GByte hard disk that is not shown on that picture. It has an internal 4 GByte NAND flash which is dedicated to video recording. When broadcast video is played, video recording is done on the NAND flash. On stand-by, no video decoding occurs and this enables to record on the optional hard disk drive.